
VERIFYING PROPORTIONALITY IN PARTICIPATORY BUDGETING

Master's Thesis, Computer Science

Mathias Christensen

Department of Computer Science
Aarhus University
202105853

Svend Christensen

Department of Computer Science
Aarhus University
202106825

Peter Rabjerg

Department of Computer Science
Aarhus University
202106408

June 2026

Advisor

Ioannis Caragiannis



AARHUS
UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

ABSTRACT

Participatory budgeting is a democratic process in which citizens decide how to allocate a public budget among proposed projects. A central challenge in this setting is to design and evaluate voting rules that are not only efficient, but also fair to cohesive minority groups. This thesis studies proportionality in approval-based participatory budgeting, with a particular focus on Extended Justified Representation (EJR) and its relaxations, EJR-x and EJR-1. We examine these axioms across different utility functions, particularly cardinality- and cost-based utility, and discuss how the choice of utility function affects whether an outcome should be considered proportional.

From a theoretical perspective, we study the computational difficulty of verifying proportionality guarantees. Although EJR verification is hard in the worst case, we present an algorithm that exploits the structure of real-world elections to verify EJR efficiently in practice. The algorithm is also adapted to EJR-x and EJR-1 and extended with a violation-degree measure that quantifies the severity of an outcome's failure of proportionality.

We evaluate the approach on 1050 real-world approval elections from Pabulib, comparing several voting rules and completion variants, including Greedy, the Method of Equal Shares, and Sequential Phragmén. The experiments show that proportional rules perform strongly in terms of EJR satisfaction, while purely greedy cost-based selection often achieves high utility at the expense of proportionality. Completed proportional rules offer the best practical trade-off, combining strong proportionality performance with better budget usage and welfare. Overall, the results show that proportionality verification is not only theoretically meaningful, but also useful as a practical diagnostic tool for real participatory budgeting elections.

Contents

1	Introduction	1
2	Preliminaries	4
2.1	The Standard PB Model	4
2.2	Voting	4
2.3	Utility Functions	5
3	Participatory Budgeting Axioms	7
3.1	Priceability	7
3.2	Exhaustiveness	8
4	Proportionality	11
4.1	Cohesive Groups	11
4.2	Multi-Winner Elections	12
4.3	Extended Justified Representation	12
4.4	Proportional Justified Representation	20
4.5	Full Justified Representation	22
4.6	Summary of Proportionality	23
5	Voting Rules	24
5.1	Utilitarian Welfare Rules	24
5.2	The Method of Equal Shares	32
5.3	The Sequential Phragmén Rule	39
5.4	Completion Rules	42
5.5	Summary of Axiomatic Guarantees	44
6	Verifying Proportionality Axioms	45
6.1	Verifying Proportionality is Hard	45
6.2	A Naive Verification Algorithm	47
6.3	The Apriori Algorithm	48
6.4	An Efficient EJR Verification Algorithm	48

6.5	Proof of Correctness	52
6.6	Algorithm Implementation Complexity	53
6.7	Adapting the Algorithm to EJR-x & EJR-1	53
6.8	Parameterized Complexity of EJR Verification	54
6.9	Violation Degree	58
6.10	Why PJR and FJR Remain Infeasible	60
7	Experiments	63
7.1	Pabulib & Data	63
7.2	Evaluated Rules and Criteria	64
7.3	EJR Verification Results	65
7.4	Violation Degree	68
7.5	Theoretical Runtime	72
7.6	General Outcome Statistics	73
8	Discussion	76
8.1	Verifying EJR is W[1]-hard and yet doable in practice	76
8.2	Violation degrees	77
8.3	The Greedy Voting Rule in Practice	77
8.4	The Role of Completion Rules	78
8.5	Why EJR-x and EJR-1 Seem Indistinguishable in Practice	79
8.6	Practical Takeaways	80
9	Future Work	81
10	Conclusion	83
	References	85
A	Appendix	88
A.1	Alternative Way to Find Unsatisfied Voters	88
A.2	Algorithms	88
A.3	Experiment Data	88

1 Introduction

Participatory budgeting (PB) is a democratic process through which citizens directly decide how a portion of a public budget is allocated among a set of proposed projects. Originating in Porto Alegre, Brazil in 1989, the practice has since spread to thousands of municipalities worldwide, covering cities across Europe, the Americas, and Asia ([Aziz and Shah, 2020](#)). In a typical participatory budgeting process, residents submit project proposals like a new playground, a bicycle path, a community center. And then vote on which projects should receive funding within a fixed budget. The appeal of PB lies not only in the practical allocation it produces, but also in the democratic legitimacy it confers: by giving every citizen a direct voice in public spending decisions, it fosters civic engagement and public trust.

From the perspective of computational social choice, participatory budgeting raises fundamental questions about how to aggregate individual preferences into a collective decision that is both efficient and fair. The goal of *efficiency* pushes towards maximizing total voter utility, while the goal of *fairness* demands that no group of voters be systematically ignored in favor of the majority. These two objectives are often in tension. A purely utilitarian rule, for instance, may consistently channel the entire budget towards the preferences of a large majority, leaving minority groups without representation regardless of how strongly they care about their projects.

The central fairness concept studied in this thesis is *proportionality*: the principle that a group of voters comprising a t fraction of the population should collectively see at least a t fraction of the budget spent on projects they support. This intuition has been formalized into *justified representation* axioms. The central axiom studied in this thesis is *Extended Justified Representation* (EJR), which was originally proposed for approval-based committee elections by [Aziz et al. \(2017\)](#) and later adapted to the general participatory budgeting setting by [Peters et al. \(2021\)](#). The strongest is *Full Justified Representation* (FJR), which demands that in any sufficiently large and cohesive group of voters, at least one member is fully satisfied. The weakest variant, *Proportional Justified Representation* (PJR), requires only that the cohesive group as a whole is collectively represented. We also study two natural relaxations of EJR: *EJR- x* and *EJR- I* , introduced by [Brill et al. \(2023\)](#).

A key observation is that proportionality is not absolute: it depends on how voter utility is measured. In approval-based PB, voters simply indicate which projects they support. One can then measure the utility a voter derives from an outcome either by counting the number of their approved projects that were funded (*cardinality-based* utility, $\mu^\#$) or by summing the costs of those projects (*cost-based* utility, μ^c). These two measures can disagree significantly, and a voting rule may satisfy EJR with respect to one while violating it with respect to the other.

Alongside proportionality, we also examine two structural properties of outcomes. *Exhaustiveness* requires that no additional project can be added to the outcome without exceeding the budget,

a minimal soundness condition saying that the budget is not needlessly wasted. *Priceability*, introduced by [Peters and Skowron \(2020\)](#), requires that the outcome can be interpreted as arising from a fair market process where each voter receives an equal share of the budget to spend on projects they approve.

On the practical side, we study several voting rules. *Utilitarian welfare maximization* directly maximizes the sum of voter utilities, at the cost of proportionality guarantees. Practical deployments most commonly employ the *Greedy* rule, which selects projects iteratively in order of utility per unit cost. More recently, the *Method of Equal Shares* (MES), introduced by [Peters and Skowron \(2020\)](#), has attracted considerable attention as a rule with strong theoretical proportionality guarantees: it satisfies EJR- x and is priceable. *Sequential Phragmén* is another proportional rule with related guarantees.

A natural question is whether these theoretical guarantees actually matter in practice, and what happens with rules that lack them. This requires being able to verify whether a given outcome satisfies EJR. Unfortunately, this turns out to be computationally hard: checking EJR is coNP-complete in general. A central contribution of this thesis is a practical algorithm for EJR verification that, despite this worst-case intractability, runs efficiently on real-world elections. The key insight is that real-world PB instances exhibit far fewer cohesive groups than the worst case, allowing aggressive pruning to reduce the effective search space dramatically.

Armed with this algorithm, we conduct an empirical study on 1050 real-world approval elections from the Pabulib library ([Faliszewski et al., 2023](#)), evaluating seven voting rules against EJR and its relaxations under both cardinality- and cost-based utility.

Contributions. The main contributions of this thesis are:

1. A formal treatment of proportionality axioms for approval-based PB – EJR, EJR- x , EJR-1, PJR, and FJR – including completeness results establishing that verifying each is coNP-complete.
2. A practical algorithm for verifying EJR (and its relaxations) that exploits structural properties of real-world elections to achieve feasible runtimes despite theoretical intractability.
3. An empirical study on 1050 real PB elections comparing seven voting rules on proportionality satisfaction, violation degree, budget usage, and welfare, under both cardinality- and cost-based utility.

The remainder of this thesis is organized as follows. Section 2 introduces the formal PB model and the utility functions used throughout. Section 3 defines the axiomatic framework, covering priceability, exhaustiveness, and the proportionality hierarchy. Section 4 develops the proportionality axioms in detail, proving satisfiability and studying the relationships between them. Section 5

presents the voting rules studied in this thesis and their axiomatic properties. Section 6 establishes the computational hardness of proportionality verification and presents our algorithm. Section 7 reports the empirical results on Pabulib data. Section 8 discusses the findings in depth. Section 9 gives ideas for possible extensions to our work and participatory budgeting as a whole. Finally, Section 10 summarizes the main findings and concludes the thesis.

2 Preliminaries

We begin by introducing the standard model of Participatory Budgeting (PB) that will be used throughout this work, following the classical formulation of approval-based and utility-based PB from the computational social choice literature (Peters et al., 2021).

2.1 The Standard PB Model

A PB *instance* (also called an *election*) is a tuple $I = \langle N, P, A, b, c \rangle$, where $N = \{1, 2, \dots, n\}$ is a finite set of voters, $P = \{p_1, \dots, p_m\}$ is a finite set of projects, $b \in \mathbb{N}$ is the total available budget, and $c : P \rightarrow \mathbb{R}_{>0}$ is a cost function assigning a strictly positive cost to each project and A is a voting profile defined below. For a project set $S \subseteq P$, we naturally extend the cost function to be defined as $c(S) = \sum_{p \in S} c(p)$ for the total cost of the projects in S .

An *outcome* of an election is a budget allocation $W \subseteq P$ satisfying $c(W) \leq b$. A *voting rule* is a function $\mathcal{R}$ that maps a given election to a feasible outcome $W = \mathcal{R}(I)$. Several state-of-the-art voting rules are discussed in Section 5. We refer to this setup as the *general PB model*.

2.2 Voting

Each voter i casts a ballot A_i capturing their individual valuation of the projects under consideration, and the vector $A = (A_1, \dots, A_n)$ is called the *profile*. The precise format of a ballot depends on the type of election:

- **Cardinal elections.** Each voter $i \in N$ assigns a score $A_i(p) \in \mathbb{R}_{\geq 0}$ to each project $p \in P$. In practice, scores are often bounded so that $A_i(p) \in [0, k]$ for some $k \in \mathbb{N}$.
- **Approval elections.** Each voter $i \in N$ either approves or rejects each project, i.e. $A_i(p) \in \{0, 1\}$. The *approval set* of voter i is $A_i := \{p \in P : A_i(p) = 1\}$.

In an approval election, voters simply indicate which projects they support, whereas cardinal elections additionally allow voters to express the degree of their support. Cardinal ballots are therefore strictly more expressive, but approval ballots remain the dominant format in real-world PB processes and consequently the most studied in the literature (Rey and Maly, 2023). Their simple binary structure tends to allow for stronger axiomatic results and requires less cognitive effort from the voters. For these reasons, our work focuses entirely on approval elections.

Throughout the thesis, we will often speak of an *approval-based PB instance*, by which we always mean an election instance $I = \langle N, P, A, b, c \rangle$ in which voters submit approval ballots.

2.3 Utility Functions

To evaluate how well an election outcome serves voters, we must first formalize the concept of voter utility. We do this through utility functions, which assign a numerical value to the utility a voter derives from a given election outcome.

For cardinal elections, voters are asked to assign a score reflecting their preferences to each project. Therefore, there is no need to consider anything other than the assigned score when measuring their utility. Thus, the utility for a voter is simply the sum of the score they have assigned for each project that has been selected $\mu_i(W) = \sum_{p \in W} \mu_i(p)$. For approval elections, it is less clear which measure of utility to use for a voter. We therefore have to consider different ways of translating election outcomes into utility for individual voters given their approval sets. We define *approval-based utility functions* below.

Definition 1. A utility function $\mu : 2^P \rightarrow \mathbb{R}_{\geq 0}$ is a function satisfying $\mu(S) \leq \mu(S')$ whenever $S \subseteq S'$ and $\mu(S) = 0$ if and only if $S = \emptyset$. (Brill et al., 2023)

For single projects, we sometimes use the shorthand notation $\mu_i(p) = \mu_i(\{p\})$.

For a voter $i \in N$, their utility from a project set $S \subseteq P$ is defined as:

$$\mu_i(S) = \mu(A_i \cap S),$$

where A_i represents the projects voter i approved. We assume that $\sum_{i \in N} \mu_i(p) > 0$ for each $p \in P$, meaning that every candidate project is assigned positive utility by at least one voter. If this is not the case for some project, it is simply removed from consideration.

We further assume utility functions are additive: $\mu(S) = \sum_{p \in S} \mu(p)$. We are often interested throughout the paper in strictly increasing utility functions, i.e., μ is strictly increasing if $\mu(S) < \mu(S')$ for all $S, S' \subseteq P$ with $S \subset S'$. The two most commonly studied utility functions occupy opposite ends of a spectrum:

- **Cardinality-Based Utility:** assigns utility $\mu^\#(S) = |S|$ equal to the number of approved projects that receive funding for some S .
- **Cost-Based Utility:** assigns utility $\mu^c(S) = c(S)$ equal to the total cost of the approved projects that receive funding for some S .

This raises a natural question: which model better captures voter preferences? Consider a choice between funding two \$500,000 projects versus one \$1.2 million project. Cardinality-based utility would pick the former, and cost-based utility would pick the latter. There is no obvious answer which one provides more utility. This ambiguity motivates a broader class of utility functions that contains both as special cases.

Definition 2 (DNS functions). An additive utility function μ has *weakly decreasing normalized satisfaction* (DNS) if for all projects $p, p' \in P$ with $c(p) \leq c(p')$ the following two inequalities hold:

$$\mu(p) \leq \mu(p') \quad \text{and} \quad \frac{\mu(p)}{c(p)} \geq \frac{\mu(p')}{c(p')}.$$

In this case, we call μ a DNS function.

A DNS function enforces two natural constraints: (i) more expensive projects yield at least as much utility as cheaper ones, and (ii) more expensive projects do not provide greater utility per unit cost. Both utility functions introduced above satisfy DNS: the cardinality function does so since all projects yield equal satisfaction, i.e. $\mu(p) = \mu(p') = 1$, while the cost-based function satisfies it because $\mu(p) = c(p)$ and thus $\frac{\mu(p)}{c(p)} = \frac{\mu(p')}{c(p')}$.

In Section 5, we will encounter concrete examples of this, with voting rules such as the Method of Equal Shares and Sequential Phragmén satisfying fairness guarantees specifically for DNS utility functions.

3 Participatory Budgeting Axioms

With the formal model in place, we now turn to the question of what it means for a voting rule to behave well. A natural first instinct might be to simply maximize the total utility across all voters, but this can lead to outcomes that systematically ignore minority groups in favor of the majority. In PB, where the goal is to allocate public funds fairly across a diverse population, such outcomes are generally considered undesirable. The axiomatic approach instead captures fairness through formal properties that a rule ought to satisfy. We begin with two foundational properties: *priceability* and *exhaustiveness*, which concern the structural soundness of an outcome. We then move to proportionality axioms, such as Extended Justified Representation (EJR), which capture the idea that groups of voters with shared preferences should receive appropriate representation in the final outcome.

3.1 Priceability

The first axiom we will examine is *priceability*, introduced by [Peters and Skowron \(2020\)](#). Informally, priceability captures the idea that a budget allocation is *fair* if it can be interpreted as the outcome of a market process, where each voter is given an equal share of the total budget and the selected projects are funded collectively, with only voters who approve a project contributing to its cost. If a project is approved by many voters, its cost is spread broadly and each supporter pays little. If a project has few supporters, each must contribute a larger fraction of their share, and if this fraction exceeds what they are entitled to spend, the project should not be selected. In this way, priceability also acts as a proportionality axiom, ensuring that groups of voters with shared preferences can collectively fund projects they care about in proportion to their size. We present a formal definition adapted from that of [Rey and Maly \(2023\)](#).

Definition 3. (Priceability). Given an approval-based PB instance, an outcome W satisfies priceability, or is said to be priceable, if there exists an entitlement $\alpha \in \mathbb{R}_{\geq 0}$ and a collection of contribution functions $\gamma_i : P \rightarrow [0, \alpha]$ such that the following conditions are met.

- C1:** $\gamma_i(p) > 0 \implies \mu_i(p) > 0$ for all $p \in P$ and $i \in N$: Voters only contribute to projects with positive utility.
- C2:** $\gamma_i(p) > 0 \implies p \in W$ for all $p \in P$ and $i \in N$: Only projects in the outcome W receive funding.
- C3:** $\sum_{p \in P} \gamma_i(p) \leq \alpha$ for all $i \in N$: No voter contributes more than their entitlement α .
- C4:** $\sum_{i \in N} \gamma_i(p) = c(p)$ for all $p \in W$: All the projects in W are sufficiently funded.
- C5:** $\sum_{i \in N: A_i(p)=1} (\alpha - \sum_{p' \in P} \gamma_i(p')) < c(p)$ for all $p \in P \setminus W$: The voters approving an unselected project p do not have $c(p)$ or more in total unspent entitlement.

The pair $(\alpha, (\gamma_i)_{i \in N})$ is called a *price system*.

We can strengthen the notion of priceability by imposing an additional constraint on the price system (Brill et al., 2023). Intuitively, the standard priceability conditions ensure that the budget is spent consistently, but they do not prevent a situation where voters pay more for a chosen project than the cost of some unchosen project they also approve. The following strengthening rules this out, requiring that cheaper projects are always bought first. This additional constraint, which we call C6-priceability, turns out to be precisely what is needed to prove a strong fairness result later.

Definition 4 (C6-Priceability). Given an approval-based PB instance, an outcome W that is priceable for price system $(\alpha, (\gamma_i)_{i \in N})$ is said to be C6-priceable if the following condition is met:

C6:

$$\sum_{i \in N: A_i(p)=1} \gamma_i(p') \leq c(p)$$

for all $p \in P \setminus W$ and $p' \in W$.

Intuitively, this additional constraint says that no voters can save money by collectively moving their contributions to a project that they all support.

Another technical requirement often used together with **C6** in proofs is the requirement $\alpha \cdot n > b$. At first sight this may seem counterintuitive, since it means that the total entitlement assigned to the voters is slightly larger than the actual budget. However, this condition should be understood as a small perturbation of the usual entitlement $\alpha = b/n$.

The idea is that, after the outcome has been fixed, we may increase every voter's entitlement by a sufficiently small amount ϵ . This increase is chosen so small that no unselected project becomes affordable from the remaining voter budgets. Hence, the same outcome and the same contribution functions still satisfy the priceability conditions, while the technical condition $\alpha \cdot n > b$ also becomes true. Equivalently, instead of using $\alpha = b/n$, we use

$$\alpha = \frac{b + \epsilon}{n}$$

for some sufficiently small $\epsilon > 0$.

3.2 Exhaustiveness

Another axiom we will examine is *exhaustiveness*, introduced by Aziz et al. (2018b). This axiom is perhaps the most natural of all, and requires that the budget should not be underused. Meaning that an exhaustive outcome is one where no other project $p \in P \setminus W$ can be afforded with the remaining budget. On the one hand, in some cases, the remaining budget from a non-exhaustive outcome can be reused, e.g., in next year's election, where it can be put to better use. On the other hand, it could

be that unspent money needs to be returned. In the second case, obtaining an exhaustive outcome is crucial.

It is notable that exhaustiveness is not always compatible with other axioms such as priceability, and enforcing it may require modifying an otherwise desirable outcome. We will use the definition of exhaustiveness as given by [Peters et al. \(2021\)](#).

Definition 5 (Exhaustiveness). Given an approval-based PB instance, an outcome W is *exhaustive* if for any project $p \in P \setminus W$ it holds that $c(W \cup \{p\}) > b$.

That is, no project outside the winning outcome can be added to it without violating the budget constraint. This is a weak requirement in the sense that it places no constraint on *which* projects are selected, only that the outcome cannot be trivially extended. As mentioned, exhaustiveness and priceability are not always compatible: there exist election instances in which no outcome simultaneously satisfies both axioms.

Proposition 1. *There exists an approval-based PB instance such that there is no feasible outcome W which is both priceable and exhaustive, even though there exist feasible budget allocations that are priceable, and others that are exhaustive. ([Peters et al., 2021](#))*

This can be demonstrated by a simple example, showing an election instance I , where no exhaustive outcome is priceable.

Example 1. Consider the following election instance with $b = 1$.

	p_1	p_2	p_3
cost	1	$1/3$	$1/3$
A_1	✓		
A_2	✓		
A_3		✓	✓

The only feasible and exhaustive outcomes are $W_1 = \{p_1\}$ and $W_2 = \{p_2, p_3\}$. We show that neither is priceable.

First consider $W_1 = \{p_1\}$. Since only voters 1 and 2 approve p_1 , they must together contribute its full cost, namely 1. Hence any price system for W_1 must have $\alpha \geq \frac{1}{2}$. However, voter 3 approves both unselected projects p_2 and p_3 , and contributes nothing under W_1 . Therefore, voter 3 has unspent entitlement α . By condition C5, this must be at most the cost of each of p_2 and p_3 , so $\alpha < \frac{1}{3}$. This contradicts $\alpha \geq \frac{1}{2}$.

Now consider $W_2 = \{p_2, p_3\}$. Since only voter 3 approves p_2 and p_3 , voter 3 must pay their total cost, which is $\frac{2}{3}$. Hence $\alpha \geq \frac{2}{3}$. On the other hand, voters 1 and 2 approve the

unselected project p_1 and contribute nothing under W_2 . By condition C5, their total unspent entitlement must be at most $c(p_1) < 1$, so $2\alpha < 1$, implying $\alpha < \frac{1}{2}$. This contradicts $\alpha \geq \frac{2}{3}$. Thus, no exhaustive outcome is priceable. Note that the outcomes $\{p_2\}$ and $\{p_3\}$ are priceable but not exhaustive.

4 Proportionality

The notion of proportionality is central in participatory budgeting. Intuitively, it captures the idea that groups of voters with shared preferences should be represented in the outcome in proportion to their size. A group of voters comprising a t fraction of the population should collectively see a t fraction of the budget spent on projects they support. Proportionality thus serves as a baseline guarantee: no group should be made systematically worse off simply by virtue of being a minority. Consider, for instance, a small group of voters who care deeply about local neighborhood improvements such as a playground, a community center, or a bike path. Under a purely utilitarian rule, their preferences may be drowned out by the numerical weight of the majority, leaving their portion of the budget consistently spent on projects they have little stake in.

The standard approach to proportionality is to identify so-called *cohesive groups*: Groups of voters that are large enough and share enough common preferences to jointly deserve some representation. And then require that the outcome does not neglect any such group. This gives rise to a family of proportionality axioms of varying strength. The strongest among these, *Full Justified Representation* (FJR), considers a superset of cohesive groups and demands that at least one voter is well-represented in each such group. The natural relaxation, *Extended Justified Representation* (EJR), requires that, in any cohesive group, at least one voter is well represented. The weaker notion of *Proportional Justified Representation* (PJR) instead requires that the cohesive group as a whole is collectively represented. The latter two axioms have natural relaxations that we will also study, the need for which arises due to the difficulty of satisfying the stronger versions.

In this section we define these axioms formally and study central results regarding them.

4.1 Cohesive Groups

Definition 6 (T-cohesive groups). Given an approval based PB instance, a non-empty group of voters $N' \subseteq N$ is said to be T -cohesive, for a nonempty set of projects $T \subseteq P$, if the following two conditions are satisfied:

- for all $p \in T$ and $i \in N'$ we have $A_i(p) = 1$, that is, every voter in N' approves all projects in T .
- $\frac{|N'|}{n} \cdot b \geq c(T)$, that is N' has large enough budget share to afford the projects in T .

Notice that for any T -cohesive group N' and voter $i \in N'$ it holds that $\mu_i(T) = \mu(T \cap A_i) = \mu(T)$ since $T \subseteq \cap_{i \in N'} A_i$.

Example 2 (Cohesive groups). Consider the following instance

	p_1	p_2	p_3	p_4
cost	2	4	3	11
A_1		✓		✓
A_2		✓		✓
A_3	✓	✓	✓	✓

Table 1: Running example instance with $b = 15$.

Every voter has control over $15/3 = 5$ units of money. Since $c(p_2) = 4 \leq 5$ and all 3 agents approve of p_2 , every nonempty subset $N' \subseteq N$ is going to be $\{p_2\}$ -cohesive. There are 7 such cohesive groups.

Voter 3 has large enough budget share to afford $\{p_1, p_3\}$, which means that the group $\{3\}$ is $\{p_1, p_3\}$ -cohesive and also $\{p_1\}$ -cohesive, and $\{p_3\}$ -cohesive.

Lastly, N is going to be $\{p_2, p_4\}$ -cohesive as well as $\{p_4\}$ -cohesive, which exhaustively gives us a total of 12 cohesive groups.

4.2 Multi-Winner Elections

The proportionality axioms we study generalize well-known notions from multi-winner elections (MWE). In MWE, every project has unit cost, and the goal is to select a fixed-size committee of k candidates. In this setting, axioms such as Proportional Justified Representation (PJR), and Extended Justified Representation (EJR) were originally introduced as notions of fairness (Aziz et al., 2017, 2018b). Participatory budgeting generalizes this setting by replacing the fixed committee size with a budget limit and allowing projects to have arbitrary costs. The central axioms we study are the natural adaptations of PJR and EJR to this more general setting.

4.3 Extended Justified Representation

Extended Justified Representation (EJR) was originally proposed for approval-based committee elections by Aziz et al. (2017). This axiom attempts to ensure that any sufficiently large and cohesive group of voters receives proportional representation. EJR was first adapted for the general PB model by Peters et al. (2021).

Definition 7 (Extended Justified Representation). Given an approval based PB instance and a utility function μ , an outcome W satisfies extended justified representation with respect to μ , (μ -EJR) if and only if for any T -cohesive group N' , there exists $i \in N'$ such that

$$\mu_i(W) \geq \mu_i(T).$$

Example 3 (Extended Justified Representation). We now examine some outcomes that satisfy μ -EJR with respect to the instance I from Table 1, using $\mu = \mu^\#$ for simplicity.

The following two outcomes are examples that both satisfy $\mu^\#$ -EJR:

- $W_1 = \{p_2, p_4\}$ with $c(W_1) = 15$,
- $W_2 = \{p_1, p_2, p_3\}$ with $c(W_2) = 9$.

The important cohesive groups to consider are $\{3\}$ being $\{p_1, p_3\}$ -cohesive (requiring $\mu_3(W) \geq 2$) and N being $\{p_2, p_4\}$ -cohesive (requiring some $i \in N$ with $\mu_i(W) \geq 2$), so let's confirm that these are indeed also satisfied.

Outcome $W_1 = \{p_2, p_4\}$:

- For $\{3\}$ being $\{p_1, p_3\}$ -cohesive: we need $\mu_3(W_1) \geq \mu_3(\{p_1, p_3\}) = 2$. Which is satisfied since $\mu_3(W_1) = |W_1 \cap A_3| = |\{p_2, p_4\}| = 2 \checkmark$
- For N being $\{p_2, p_4\}$ -cohesive: we need some $i \in N$ with $\mu_i(W_1) \geq 2$. In fact for any i it holds that $\mu_i(W_1) = |\{p_2, p_4\}| = 2 \geq 2$. $\checkmark$

All remaining cohesive groups involve only $\{p_2\}$ which is clearly satisfied or single projects whose constraint is implied by the above, so W_1 satisfies $\mu^\#$ -EJR.

Outcome $W_2 = \{p_1, p_2, p_3\}$:

- For $\{3\}$ being $\{p_1, p_3\}$ -cohesive: we have $\mu_3(W_2) = |W_2 \cap A_3| = |\{p_1, p_2, p_3\}| = 3 \geq 2 \checkmark$
- For N being $\{p_2, p_4\}$ -cohesive: we need some $i \in N$ with $\mu_i(W_2) \geq 2$. As already establishes, we have $\mu_3(W_2) = 3 \geq 2 \checkmark$

Hence W_2 also satisfies $\mu^\#$ -EJR.

Example 4 (EJR failure). To see an outcome that *fails* μ^c -EJR, (notice change of utility function) consider again $W_2 = \{p_1, p_2, p_3\}$ with $c(W_2) = 9$.

Since N is $\{p_2, p_4\}$ -cohesive we need some i with $\mu_i^c(W_2) \geq \mu_i^c(\{p_2, p_4\}) = 15$ which is impossible since $\mu_i^c(W_2) = 9$ for $i \in N$. Hence W_2 *fails* μ^c -EJR.

Notably, EJR requires only that at least one voter per T-cohesive group have $|T|$ approved projects implemented in the cardinality-based utility model. This requirement may seem weak, but it implies that cohesive groups have high average utility in the outcome and rules out outcomes in which entire cohesive groups are ignored. Furthermore, EJR is always satisfiable:

Theorem 1. μ -EJR is always satisfiable for any strictly increasing utility function μ . ([Peters et al., 2021](#))

To be able to give a proof of this theorem, we first need to define the following voting rule:

Definition 8. Given an approval based PB instance, the Greedy Cohesive Rule (GCR) with respect to μ ($GCR[\mu]$) selects the outcome W that results from the following procedure:

Algorithm 1: The GCR Algorithm

Input: An approval based PB election I

Output: An election outcome W .

```

1 Function GCR ( $I$ )
2    $W \leftarrow \emptyset$ 
3    $N^W \leftarrow N$ 
4    $C(W, N^W) \leftarrow \{(T, N') \mid T \subseteq P \setminus W \wedge \exists T\text{-cohesive group } N' \subseteq N^W\}$ 
5   while  $C(W, N^W) \neq \emptyset$  do
6      $(T, N') \leftarrow \operatorname{argmax}_{(\hat{T}, \hat{N}) \in C(W, N^W)} \mu(\hat{T})$ 
7      $W \leftarrow W \cup T$ 
8      $N^W \leftarrow N^W \setminus N'$ 
9      $C(W, N^W) \leftarrow \{(T, N') \mid T \subseteq P \setminus W \wedge \exists T\text{-cohesive group } N' \subseteq N^W\}$ 
10  return  $W$ 
    
```

This rule was first introduced by [Peters et al. \(2021\)](#) and later simplified to the version shown above by [Brill et al. \(2023\)](#).

Intuitively, the algorithm builds the set $C(W, N^W)$, which contains all cohesive groups. Line 6 picks the cohesive group that gives maximum utility with respect to μ (in case of a tie-breaker, pick one arbitrarily). Add these projects to the outcome W , and remove the voters from that cohesive group from consideration, since they are now satisfied. Repeat until no cohesive groups remain.

Example 5 (GCR). Let's look at an example outcome W produced by $GCR[\mu]$ for $\mu = \mu^\#$ using the instance I from Table 1.

In line 3, $C(W, N^W)$ consists of all 12 cohesive groups. There are 3 cohesive groups with $\mu(\hat{T}) = 2$, so let's arbitrarily pick the $\{p_1, p_3\}$ -cohesive group $\{3\}$.

Thus $W = \{p_1, p_3\}$, $N^W = \{1, 2\}$.

Now, the only cohesive groups left to consider are those that do not contain projects already in W with $N' \subseteq N^W$. These are the $\{p_2\}$ cohesive groups $\{1\}, \{2\}, \{1, 2\}$. They all provide the same utility, so let's arbitrarily pick the $\{p_2\}$ -cohesive group $\{1\}$.

Thus $W = \{p_1, p_2, p_3\}$, $N^W = \{2\}$. Now there are no more cohesive groups to consider, so the algorithm terminates with $W = \{p_1, p_2, p_3\}$.

Notice that this outcome is feasible and satisfies $\mu^\#$ -EJR as seen in example 3, which is not a coincidence as we will now see.

With a good understanding of the GCR algorithm, we are ready to prove theorem 1:

Proof. We show that any outcome W returned by $\text{GCR}[\mu]$ satisfies μ -EJR.

Feasibility. Let W be an outcome computed by $\text{GCR}[\mu]$. Each time a set W' is selected at line 7, the definition of W' -cohesiveness guarantees that

$$c(W') \leq \frac{|N'| \cdot b}{n},$$

where N' is the corresponding cohesive group. Since each voter is removed from N^W as a member of exactly one such group, summing over all selected sets gives

$$c(W) \leq \frac{|N| \cdot b}{n} = b,$$

so W is a feasible outcome.

Satisfaction of μ -EJR. Suppose for contradiction that there exists a utility function μ and an approval based PB instance such that some outcome W returned by $\text{GCR}[\mu]$ does not satisfy μ -EJR. By definition of μ -EJR, there must then exist a T -cohesive group N' such that

$$\mu_i(W) < \mu_i(T) = \mu(T) \quad \text{for all } i \in N'. \quad (1)$$

By the strict increasingness of μ , condition (1) implies $T \not\subseteq W$. Let i_0 be the first voter in N' removed by the algorithm. Such a voter must exist otherwise, when we get to line 9, we still have $N' \subseteq N^W$, which means N' could be added to the set $C(W, N^W)$ which contradicts termination of the algorithm.

Thus i_0 was removed as a member of some T' -cohesive group N'' . Since $T' \subseteq W$, we have:

$$\mu_{i_0}(W) \geq \mu(T').$$

In line 7, T' was chosen instead of T , which by definition means

$$\mu(T') \geq \mu(T)$$

combining these inequalities give

$$\mu_{i_0}(W) \geq \mu(T)$$

which contradicts (1). □

GCR runs in exponential time, and this is unavoidable. It can be shown that in the general PB model, no polynomial-time algorithm can always compute an outcome satisfying μ -EJR, unless $P = NP$. [Peters et al. \(2021\)](#) show this via a reduction from the Knapsack problem, which yields hardness for strongly polynomial-time algorithms. [Brill et al. \(2023\)](#) give a related result: for the restricted class of strictly cost-responsive utility functions, no polynomial-time algorithm can

always satisfy μ -EJR unless $P = NP$. This also applies to the approval-based PB model. We first define this class and present its proof in greater detail below.

Definition 9 (Strictly cost-responsive). A utility function μ is *strictly cost-responsive* if for all $W, W' \subseteq P$ with $c(W) < c(W')$, we have $\mu(W) < \mu(W')$.

Notably, this class includes μ^c but not $\mu^\#$.

Theorem 2. *There is no polynomial-time algorithm that, given an approval-based PB instance and a strictly cost-responsive utility function μ , always computes an outcome satisfying μ -EJR, unless $P = NP$.*

Proof. We reduce from the Subset Sum problem, which is known to be NP-hard (Karp, 1972). An instance of Subset Sum consists of a set of integers $\{s_1, \dots, s_m\}$ and a target $t \in \mathbb{N}$. The question is whether there exists $X \subseteq [m]$ such that $\sum_{j \in X} s_j = t$.

Given such an instance, construct an approval-based PB instance as follows. There is a single voter who approves all projects. The set of projects is $P = \{p_1, \dots, p_m\}$ with costs $c(p_j) = s_j$, and the budget limit is $b = t$.

We claim that any algorithm $\mathcal{A}$ computing an outcome satisfying μ -EJR can be used to decide Subset Sum. Since there is only one voter, T -cohesiveness of $N' = \{1\}$ reduces to the condition $c(T) \leq \frac{|N'|}{n} \cdot b = b$, i.e., T is any feasible set. μ -EJR then requires that the returned outcome W satisfies

$$\mu(W) \geq \mu(T)$$

for every feasible T . In other words, this boils down to maximizing μ over all feasible outcomes. Now, since μ is strictly cost-responsive, maximizing μ over feasible outcomes is equivalent to finding a feasible outcome of maximum cost. Such an outcome has cost exactly b if and only if the Subset Sum instance has a solution. Therefore, the original instance is a yes-instance of Subset Sum if and only if $\mathcal{A}$ computes an outcome W such that $\sum_{p \in W} c(p) = b$.

Thus $\mathcal{A}$ solves Subset Sum, which means $\mathcal{A}$ runs in exponential time unless $P = NP$. $\square$

This hardness result gives motivation to look at a slight relaxation of μ -EJR.

4.3.1 Extended Justified Representation Up to One Project

Definition 10 (Extended Justified Representation up to one project). Given an approval based PB instance and a utility function μ , an outcome W satisfies extended justified representation up to one project with respect to μ , (μ -EJR-1) if and only if for any T -cohesive group N' , we have that $T \subseteq W$

or there exists $i \in N'$ and project $p \in T \setminus W$ such that

$$\mu_i(W \cup \{p\}) > \mu_i(T).$$

Example 6 (Extended Justified Representation up to one project). We examine μ -EJR-1 with respect to the instance in Table 1, here we let $\mu = \mu^c$. Recall that the binding cohesive groups are $\{3\}$ being $\{p_1, p_3\}$ -cohesive and N being $\{p_2, p_4\}$ -cohesive.

It should be clear that the outcomes that satisfy μ -EJR also satisfy μ -EJR-1, but let's see that it is in fact strictly weaker.

Outcome $W' = \{p_1, p_2, p_3\}$: We showed earlier that W' fails μ^c -EJR. We now check whether W' satisfies the weaker μ^c -EJR-1.

- For $\{3\}$ being $\{p_1, p_3\}$ -cohesive we have $\{p_1, p_3\} \subseteq W'$, so we are immediately done. ✓
- For N being $\{p_2, p_4\}$ -cohesive: Since $\{p_2, p_4\} \not\subseteq W'$, we need some $i \in N$ and $p \in T \setminus W' = \{p_4\}$ with $\mu_i(W' \cup \{p\}) > \mu_i(\{p_2, p_4\}) = 15$. Taking $i = 3$ and the only option for p which is p_4 , gives $\mu_3(W' \cup \{p_4\}) = 20$. ✓

Hence W' fails μ -EJR but satisfies μ -EJR-1, illustrating that μ -EJR-1 is strictly weaker than μ -EJR.

Very interestingly, in the special case of the cardinality utility function $\mu^\#$, the notions $\mu^\#$ -EJR-1 and $\mu^\#$ -EJR coincide (Peters et al., 2021). The reason is that under $\mu^\#$, utility simply counts the number of approved projects, and adding any approved project increases utility by exactly one. As a result, the strict inequality in the definition of EJR-1 becomes equivalent to the weak inequality used in EJR. Formally, we define this here:

Theorem 3. *In the approval setting, $\mu^\#$ -EJR-1 and $\mu^\#$ -EJR coincide.*

Proof. We show that in the approval setting, the condition defining $\mu^\#$ -EJR-1 is equivalent to that of $\mu^\#$ -EJR.

Let $i \in N$, $W \subseteq P$, and $T \subseteq P$ satisfying $T \subseteq A_i$. In the approval setting, we have

$$\mu_i^\#(W) = |W \cap A_i|.$$

For any project $p \in T \setminus W$, it holds that

$$\mu_i^\#(W \cup \{p\}) = |W \cap A_i| + 1.$$

Thus, the $\mu^\#$ -EJR-1 condition

$$\mu_i^\#(W \cup \{p\}) > \mu_i^\#(T)$$

is equivalent to

$$|W \cap A_i| + 1 > |T \cap A_i|.$$

Since both sides are integers, this holds if and only if

$$|W \cap A_i| \geq |T \cap A_i|.$$

On the other hand, the $\mu^\#$ -EJR condition requires

$$\mu_i^\#(W) \geq \mu_i^\#(T) \iff |W \cap A_i| \geq |T \cap A_i|.$$

Hence, the two conditions coincide, and therefore $\mu^\#$ -EJR-1 and $\mu^\#$ -EJR are equivalent in the approval setting. $\square$

As we will see later, Theorem 3 allows us to compute outcomes satisfying $\mu^\#$ -EJR in polynomial time using the Method of Equal Shares.

EJR-1 is the most relaxed version of EJR that we are interested in. As we will see, many of the results established for EJR-1 also hold for the strictly stronger notion of μ -EJR-x, because we work primarily in the approval setting, which we introduce next.

4.3.2 Extended Justified Representation Up to Any Project

Definition 11 (Extended Justified Representation up to any project). Given an approval PB instance and a utility function μ , an outcome W satisfies extended justified representation up to any project with respect to μ , (μ -EJR-x) if and only if for any T -cohesive group N' , we have $T \subseteq W$ or there exists $i \in N'$ such that

$$\mu_i(W \cup \{p\}) > \mu_i(T)$$

for any project $p \in T \setminus W$.

Example 7 (Extended Justified Representation up to any project). Let's look at an example that shows EJR-x is strictly stronger than EJR-1.

Consider the following instance:

	p_1	p_2	p_3	p_4
cost	5	5	6	1
A_1	✓		✓	✓
A_2		✓	✓	✓

 Table 2: Example with $b = 10$.

The cohesive groups are $\{1\}$ being $\{p_1\}$ -cohesive and $\{p_4\}$ -cohesive. $\{2\}$ being $\{p_2\}$ -cohesive and $\{p_4\}$ -cohesive. Lastly $\{1, 2\}$ is $(\{p_3\}, \{p_4\}, \{p_3, p_4\})$ -cohesive.

Consider the outcome $W = \{p_1, p_2\}$ with $\mu = \mu^c$, and let $T = \{p_3, p_4\}$. The group $\{1, 2\}$ is T -cohesive. Since

$$\mu_1(W \cup \{p_3\}) = c(\{p_1, p_3\}) = 11 > \mu_1(T) = 7.$$

this cohesive group satisfies μ^c -EJR-1. However, because

$$\mu_1(W \cup \{p_4\}) = \mu_2(W \cup \{p_4\}) = 6 \not\geq 7$$

the group violates μ^c -EJR-x.

It is easy to verify that all other cohesive groups satisfy both μ^c -EJR-1 and μ^c -EJR-x, thus the instance as a whole satisfies μ^c -EJR-1 but not μ^c -EJR-x.

We have

Proposition 2. *For any strictly increasing utility function μ , μ -EJR implies μ -EJR-x.*

Proof. Assume W satisfies μ -EJR. By definition, for any T -cohesive group N' , there exists $i \in N'$ satisfying

$$\mu_i(W) \geq \mu_i(T).$$

Either $T \subseteq W$, and thus μ -EJR-x is satisfied, or by assumption that μ is strictly increasing, we have for any $p \in T \setminus W$:

$$\mu_i(W \cup \{p\}) > \mu_i(W) \geq \mu_i(T).$$

□

We will later show that μ -EJR-x is in fact achievable and practical by an efficient voting rule. Specifically, we will show that the Method of Equal Shares, parameterized with a DNS utility function, always returns an outcome satisfying the strong guarantee of μ -EJR-x, which is a quite significant result.

4.4 Proportional Justified Representation

If our goal is to show something general for all DNS functions, then we need proportionality rules that are even weaker than EJR-1. This fact follows from the following proposition.

Proposition 3. *There is an approval-based PB instance for which no outcome satisfies μ^c -EJR-1 and $\mu^\#$ -EJR-1 simultaneously.*

The proof is credited to Brill et al. (2023) and given here in more detail

Proof. Consider $n = 2$ voters and $m = 12$ projects $p_1, \dots, p_{12}$, where $c(p_1) = c(p_2) = 5$ and $c(p_j) = 1$ for all $j \geq 3$. Set the budget $b = 10$. The approval sets are:

$$A_1 = \{p_1, p_2, p_3, p_4, p_5, p_6, p_7\}, \quad A_2 = \{p_1, p_2, p_8, p_9, p_{10}, p_{11}, p_{12}\}.$$

Under $\mu^\#$, consider the cohesive groups of individual voters. Voter 1 alone is T_1 -cohesive for $T_1 = \{p_3, p_4, p_5, p_6, p_7\}$, since

$$c(T_1) = 5 = \frac{1}{2} \cdot 10 = \frac{|N'|}{n} \cdot b.$$

Likewise, voter 2 alone is T_2 -cohesive for $T_2 = \{p_8, p_9, p_{10}, p_{11}, p_{12}\}$.

Now suppose $p_1 \in W$ (the case $p_2 \in W$ is symmetric). Since $c(p_1) = 5$ and $b = 10$, any feasible outcome containing p_1 can include at most 5 additional unit-cost projects. For $\mu^\#$ -EJR-1 to hold for voter 2's cohesive group, we need some $p \in T_2 \setminus W$ such that $\mu_2^\#(W \cup \{p\}) > \mu^\#(T_2) = 5$. Since $p_1 \in W$, voter 2 has at most $1 + k$ approved projects in W for some $k \leq 5$, and adding one more gives $\mu_2^\#(W \cup \{p\}) = 1 + k + 1$. This exceeds 5 only if $k \geq 4$. By the same argument applied symmetrically to voter 1, W would need to contain p_1 (or p_2), at least 4 projects from $\{p_3, \dots, p_7\}$, and at least 4 from $\{p_8, \dots, p_{12}\}$, giving total cost at least $5 + 4 + 4 = 13 > 10$, which is infeasible.

Thus, any outcome satisfying $\mu^\#$ -EJR-1 must exclude both p_1 and p_2 and include all ten unit-cost projects. Hence the only possible such outcome is:

$$W^* = \{p_3, p_4, p_5, p_6, p_7, p_8, p_9, p_{10}, p_{11}, p_{12}\},$$

which is the unique outcome satisfying $\mu^\#$ -EJR-1.

Under μ^c , both voters together form a $\{p_1, p_2\}$ -cohesive group, since

$$c(\{p_1, p_2\}) = 10 = \frac{|N'|}{n} \cdot b.$$

For W^* to satisfy μ^c -EJR-1 for this group, there must exist some voter $i \in \{1, 2\}$ and some $p \notin W^*$ such that $\mu_i^c(W^* \cup \{p\}) > \mu^c(\{p_1, p_2\}) = 10$. Since such a p can only be picked from $\{p_1, p_2\}$ we have $\mu_i^c(W^* \cup \{p\}) \leq 10$. Hence W^* violates μ^c -EJR-1. $\square$

Proposition 3.6 tells us that to achieve stronger proportionality guarantees, we need to fix a specific satisfaction function. As mentioned, this is not always desirable, which motivates the introduction of weaker notions of proportionality.

Instead of requiring, as in EJR, that for any cohesive group there exists a single voter that is well-represented, PJR weakens this requirement by considering the satisfaction of the group as a whole. In particular, the set of projects in W approved by *at least one* voter in the group must collectively provide sufficient satisfaction (Brill et al., 2023). As a result, representation is ensured at the group level rather than at the level of individual voters.

Definition 12 (Proportional Justified Representation). Given an approval-based PB instance, an outcome W satisfies μ -PJR if and only if for any T -cohesive group N' it holds that

$$\mu \left(W \cap \bigcup_{i \in N'} A_i \right) \geq \mu(T).$$

Since $A_i \cap W \subseteq (\bigcup_{i \in N'} A_i) \cap W$, we have

$$\mu_i(W) \leq \mu \left(W \cap \bigcup_{i \in N'} A_i \right)$$

for any satisfaction function μ and $\mu_i(T) = \mu(T)$ following from the definition of T -cohesive groups. Hence, any outcome that satisfies μ -EJR also satisfies μ -PJR. Intuitively, if one voter in the group is sufficiently well represented, then the group as a whole must also be. Since EJR is always satisfiable for strictly increasing utility functions, we get

Corollary 1. *μ -PJR is always satisfiable for any strictly increasing μ .*

Interestingly, in the special case of a single voter, we have

$$\mu_i(W) = \mu \left(W \cap \bigcup_{i \in N'} A_i \right),$$

and thus μ -EJR and μ -PJR coincide. Consequently, any hardness result for μ -EJR directly carries over to μ -PJR in this setting.

Corollary 2. *For an approval-based PB instance with a single voter and a utility function that is strictly cost-responsive, there is no polynomial-time algorithm computing an outcome W satisfying μ -PJR, unless $P = NP$.*

This computational hardness motivates a relaxation of μ -PJR, much like the relaxation of EJR considered earlier.

4.4.1 Proportional Justified Representation Up to Any Project

Definition 13 (μ -PJR- x). Given an approval-based PB instance, an outcome W is said to satisfy μ -PJR- x if and only if for any T -cohesive group N' either $T \subseteq W$ or for any $p \in T \setminus W$ it holds that

$$\mu \left((W \cap \bigcup_{i \in N'} A_i) \cup \{p\} \right) > \mu(T).$$

By definition, μ -PJR- x is implied by μ -EJR- x .

Proposition 4. *For any strictly increasing utility function μ , μ -PJR implies μ -PJR- x .*

Proof. Assume W satisfies μ -PJR. By definition, we have for any T -cohesive group N' that

$$\mu(W \cap (\cup_{i \in N'} A_i)) \geq \mu(T).$$

If $T \subseteq W$, then μ -PJR- x clearly holds. Otherwise, let $p \in T \setminus W$. By assumption that μ is strictly increasing, we have

$$\mu((W \cap \bigcup_{i \in N'} A_i) \cup \{p\}) > \mu(W \cap \bigcup_{i \in N'} A_i) \geq \mu(T),$$

meaning that μ -PJR- x holds. □

We will see in the following section how μ -PJR- x can be satisfied for all DNS functions using the Method of equal shares voting rule with $\mu^\#$.

4.5 Full Justified Representation

The most powerful proportionality axiom that remains always satisfiable is *Full Justified Representation*, introduced by [Peters et al. \(2021\)](#) as a natural strengthening of EJR.

Definition 14 (Full Justified Representation). Given an approval-based PB instance and utility function μ , an outcome W satisfies μ -FJR if and only if for every group $N' \subseteq N$ and every nonempty $T \subseteq P$ with $|N'| \cdot b/n \geq c(T)$, there exists $i \in N'$ such that

$$\mu_i(W) \geq \mu_i(T).$$

Compared to μ -EJR, the approval requirement is dropped entirely: any group N' and any affordable set T form a valid pair to check, regardless of whether members of N' approve the projects in T . The groups checked under FJR therefore form a superset of the cohesive groups checked under EJR, which gives the following implication.

Proposition 5. *μ -FJR implies μ -EJR.*

Proof. Every cohesive group witnessing a violation of μ -EJR also witnesses a violation of μ -FJR, since the FJR condition imposes no approval requirement on N' . $\square$

In particular, this means that the hardness result of Theorem 2 carries over: Computing an outcome satisfying μ -FJR cannot be done in polynomial time unless $P = NP$.

What remains to be shown is that μ -FJR is always satisfiable, which follows by a greedy argument similar to the proof of Theorem 1, replacing cohesive groups with the weaker notion used in the FJR definition. We refer to [Peters et al. \(2021\)](#) for the full proof.

A drawback of FJR is the lack of practical voting rules satisfying it. As we will see in Section 5, rules such as $\text{MES}[\mu^\#]$ satisfy $\mu^\#$ -EJR, but no efficient rule is known to satisfy any variant of FJR.

4.6 Summary of Proportionality

Throughout this section, we have introduced several proportionality axioms for approval-based participatory budgeting. To summarize the relationships between these axioms, we draw several taxonomies in Figure 1. An arrow from one axiom to another means that every outcome satisfying the first also satisfies the other. Some arrows are only valid for certain utility functions and are labeled accordingly.

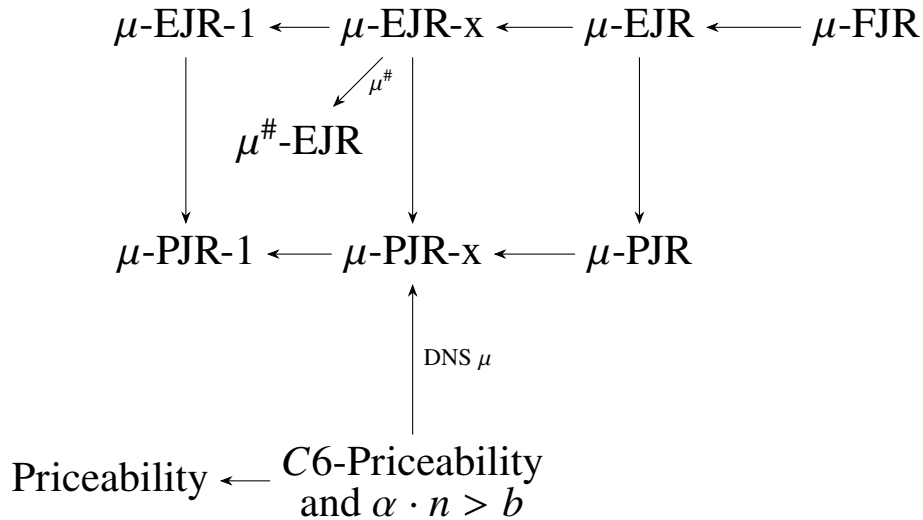


Figure 1: Proportionality Taxonomies in Participatory Budgeting

The arrow from C6-priceability to μ -PJR-x is something that will be stated in Theorem 6 in the following section.

5 Voting Rules

Once voting ballots have been collected, we need to determine how to use them to make a collective funding decision. This is done using Participatory Budgeting voting rules. Given a set of projects, their costs, a budget, and a profile of voter ballots, a voting rule will select a feasible subset of projects. This aggregation step is the central part of decision-making in Participatory Budgeting.

The key challenge in defining voting rules for the PB setting is the combinatorial nature of feasible outcomes. As the number of projects grows, the number of possible project subsets grows exponentially. Therefore, voting rules must balance both computational tractability along with normative properties such as fairness. In this section, we introduce several voting rules commonly studied in the PB literature and discuss their main computational and axiomatic properties. We begin with utilitarian welfare rules in Section 5.1, first considering exact welfare maximization and then practical greedy approximations. We then turn to proportional voting rules, in particular the Method of Equal Shares in Section 5.2, which provides strong proportionality guarantees. For comparison, we also introduce the Sequential Phragmén rule in Section 5.3. Throughout the section, we focus on how these rules balance welfare, computational feasibility, and proportionality guarantees.

5.1 Utilitarian Welfare Rules

From a purely utilitarian view, the objective of a voting rule is to select a feasible budget allocation that maximizes the overall utility of the voters. This is what Utilitarian Welfare Rules attempts to do. We define utilitarian welfare, denoted by UTIL-W as follows. Given an approval based PB instance, a feasible budget allocation W and a utility function μ :

$$\text{UTIL-W}[\mu](I, W) = \sum_{i \in N} \mu_i(W)$$

What this does is simply add each voter’s utility for a fixed feasible outcome W . As mentioned earlier, in the approval-based model, a voting rule does not have access to the *actual* utility of the voters and must make assumptions based on voters’ ballots. In the following sections we will discuss two approaches to welfare maximization, exact and approximate.

5.1.1 Exact Welfare Maximization

The most direct way to use utilitarian welfare is to select a feasible outcome that maximizes it. For a given additive utility function μ , the exact utilitarian welfare-maximizing rule is defined as

$$\text{MAXUTIL}[\mu](I) = \arg \max_{W \in \text{FEAS}(I)} \text{UTIL-W}[\mu](I, W).$$

Thus, exact welfare maximization searches over all feasible budget allocations and returns one with maximum total utility.

We now instantiate this general rule for the two approval-based utility functions introduced earlier. The cardinality welfare-maximizing rule **MAXCARD** is defined by using $\mu^\#$:

$$\begin{aligned} \text{MAXCARD}(I) &= \arg \max_{W \in \text{FEAS}(I)} \text{UTIL-W}[\mu^\#](I, W) \\ &= \arg \max_{W \in \text{FEAS}(I)} \sum_{i \in N} |\{p \in W : A_i(p) = 1\}|. \end{aligned}$$

The cost welfare-maximizing rule **MAXCOST** is similarly obtained by using μ^c :

$$\begin{aligned} \text{MAXCOST}(I) &= \arg \max_{W \in \text{FEAS}(I)} \text{UTIL-W}[\mu^c](I, W) \\ &= \arg \max_{W \in \text{FEAS}(I)} \sum_{i \in N} c(\{p \in W : A_i(p) = 1\}). \end{aligned}$$

Example 8 (Welfare Maximizing Rules). Consider the election instance I and the profile A of approval ballots with budget $b = 27$.

	p_1	p_2	p_3	p_4	p_5
Cost	6	7	8	9	18
A_1	✓	✓		✓	
A_2	✓		✓	✓	
A_3		✓	✓		✓
A_4	✓				✓

The outcomes of cardinality- and cost-based welfare-maximizing rules for the election instance are shown below.

	Outcomes	Achieved Social Welfare
$\mu^\#$	$\{p_1, p_2, p_3\}, \{p_1, p_2, p_4\}, \{p_1, p_3, p_4\}$	7
μ^c	$\{p_1, p_5\}, \{p_4, p_5\}$	54

It is worth noting that the outcomes that maximize differ between the two rules. This again demonstrates the importance of utility function selection.

Any voting rule following these welfare-maximizing definitions may produce multiple budget allocations for some election instances, as shown in Example 8. Such voting rules are called *irresolute rules* as opposed to *resolute* rules that always produce a single feasible budget allocation.

Irresoluteness is generally an undesirable property, but one that can be fixed by implementing some arbitrary tie-breaking measure among all maximizing outcomes.

While exact welfare maximization aims to maximize total utility, it does not guarantee proportional representation among voter groups, as the following proposition shows.

Proposition 6. *Given an approval based PB instance, and a utility function μ , the exact utilitarian welfare-maximizing rule $\text{MAXUTIL}[\mu]$ does not satisfy μ -PJR in general.*

Proof. We show this by constructing an election instance in which the utilitarian welfare-maximizing outcome violates μ -PJR. Consider an instance with n voters, budget $b = n$ and two projects p_1 and p_2 with costs $c(p_1) = 1$ and $c(p_2) = n$. Because $c(\{p_1, p_2\}) = n + 1 > b$, the two projects cannot both be selected. Now consider the following approval profile. Voter 1 approves only p_1 , while every other voter approves only p_2 :

$$A_1 = \{p_1\}, \quad A_2 = \dots = A_n = \{p_2\}$$

The utilitarian welfare of selecting p_1 is simply the utility received by voter 1:

$$\text{UTIL-W}[\mu](I, \{p_1\}) = \mu_1(p_1).$$

The utilitarian welfare of selecting p_2 is the total utility received by the remaining $n - 1$ voters:

$$\text{UTIL-W}[\mu](I, \{p_2\}) = \sum_{i=2}^n \mu_i(p_2)$$

Since approved projects yield positive utility, we can choose n sufficiently large to guarantee the utilitarian welfare-maximizing to be $W = \{p_2\}$, such that:

$$\sum_{i=2}^n \mu_i(p_2) > \mu_1(p_1)$$

We show that this outcome violates μ -PJR. Consider the group of voters $S = \{1\}$ and the project set $T = \{p_1\}$. The group S is T -cohesive as voter 1 approves p_1 and:

$$\frac{|S|}{|N|} \cdot b = \frac{1}{n} \cdot n = 1 = c(T)$$

However, the welfare-maximizing outcome is $W = \{p_2\}$, and voter 1 approves no selected project. This means the cohesive group S receives no utility from W , whereas it would receive positive utility from T . Stated formally using the definition of μ -PJR:

$$\mu\left(W \cap \bigcup_{i \in S} A_i\right) = \mu(\{p_2\} \cap \{p_1\}) = 0 < \mu(T)$$

Therefore, W violates μ -PJR, meaning that exact welfare maximization does not satisfy μ -PJR in general. $\square$

We can also show that exact welfare maximizing outcomes are guaranteed to be exhaustive.

Proposition 7. *Given an approval based PB instance, and an additive utility function μ that assigns positive utility to approved projects. Then every outcome returned by $\text{MAXUTIL}[\mu]$ is exhaustive.*

Proof. Let W be an outcome returned by $\text{MAXUTIL}[\mu]$. Suppose, for contradiction, that W is not exhaustive. Then there exists a project $p \notin W$ such that $W \cup \{p\}$ is feasible, meaning $c(W \cup \{p\}) \leq b$. Since projects with no approvals are removed from consideration, p is approved by at least 1 voter. As μ assigns positive utility to approved projects, adding p strictly increases the total utilitarian welfare. Thus,

$$\text{UTIL-}W[\mu](I, W \cup \{p\}) > \text{UTIL-}W[\mu](I, W)$$

This contradicts the assumption that W maximizes utilitarian welfare, and therefore W must be exhaustive. $\square$

In practice, exact welfare-maximizing rules are rarely used, partly due to their computational complexity. In general, these optimization problems exhibit a knapsack-style structure and are weakly NP-hard, as shown by [Talmon and Faliszewski \(2019\)](#), with one exception: the approval-based cardinality rule MAXCARD , which is polynomial-time solvable due to the bounded nature of cardinality-based approval utilities. For this reason, approximate welfare maximization is often turned to in practice.

5.1.2 Approximate Welfare Maximization

In this section, we introduce approximate welfare-maximization rules from the PB literature. To do this, we begin by introducing a general greedy scheme, first introduced in the Participatory Budgeting context by [Talmon and Faliszewski \(2019\)](#), under the name "proportional greedy scheme". Here, "proportional" refers to the fact that we care about utility in proportion to cost. We will refer to this rule as the Greedy Scheme.

Definition 15. (Greedy Scheme) Given an approval based PB instance, and a utility function μ , the greedy scheme $\text{GREED}(I, \mu)$ constructs a budget allocation W iteratively. In each round, let P^* be the set of all the affordable projects that maximize the utility relative to their cost:

$$P^* = \arg \max_{p \in (P \setminus W) : c(W) + c(p) \leq b} \frac{\text{UTIL-}W[\mu](I, \{p\})}{c(p)}$$

If P^* is empty, the procedure stops, and W is the outcome of $\text{GREED}(I, \mu)$. If P^* is nonempty, a project p^* from P^* is selected and added to W , and a new iteration begins. ([Rey and Maly, 2023](#))

The outcome of $\text{GREED}(I, \mu)$ is not guaranteed to be unique, as P^* can contain multiple elements for some rounds. To make the outcome unique, one simply needs to implement some tie-breaking mechanism that decides a single p^* from P^* in each round. By construction, the greedy scheme is exhaustive, as it only terminates when no remaining project can be funded with the remaining budget. It is important to note again that the above scheme is designed for additive utility functions, whereas the greedy scheme for non-additive utility functions looks slightly different. The following results stem from the extensive literature on the knapsack problem.

Proposition 8. (*Dantzig, 1957*) *Given an approval based PB instance, and an additive utility function μ . Let W be an outcome returned by the greedy scheme $\text{GREED}(I, \mu)$. Then either $W = P$, or there exists a project $p \in P \setminus W$ such that:*

$$\sum_{i \in N} \mu_i(W \cup \{p\}) \geq \max_{W': c(W') \leq b} \sum_{i \in N} \mu_i(W').$$

This means that an outcome W returned by the Greedy Scheme always has maximal utility up to one project.

Proposition 9. (*Kellerer et al., 2004*) *Given an approval based PB instance, and an additive utility function μ . Let W^* be an outcome maximizing utilitarian welfare, let W be an outcome returned by the greedy scheme $\text{GREED}(I, \mu)$, and let $p^* \in P$ be a project maximizing standalone utilitarian welfare:*

$$p^* \in \arg \max_{p \in P} \sum_{i \in N} \mu_i(p).$$

Then

$$2 \cdot \max \left\{ \sum_{i \in N} \mu_i(W), \sum_{i \in N} \mu_i(p^*) \right\} \geq \sum_{i \in N} \mu_i(W^*).$$

Equivalently, the better of the greedy outcome and the single project with the highest standalone utilitarian welfare is a factor-2 approximation of the optimal utilitarian welfare.

The proposition accounts for the edge case where the greedy scheme alone fails to achieve a 2-approximation: a single expensive project may have the highest total utility, yet its cost renders all other projects unaffordable, causing the greedy scheme to miss it entirely, an example can be seen in the proof for Proposition 7. By taking the better of the greedy outcome W and the standalone maximizer p^* , we can modify GREED into a true factor-2 approximation. Formally, define the modified outcome as:

$$W' = \arg \max_{W'' \in \{W, \{p^*\}\}} \sum_{i \in N} \mu_i(W'')$$

Then by Proposition 9, W' satisfies $2 \cdot \sum_{i \in N} \mu_i(W') \geq \sum_{i \in N} \mu_i(W^*)$, guaranteeing a 2-approximation.

Because we assume the utility function is additive, we can reformulate the greedy scheme to select projects according to a given ordering. We introduce the ordinal variant of the greedy scheme.

Definition 16. (Ordinal Greedy Scheme). Consider an approval based PB instance and a strict ordering $\triangleright$ over P . The ordinal greedy scheme $\text{GREED}_{\triangleright}(I)$ is a procedure selecting a budget allocation W iteratively. The budget allocation W is initially empty. Projects are considered in the order defined by $\triangleright$. When project p is being considered for the current budget allocation W , p is selected if and only if $c(W \cup \{p\}) \leq b$. If there is a next project according to $\triangleright$, it is considered, otherwise W is the output of $\text{GREED}_{\triangleright}(I)$.

Note that the winning budget allocation produced by $\text{GREED}_{\triangleright}$ is always unique as opposed to GREED . The procedures GREED and $\text{GREED}_{\triangleright}$ are connected in the following way. Consider an election instance I . An ordering of the projects in P , denoted $\triangleright$, is said to be compatible with μ if for projects $p, p' \in P$ we have that:

$$p \triangleright p' \implies \frac{\sum_{i \in N} \mu_i(p)}{c(p)} \geq \frac{\sum_{i \in N} \mu_i(p')}{c(p')}$$

Finally, we have that:

$$\text{GREED}(I, \mu) = \{\text{GREED}_{\triangleright}(I) \mid \triangleright \text{ is compatible with } \mu\}.$$

Algorithmically, the function can be written as:

Algorithm 2: Greedy Scheme Ordinal

Input: Approval based PB instance I and utility function μ

Output: election outcome W .

```

1 Function greedy_scheme_ordinal( $I, \mu$ )
2    $sorted \leftarrow \arg \text{sort}_{p \in P} \frac{-\text{UTIL-}W[\mu](I, \{p\})}{c(p)}$             $\triangleright$  Sort with largest util first
3    $W \leftarrow \emptyset$ 
4   foreach  $p$  in  $sorted$  do
5     if  $b \geq c(p)$  then
6        $W \leftarrow W \cup \{p\}$ 
7        $b \leftarrow b - c(p)$ 
8   return  $W$ 
    
```

Where the sorting knows how to tie-break, and has the largest utilities first. We can show that the greedy welfare scheme, like welfare maximization, does not satisfy μ -PJR.

Proposition 10. *Given an approval based PB instance, and an additive utility function μ . Then the greedy scheme $\text{GREED}(I, \mu)$ does not satisfy μ -PJR in general.*

Proof. We show this by constructing an election instance in which the greedy outcome violates μ -PJR. The construction follows the same general idea as Proposition 6. Consider an instance with

n voters and budget $b = n$. There are $n + 1$ projects $p_1, p_2, \dots, p_{n+1}$ each with cost $c(p_i) = 1$ for $i \in \{1, \dots, n + 1\}$. Say voter 1 approves only p_1 while every remaining voter approves all other projects:

$$A_1 = \{p_1\}, \quad A_2 = \dots = A_n = \{p_2, \dots, p_{n+1}\}$$

Because approved projects yield positive utility, we can choose n large enough such that every project p_i , for $i \geq 2$, has strictly larger total utility than p_1 :

$$\sum_{j=2}^n \mu_j(p_i) > \mu_1(p_1)$$

As all projects have the same cost, this also means that every project p_i , with $i \geq 2$, has a strictly larger utility per cost ratio than p_1 :

$$\frac{\sum_{j=2}^n \mu_j(p_i)}{c(p_i)} > \frac{\mu_1(p_1)}{c(p_1)}$$

The greedy scheme will therefore select the projects $p_2, \dots, p_{n+1}$ before considering p_1 . These n projects together will use the entire budget, meaning the greedy outcome will be $W = \{p_2, \dots, p_{n+1}\}$. We now show that this outcome violates μ -PJR. Consider the voter group $S = \{1\}$ and the project set $T = \{p_1\}$. The group is T -cohesive as voter 1 approves p_1 and:

$$\frac{|S|}{|N|} \cdot b = \frac{1}{n} \cdot n = 1 = c(T)$$

The greedy outcome, however, contains no project approved by voter 1, meaning the cohesive group S receives no utility from W but would receive positive utility from T . Formally:

$$\mu\left(W \cap \bigcup_{i \in S} A_i\right) = \mu(\emptyset) = 0 < \mu(T)$$

Therefore, W violates μ -PJR, showing that the greedy scheme does not satisfy μ -PJR in general. $\square$

We will now shift our focus to the two welfare-maximizing approximation rules most commonly discussed in the literature and in practice. These being the greedy approximations of cardinality-utility and cost-utility, which were originally introduced by (Talmon and Faliszewski, 2019). We will call these two rules GREEDCARD and GREEDCOST respectively. Because both $\mu^\#$ and μ^c are additive utility functions, we can apply them using the greedy ordinal scheme. The rule GREEDCARD will then select projects in order of their total approval score divided by their cost, and GREEDCOST will simply select projects in the order of their total approval score. Note that GREEDCOST is actually the rule most widely used in practice, with an ordinal greedy implementation (Rey and Maly, 2023).

Proposition 11. *GREEDCARD and GREEDCOST runs in polynomial time.*

Proof. The computational complexity of GREEDCARD and GREEDCOST using the ordinal scheme is simple to analyze. First, each of the m projects is mapped to its total voter utility. Assuming we can compute the utility $\mu_i(p)$ in $O(1)$, we can find the utility of each project in complexity $O(nm)$. Then, sorting the projects by utility takes $O(m \log m)$. Finally, we iterate over this ordering and construct the budget allocation W in $O(m)$ time. This yields a total complexity of $O(nm + m \log m)$. $\square$

In practice this bound simplifies to $O(nm)$ since $m \leq n$ for all real elections. This makes the approximate welfare maximization using the greedy scheme a feasible approach on real elections, which is also why it is the most widely used practice. Of 1140 real-world approval elections conducted mainly in Poland, 1134 were completed using some variant of the greedy scheme (Pabulib, 2026).

Example 9 (Approximate Welfare Maximizing Rules). Consider an approval based PB instance with budget $b = 12$.

	p_1	p_2	p_3	p_4
Cost	1	2	3	9
A_1	✓	✓	✓	✓
A_2	✓		✓	✓
A_3	✓			✓
A_4				✓

For I and A , we consider the greedy approximation of utilitarian welfare, for utility functions $\mu^\#$ and μ^c . Because these functions are additive, we can consider the ordinal greedy scheme. We present the ordering of the projects used for the procedure, along with the social welfare achieved by the rule. The maximum social welfare achieved by the exact rule is also shown for comparison.

	Ordering	Outcome	Achieved Social Welfare	Maximal Social Welfare
$\mu^\#$	$p_1 \triangleright p_3 \triangleright p_2 \triangleright p_4$	$\{p_1, p_2, p_3\}$	6	8
μ^c	$p_4 \triangleright p_1 \triangleright p_3 \triangleright p_2$	$\{p_1, p_2, p_4\}$	41	42

5.2 The Method of Equal Shares

The idea of the Method of Equal Shares (MES) was originally introduced under a different name, Rule X, by (Peters and Skowron, 2020) in the context of approval-based committee elections. It was later generalized to the participatory budgeting setting by (Peters et al., 2021).

MES is one of the most prominent rules currently proposed for PB due to its practical applicability and theoretical results.

Definition 17 (Method of Equal Shares for approval ballots). Given an approval based PB instance, and utility function μ , $\text{MES}[\mu]$ produces an outcome W iteratively by the following procedure.

For each voter $i \in N$, we define their load $\ell_i : 2^P \rightarrow \mathbb{R}_{\geq 0}$, initially $\ell_i(\emptyset) = 0$. The load represents how much virtual money a voter has spent. Next, given a current outcome W and scalar $\alpha \geq 0$, the contribution of a voter for project $p \in P \setminus W$ is defined as

$$\gamma_i(W, \alpha, p) = \min(b/n - \ell_i(W), \alpha \cdot \mu_i(p)).$$

Notice that $\gamma_i(W, \alpha, p) = 0$ if $A_i(p) = 0$.

For current outcome W , a project $p \in P \setminus W$ is said to be α -affordable if

$$\sum_{i \in N} \gamma_i(W, \alpha, p) = c(p).$$

i.e., the voters can contribute enough to pay for p .

Initially W is empty. At a given round, if no project is α -affordable, $\text{MES}[\mu]$ terminates. Otherwise, pick the project that is α -affordable for minimal α and add it to W . The loads of the voters are updated to

$$\ell_i(W \cup \{p\}) = \ell_i(W) + \gamma_i(W, \alpha, p),$$

and a new round starts.

Intuitively, when MES adds a project p to the outcome, all voters who approve of p must pay in proportion to their utility. α can be thought of as the cost per unit of utility. If a voter cannot afford to pay for p , the voter must pay the remaining budget that they have left, i.e., $b/n - \ell_i(W)$. This means that MES favors projects that can be funded at a low cost per unit of utility, taking into account the remaining budgets of their supporters (Peters et al., 2021). Since α increases throughout the the execution of MES, projects are added in decreasing order of utility per cost.

Example 10 (Method of Equal Shares). Consider the running example again from Table 1. We will use $\mu = \mu^\#$ and make the following definitions:

- Available contribution: for voter i this is defined as $A_i(p) \cdot (b/n - \ell_i(W))$, i.e. voter i 's remaining budget share if i approves of p and 0 otherwise.

- **Affordable:** Boolean indicating whether supporters of project p have enough budget to afford p
- **Full price:** Defined as $\max_{i \in N} \gamma_i(W, \alpha, p)$, i.e., the highest price any voter pays for p
- **Full price per utility:** The price per unit of utility paid by those paying full price. Using $\mu^\#$ means that this is simply the same as full price. Since projects are added in decreasing order of utility per cost, the project with the smallest full price per utility will be picked in a given round.

These definitions are inspired by (Rey and Maly, 2023). Let's look at the $\text{MES}[\mu^\#]$ execution:

	p_1	p_2	p_3	p_4
Cost	2	4	3	11
Utility	1	1	1	1
Available Contribs.	0, 0, 5	5, 5, 5	0, 0, 5	5, 5, 5
Affordable	✓	✓	✓	✓
Full price (per util.)	2	$\frac{4}{3}$	3	$\frac{11}{3}$
Available Contribs.	0, 0, $\frac{11}{3}$	—	0, 0, $\frac{11}{3}$	$\frac{11}{3}, \frac{11}{3}, \frac{11}{3}$
Affordable	✓	—	✓	✓
Full price (per util.)	2	—	3	$\frac{11}{3}$
Available Contribs.	—	—	0, 0, $\frac{5}{3}$	$\frac{11}{3}, \frac{11}{3}, \frac{5}{3}$
Affordable	—	—	×	×
Selected	✓	✓	×	×

The example shows that $\text{MES}[\mu^\#]$ will return outcome $W = \{p_1, p_2\}$. Notice that the outcome is not exhaustive, since p_3 could be added to the outcome while maintaining feasibility.

Since MES is defined as an iterative procedure, its computational efficiency follows directly from its structure (Peters and Skowron, 2020):

Proposition 12. *MES runs in polynomial time.*

Proof. In each round, MES selects the most affordable project by iterating over all remaining projects and their approvers. Since at most $m = |P|$ rounds are performed, each taking $O(m \cdot n)$ time, the total runtime is $O(m^2 \cdot n)$. $\square$

MES is not appealing only due to its polynomial runtime, it also comes with strong theoretical proportionality guarantees, which we now examine in greater detail.

Theorem 4. *MES is priceable.*

Proof. Let $W = \{p_1, \dots, p_k\}$ be the outcome of running $\text{MES}[\mu^\#]$ on some approval-based PB instance I . Define $W_t = \{p_1, \dots, p_t\}$ with $W_0 = \emptyset$ and let α_t be the minimum α for which project p_t was α -affordable when selected.

From these definitions we construct a price system $(\alpha^*, (\gamma_i)_{i \in N})$ that satisfy conditions C1-C5.

Let $\alpha^* = b/n$ and let

$$\gamma_i(p) = \begin{cases} \gamma_i(W_{t-1}, \alpha_t, p_t) & \text{if } p \in W \\ 0 & \text{otherwise} \end{cases}$$

where t is the round p was added to W . This means that $\gamma_i(p)$ defines how much voter i paid for project p in the round that it was added.

- C1** (*Voters only contribute to projects with positive utility*): We need $\gamma_i(p) > 0 \implies \mu_i(p) > 0$ for all $p \in P, i \in N$. We have for any $p \in P, i \in N : \mu_i(p) = 0 \implies \gamma_i(p) = 0$, which is the contrapositive of the required condition.
- C2** (*Only projects in W receive funding*): We need $\gamma_i(p) > 0 \implies p \in W$ for all $p \in P, i \in N$. This follows directly from the definition of $\gamma_i(p)$, since $\gamma_i(p) = 0$ for all $p \notin W$.
- C3** (*No voter exceeds their entitlement*): We need $\sum_{p \in P} \gamma_i(p) \leq \alpha^*$ for all $i \in N$. This holds trivially, since voter i 's total spending cannot exceed the initial budget $b/n = \alpha^*$.
- C4** (*All projects in W are fully funded*): We need $\sum_{i \in N} \gamma_i(p) = c(p)$ for all $p \in W$. This follows directly from the affordability condition in MES: when p_t is selected, α_t is chosen as the minimum value such that $\sum_{i \in N} \gamma_i(W_{t-1}, \alpha_t, p_t) = c(p_t)$.
- C5** (*No group of supporters of an unselected project has more remaining budget than its cost*): We need $\sum_{i \in N: A_i(p)=1} (\alpha^* - \sum_{p' \in P} \gamma_i(p')) < c(p)$ for all $p \in P \setminus W$. Since $p \notin W$, it was never selected by MES, so $\sum_{i \in N} \gamma_i(W, \alpha, p) < c(p)$. By definition, for $i \in N$ with $A_i(p) = 1$, we have $\mu^\#(p) = 1$:

$$\gamma_i(W, \alpha^*, p) = \min(\alpha^* - \ell_i(W), \mu_i(p) \cdot \alpha^*) = b/n - \ell_i(W).$$

Combining these, we get

$$\sum_{i \in N: A_i(p)=1} \left(\alpha^* - \sum_{p' \in P} \gamma_i(p') \right) = \sum_{i \in N: A_i(p)=1} b/n - \ell_i(W) = \sum_{i \in N: A_i(p)=1} \gamma_i(W, \alpha^*, p) < c(p).$$

□

Theorem 5. $\text{MES}[\mu^\#]$ satisfies μ -PJR-x for all DNS functions μ

Proof Outline. The outline follows the argument of (Brill et al., 2023).

The key idea is to relate μ -PJR-x to C6-priceability. In particular, it is shown (Theorem 4.5 in the paper) that any outcome that is C6-priceable with $\alpha \cdot n > b$ satisfies μ -PJR-x for all DNS functions μ .

It is therefore enough to show that $\text{MES}[\mu^\#]$ always returns such outcomes. This is established in Corollary 4.6 of the paper. Combining these results, we conclude that $\text{MES}[\mu^\#]$ satisfies μ -PJR-x for all DNS functions μ .

□

Theorem 6. Let μ be a DNS function. Then $\text{MES}[\mu]$ satisfies μ -EJR-x. (Brill et al., 2023)

Proof. Consider an approval based PB instance and let $W = \{p_1, \dots, p_k\}$ be the ordered outcome returned by $\text{MES}[\mu]$, where p_1 is selected first, p_2 second, and so on. For $1 \leq j \leq k$, write $W_j := \{p_1, \dots, p_j\}$.

Let $N' \subseteq N$ be a T -cohesive group. If $T \subseteq W$ then μ -EJR-x is satisfied for N' by definition, so assume $T \not\subseteq W$.

First, define p^* to be the *cheapest* project in $T \setminus W$, i.e.,

$$p^* = \arg \min \{c(p) \mid p \in T \setminus W\}.$$

Let k^* be the first round after which some voter $i^* \in N'$ has load exceeding $\frac{b}{n} - \frac{c(p^*)}{|N'|}$. Such a round must exist, because otherwise the voters in N' could still collectively afford p^* throughout the entire run of $\text{MES}[\mu]$, contradicting termination. Set $W^* := W_{k^*}$.

Our goal is to show that there exists $i^* \in N'$ such that

$$\mu_{i^*}(W^* \cup \{p^*\}) > \mu_{i^*}(T). \quad (1)$$

Since μ is additive, (1) is equivalent to

$$\begin{aligned} &\iff \mu_{i^*}(W^*) > \mu_{i^*}(T \setminus \{p^*\}) \\ &\iff \mu_{i^*}(W^* \cap T) + \mu_{i^*}(W^* \setminus T) > \mu_{i^*}(T \cap W^*) + \mu_{i^*}(T \setminus (W^* \cup \{p^*\})) \\ &\iff \mu_{i^*}(W^* \setminus T) > \mu_{i^*}(T \setminus (W^* \cup \{p^*\})). \end{aligned} \quad (2)$$

Once we have established (1) for p^* , it follows for *every* $p \in T \setminus W$ since μ is additive and the first DNS condition gives $\mu(p^*) \leq \mu(p)$ for all $p \in T \setminus W$ (as $c(p^*) \leq c(p)$ by choice of p^*), we have

$$\mu_{i^*}(W^* \cup \{p\}) \geq \mu_{i^*}(W^* \cup \{p^*\}) > \mu_{i^*}(T),$$

so μ -EJR-x holds for N' . It therefore suffices to prove (2).

Now we need to introduce some notation. For a project $p \in W$, let $\alpha(p)$ denote the smallest $\alpha > 0$ for which p was α -affordable when $\text{MES}[\mu]$ selected it, and set $q(p) := 1/\alpha(p)$ (utility per unit of load). Let $\gamma_{i^*}(W^*, \alpha, p)$ denote the contribution of i^* to project $p \in W^*$.

First, we bound the left hand side of (2). We have

$$\begin{aligned} \mu_{i^*}(W^* \setminus T) &= \sum_{p \in W^* \setminus T} \mu_{i^*}(\{p\}) \\ &= \sum_{p \in W^* \setminus T} \alpha(p) \cdot \mu_{i^*}(\{p\}) \cdot \frac{1}{\alpha(p)} \\ &\geq \sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p) \cdot q(p). \end{aligned} \tag{3}$$

Let $q_{\min} = \min_{p \in W^* \setminus T} q(p)$, then by combining with (3) we get

$$\mu_{i^*}(W^* \setminus T) \geq \sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p) \cdot q(p) \geq q_{\min} \sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p),$$

Next, we bound the right-hand side of (2). For $p \in T$, let

$$q^*(p) := \frac{\mu(p)}{c(p)/|N'|} = |N'| \frac{\mu(p)}{c(p)}.$$

be the utility-per-load that a voter in N' receives from p if all of N' funds p in full. We get

$$\begin{aligned} \mu_{i^*}(T \setminus (W^* \cup \{p^*\})) &= \sum_{p \in T \setminus (W^* \cup \{p^*\})} \mu_{i^*}(p) \\ &= \sum_{p \in T \setminus (W^* \cup \{p^*\})} \frac{\mu_{i^*}(p)}{c(p)/|N'|} \cdot c(p)/|N'| \\ &= \sum_{p \in T \setminus (W^* \cup \{p^*\})} q^*(p) \cdot \frac{c(p)}{|N'|}. \end{aligned} \tag{4}$$

Define $q_{\max}^* := \max_{p \in T \setminus (W^* \cup \{p^*\})} q^*(p)$, then by combining with (4) we get

$$\mu_{i^*}(T \setminus (W^* \cup \{p^*\})) = \sum_{p \in T \setminus (W^* \cup \{p^*\})} q^*(p) \cdot \frac{c(p)}{|N'|} \leq q_{\max}^* \cdot \frac{c(T \setminus (W^* \cup \{p^*\}))}{|N'|}.$$

It therefore suffices to show

$$q_{\min} \sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p) > q_{\max}^* \cdot \frac{c(T \setminus (W^* \cup \{p^*\}))}{|N'|}. \quad (5)$$

Claim 1: $q_{\min} \geq q_{\max}^*$

By definition of k^* , voters in N' could fully afford any project in $T \setminus W^*$ throughout rounds $1, \dots, k^*$, which means that each such project $p' \in T \setminus (W^* \cup \{p^*\})$ was $\frac{1}{q^*(p')}$ -affordable in each of these rounds. Since $\text{MES}[\mu]$ always selects the most affordable project, every project already selected must have been at least as affordable. Thus, we have for all $p' \in T \setminus (W^* \cup \{p^*\})$ that $\alpha(p^*) \leq \frac{1}{q^*(p')} \iff q(p^*) \geq q^*(p')$, from which $q(p^*) \geq q_{\max}^*$ follows. Also we must have $\alpha(p^*) \geq \min_{p \in W^* \setminus T} \alpha(p) \iff q(p^*) \leq q_{\min}$ using the same general logic. Combining these, we get $q_{\min} \geq q_{\max}^*$.

Claim 2: $\sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p) > \frac{c(T \setminus (W^* \cup \{p^*\}))}{|N'|}.$

From the definition of k^* , the load of i^* after round k^* satisfies (simple rewrite of the definition of k^*)

$$\ell_{i^*}(W^*) + \frac{c(p^*)}{|N'|} > \frac{b}{n}.$$

Since N' is T -cohesive, $c(T)/|N'| \leq b/n$, so

$$\ell_{i^*}(W^*) > \frac{c(T)}{|N'|} - \frac{c(p^*)}{|N'|} = \frac{c(T \setminus \{p^*\})}{|N'|} = \frac{c(T \cap W^*)}{|N'|} + \frac{c(T \setminus (W^* \cup \{p^*\}))}{|N'|}.$$

By the definition of load, we can also write the above as

$$\begin{aligned} \ell_{i^*}(W^*) &= \sum_{p \in W^*} \gamma_{i^*}(W^*, \alpha, p) \\ &= \sum_{p \in T \cap W^*} \gamma_{i^*}(W^*, \alpha, p) + \sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p) \\ &> \frac{c(T \cap W^*)}{|N'|} + \frac{c(T \setminus (W^* \cup \{p^*\}))}{|N'|}. \end{aligned} \quad (6)$$

By definition of cohesive groups, it follows that the contribution of every voter in N' for a project $p \in T \cap W^*$ (since they approve all projects in this set) is smaller than or equal to the contribution needed if the voters in N' would fund the project by themselves. In other words, we have

$$\gamma_{i^*}(W^*, \alpha, p) \leq \frac{c(p)}{|N'|},$$

for all $p \in T \cap W^*$.

From here we can bound the first sum on the left hand side of (6) as

$$\sum_{p \in T \cap W^*} \gamma_{i^*}(W^*, \alpha, p) \leq \frac{c(T \cap W^*)}{|N'|}.$$

For (6) to hold, we must have

$$\sum_{p \in W^* \setminus T} \gamma_{i^*}(W^*, \alpha, p) > \frac{c(T \setminus (W^* \cup \{p^*\}))}{|N'|}.$$

Combining Claims 1 and 2 establishes (5), which in turn gives (2) and hence (1). As argued above, (1) for the cheapest $p^* \in T \setminus W$ implies the same inequality for every $p \in T \setminus W$, so μ -EJR-x is satisfied for the arbitrarily chosen cohesive group N' . $\square$

Since μ -EJR-x strengthens μ -EJR-1, and $\mu^\#$ is a DNS utility function, Theorems 3 and 6 immediately give us the following clean conclusion.

Corollary 3. $\text{MES}[\mu^\#]$ satisfies $\mu^\#$ -EJR.

5.3 The Sequential Phragmén Rule

The Sequential Phragmén Rule takes its name from the Swedish mathematician Lars Edvard Phragmén (Janson, 2016), who first presented the rule from which it is inspired and adapted. It was studied in the multi-winner setting by (Brill et al., 2017) and later adapted to the PB setting by (Los et al., 2022). Importantly, this rule is only applicable to the approval setting.

The rule has two equivalent formulations: a continuous one and a discrete one. We present the discrete formulation given by (Brill et al., 2023).

Definition 18 (The Sequential Phragmén Rule, Discrete Formulation). Given an approval based PB instance, the sequential Phragmén rule, SEQPHRAG , produces an outcome W iteratively as follows. For each voter $i \in N$, we define their load $\ell_i : 2^P \rightarrow \mathbb{R}_{\geq 0}$, initially $\ell_i(\emptyset) = 0$. At a given round with current outcome W , the maximum load for selecting a project $p \in P \setminus W$ is defined as

$$\ell^*(W, p) = \frac{c(p) + \sum_{i \in N} A_i(p) \cdot \ell_i(W)}{|\{i \in N \mid A_i(p) = 1\}|},$$

also for a given round with current outcome W , we define $P^* \subseteq P$ as

$$P^* = \operatorname{argmin}_{p \in P \setminus W} \ell^*(W, p).$$

SEQPHRAG terminates and outputs W if there exists $p \in P^*$ with $c(W \cup \{p\}) > b$. Otherwise, select a project $p \in P^*$ (using some tiebreaker rule for resoluteness), and the voters' loads are updated as follows:

$$\ell_i(W \cup \{p\}) = \begin{cases} \ell_i(W) & \text{if } A_i(p) = 0 \\ \ell^*(W, p) & \text{otherwise} \end{cases}.$$

p is added to W and a new round starts.

Intuitively, SEQPHRAG aims to equalize the *load* placed on voters, where load represents how much of the budget each voter is responsible for funding. When a project is selected, its cost is distributed equally among its supporters, and the rule always selects whichever affordable project minimizes the maximum load any voter must bear. This gives the rule a natural proportionality aspect: cohesive groups of voters who share preferences will collectively fund their projects, without being forced to subsidize projects they do not support.

Notably, SEQPHRAG takes no utility function as input, unlike MES and Greedy, which are explicitly parameterized by $\mu^\#$ or μ^c . In a sense, the load mechanism implicitly treats each project as equally valuable to its supporters, which sits closest to a cardinality-based view of utility: what matters is whether a voter approves a project, not how costly it is. This stands in contrast to $\text{MES}[\mu^c]$, which weights representation by project cost, and $\text{MES}[\mu^\#]$, which weights by raw approval count. Whether this implicit cardinality bias helps or hurts SEQPHRAG on μ^c -EJR is therefore not clear.

Example 11 (The Sequential Phragmén Rule). Consider again the running example with $b = 15$ (same as Table 1, but repeated here for easier lookup)

	p_1	p_2	p_3	p_4
cost	2	4	3	11
A_1		✓		✓
A_2		✓		✓
A_3	✓	✓	✓	✓

Let's look at the execution of SEQPHRAG on this instance. For each round, we indicate the maximum load each project would incur. The smallest such load is highlighted in blue.

	p_1	p_2	p_3	p_4
Cost	2	4	3	11
$\ell^*(\emptyset, p)$	2	$\frac{4}{3}$	3	$\frac{11}{3}$
$\ell^*({p_2}, p)$	$2 + \frac{4}{3} = \frac{10}{3}$	—	$3 + \frac{4}{3} = \frac{13}{3}$	$\frac{11 + 3 \cdot \frac{4}{3}}{3} = 5$
$\ell^*({p_1, p_2}, p)$	—	—	$3 + \frac{10}{3} = \frac{19}{3}$	$\frac{11 + \frac{10}{3} + 2 \cdot \frac{4}{3}}{3} = \frac{17}{3}$
Selected	✓	✓	✗	✗

In the final row we get $P^* = \{p_4\}$ but since $c(\{p_1, p_2\} \cup \{p_4\}) = 17 > 15$, SEQPHRAG terminates and returns $W = \{p_1, p_2\}$.

Unfortunately, SEQPHRAG offers no guarantees with respect to EJR or its relaxations:

Proposition 13. *The sequential Phragmén rule does not satisfy μ -EJR-1 for either cardinality or cost-based utility.*

Proof. We give a counterexample in which every project has unit cost, so that cardinality and cost-based utility coincide, inspired by (Brill et al., 2017).

Consider an approval based PB instance with $|N| = 24$, $b = 12$, and project set

$$P = \{a, b, p_1, p_2, \dots, p_{12}\}.$$

All projects have unit cost, so $c(p) = 1$ for every $p \in P$. The approval profile is given by

$$A_1 = A_2 = \{a, b, p_1\},$$

$$A_3 = A_4 = \{a, b, p_2\},$$

$$A_5 = \dots = A_{10} = \{p_1, p_2, \dots, p_{12}\},$$

$$A_{11} = \dots = A_{15} = \{p_2, p_3, \dots, p_{12}\},$$

$$A_{16} = \dots = A_{24} = \{p_3, p_4, \dots, p_{12}\}.$$

On this instance, sequential Phragmén returns the outcome

$$W = \{p_1, p_2, \dots, p_{12}\}.$$

We now show that this outcome violates μ -EJR-1. Consider the voter group

$$N' = \{1, 2, 3, 4\}$$

and the project set

$$T = \{a, b\}.$$

The group N' is T -cohesive, since every voter in N' approves both projects in T , and

$$c(T) = 2 \leq b \frac{|N'|}{|N|} = 12 \cdot \frac{4}{24} = 2.$$

However, in the outcome W , voters 1 and 2 only approve the selected project p_1 , while voters 3 and 4 only approve the selected project p_2 . Hence, for every voter $i \in N'$,

$$\mu_i(W) = 1,$$

whereas

$$\mu_i(T) = 2.$$

Since $T \not\subseteq W$, the μ -EJR-1 condition requires that there exists a voter $i \in N'$ and a project $p \in T \setminus W$ such that

$$\mu_i(W \cup \{p\}) > \mu_i(T).$$

But $T \setminus W = \{a, b\}$, and adding either missing project gives every voter in N' utility exactly 2:

$$\mu_i(W \cup \{p\}) = 2 = \mu_i(T)$$

for every $i \in N'$ and every $p \in T \setminus W$. Since the definition uses a strict inequality, this is not sufficient. Therefore W violates μ -EJR-1. $\square$

Since SEQPHRAG already fails μ -EJR-1 in this example, no EJR guarantee of any strength can be expected from the rule in general.

SEQPHRAG does however still have several desirable proportionality properties.

Proposition 14 (Los et al., 2022). *For every approval-based PB instance, SEQPHRAG is priceable.*

Priceability comes at a cost: by Proposition 1, it immediately follows that SEQPHRAG is not exhaustive, which is also seen in Example 11.

Also, SEQPHRAG gives the same strong PJR guarantee as MES provides:

Proposition 15. (*Brill et al., 2023*) $SEQPHRAG$ satisfies μ -PJR- x for all DNS functions μ .

Proof Outline. The idea is to show that $SEQPHRAG$ produces a $C6$ -priceable outcome. Combined with Theorem 4.5 of (*Brill et al., 2023*), which states that any $C6$ -priceable outcome satisfies μ -PJR- x for all DNS functions μ , the result follows. $\square$

The iterative structure of $SEQPHRAG$ closely resembles that of MES , which similarly yields a polynomial-time algorithm.

Proposition 16. (*Peters and Skowron, 2020*) $SEQPHRAG$ can be computed in polynomial time in the approval based PB setting.

Proof. At most $m = |P|$ rounds are performed, and in each round the maximum load is calculated for each project in time $O(n)$, giving a total running time of $O(m^2 \cdot n)$. $\square$

5.4 Completion Rules

As discussed above, an outcome returned by a PB rule is not guaranteed to be exhaustive. This is not only a problem in theory, as rules such as Method of Equal Shares and Phragmén’s rule, which we discuss above, are non-exhaustive. In fact, statistics from running Method of Equal Shares on real election instances from *Pabulib* (2026) show that only 7% of election outcomes are exhaustive, and that the average budget spending is only around 45% (*Papasotiropoulos et al., 2025*). If we wish to enforce exhaustiveness, we can apply a *completion rule*. A completion rule modifies the result of a possibly non-exhaustive rule in order to obtain, or come closer to obtaining, an exhaustive outcome. This can be done either as a post-processing step, where additional feasible projects are added to the initially computed outcome W , or by modifying the execution of the underlying rule itself, for example by changing the voter endowments and rerunning the rule. Below we present the completion rules discussed by *Faliszewski et al. (2023)*.

The most direct completion method is **Utilitarian Greedy Completion (U)**. This method can be applied to any non-exhaustive outcome. After an initial outcome W has been computed, this rule repeatedly selects a project $p \in P \setminus W$ satisfying $c(p) \leq b - c(W)$, which maximizes the utility gained per unit of cost:

$$\frac{\sum_{i \in N} \mu_i(p)}{c(p)}$$

The selected project is added to W , and the procedure is repeated until no further project can be added without exceeding the budget. For cardinality-based utility, this corresponds to selecting the affordable project with the highest number of approvals per unit of cost. For cost-based utility, the cost term simply cancels out and the rule selects an affordable project with the highest number of voter approvals.

Another completion method is **Budget Expansion Completion (Add1)**. Unlike utilitarian greedy completion, Add1 is mainly defined for the Method of Equal Shares and related rules with voter endowments. This rule first runs $\text{MES}[\mu]$ with initial voter endowment:

$$b_i = \frac{b}{|N|}$$

For each voter $i \in N$. If the resulting outcome is not exhaustive, the voter endowment is increased by one unit, and MES is run again. This process is repeated until an exhaustive outcome is obtained, or until increasing the endowment further would lead to an infeasible outcome. In the latter case, the rule returns the feasible outcome obtained using the largest endowment considered. Thus, the final outcome W' is not guaranteed to be exhaustive, the procedure only guarantees that the returned outcome is the feasible outcome obtained using the largest endowment value considered. Because Add1 often produces outcomes that are close to exhaustive, but does not always fully exhaust the budget, it is commonly combined with utilitarian greedy completion. This combined completion rule is called **Add1U**. First, Add1 is used to obtain an outcome from MES with expanded voter endowments. Then utilitarian greedy completion is applied to add any remaining affordable projects.

Another completion method is **Completion by Tweaking Utilities (EPS)**. This method is based on the observation that equal shares always return an exhaustive outcome if every voter assigns strictly positive utility to every project. The EPS completion method enforces this condition by a slight modification to the utility function before applying a voting rule. For each voter $i \in N$ and project $p \in P$ with $\mu_i(p) = 0$, the utility is replaced by a small value $\epsilon > 0$, while positive utilities are left unchanged. The non-exhaustive voting rule is then applied with this modified utility function. The role of ϵ is only to remove zero utilities, and not to meaningfully change the preferences of the voters. If ϵ is chosen to be sufficiently small, projects that voters originally approved will still dominate projects that only receive artificial utility. In this way, EPS provides another way of obtaining an exhaustive outcome from Equal Shares, although it does so by changing the utility instance rather than by completing a previously computed outcome.

Overall, the experimental results of [Faliszewski et al. \(2023\)](#) suggest that Add1U is the most reliable completion rule for Method of Equal Shares. EPS performs the worst among the three methods in terms of average utility, while utilitarian greedy completion yields a slightly higher average utility than Add1U. However, Add1U performs better overall for cost-based utilities and comes very close to exhausting the available budget. Based on these observations, Add1U is generally adopted as the default completion rule.

5.5 Summary of Axiomatic Guarantees

We conclude this section by summarizing the main proportionality and priceability guarantees discussed above. In particular, it highlights the difference between general existence results, such as greedy cohesive rules, and concrete polynomial-time rules such as MES and SEQPHRAG.

Table 3: Overview of proportionality axioms and rules satisfying them.

Approval Ballots	
Axiom	Rules satisfying the axiom
μ -FJR	For any μ , a greedy cohesive rule.
μ -EJR	A greedy cohesive rule for any μ . Moreover, MES[$\mu^\#$] satisfies EJR.
μ -EJR-x	A greedy cohesive rule for any μ . Moreover, MES[μ] satisfies EJR-x for any DNS function μ .
μ -EJR-1	A greedy cohesive rule for any μ . Moreover, MES[μ] satisfies EJR-1 for any DNS μ .
μ -PJR	For any μ , a greedy cohesive rule.
μ -PJR-x	A greedy cohesive rule for any μ . Moreover, MES[$\mu^\#$] and SEQPHRAG satisfy PJR-x for any DNS function μ .
Priceability	MES[μ] and SEQPHRAG.
Priceability with C6 and $n\alpha > b$	MES[$\mu^\#$] and SEQPHRAG.

6 Verifying Proportionality Axioms

In this section, we study the computational problem of verifying whether a given outcome satisfies a proportionality axiom. This is relevant when no theoretical guarantee is available for the voting rule in question, and one wishes to check proportionality directly on real-world elections. We first show that verification is coNP-complete in general, and then present our own algorithm that performs efficiently in practice for verifying EJR. As we show later in this section, verifying EJR is W[1]-hard with respect to the largest cohesive group size κ , which implies that our algorithm's worst-case runtime of $O(m^\kappa \cdot n\kappa)$ is essentially optimal.

6.1 Verifying Proportionality is Hard

When we do not have theorems guaranteeing that a voting rule satisfies a given proportionality axiom, we wish to verify it directly on real-world elections. However, this turns out to be computationally hard.

Theorem 7. *Checking whether an outcome W satisfies FJR, EJR, or PJR is coNP-complete in each case.*

The hardness results for all three axioms follow from reductions from BALANCED BICLIQUE in the multi-winner voting setting: for EJR by Aziz et al. (2017), for PJR by Aziz et al. (2018a), and for FJR by Kalayci et al. (2025). Since multi-winner voting is a special case of approval based PB, all three hardness results carry over directly. The proofs for PJR and FJR follow the same structure as the EJR proof we present below.

Proposition 17. *Given an approval based PB instance, and an outcome W , it is coNP-complete to check whether W satisfies μ -EJR for either cost-based or cardinality-based utility.*

Proof. It is easy to see that this problem is in coNP: a violation can be certified in polynomial time given the T -cohesive group N' acting as violation witness, and verifying that every voter in N' is unsatisfied.

To prove coNP-hardness, we adapt the reduction of Aziz et al. (2017) to the PB setting. We reduce the classic BALANCED BICLIQUE problem (Garey and Johnson, 1979) to the complement of our problem. An instance of BALANCED BICLIQUE is given by a bipartite graph (L, R, E) with parts L and R and edge set E , and an integer k . It is a YES-instance if we can find subsets $L' \subseteq L$ and $R' \subseteq R$ with $|L'| = |R'| = k$ that form a complete bipartite subgraph, i.e. $(u, v) \in E$ for each $u \in L'$ and $v \in R'$. Otherwise, it is a NO-instance.

Given an instance $\langle (L, R, E), k \rangle$ of BALANCED BICLIQUE with $R = \{v_1, \dots, v_s\}$, we construct a PB instance as follows. Assume without loss of generality that $s \geq 3$ and $k \geq 3$ as smaller values yield trivial instances.

We introduce 4 pairwise disjoint sets of projects P_0, P_1, P'_1, P_2 , with $P_0 = L$, $|P_1| = |P'_1| = k - 1$, $|P_2| = sk + k - 3s$, and set $P = P_0 \cup P_1 \cup P'_1 \cup P_2$. We then construct 3 pairwise disjoint groups of voters N_0, N_1, N_2 , with $N_0 = \{1, \dots, s\}$, $|N_1| = k(s - 1)$, and $|N_2| = sk + k - 3s$ ($|N_2| > 0$ as we assume $k \geq 3$). The approval ballots are defined as follows:

- for each $i \in N_0$: $A_i = \{u \mid (u, v_i) \in E\} \cup P_1$. Note $A_i \subseteq P_0 \cup P_1$,
- for each $i \in N_1$: $A_i = P_0 \cup P'_1$.
- the projects in P_2 are matched one-to-one with voters in N_2 : each voter in N_2 approves exactly one project in P_2 , and each project in P_2 is approved by exactly one voter in N_2 .

Denote the resulting approval profile by $\mathbf{A}$. Finally, we set $b = 2k - 2$, assign unit cost to all projects, and let $W = P_1 \cup P'_1$. Since all projects have unit cost, the cost-based and cardinality-based utility functions coincide. The total number of voters is

$$n = s + k(s - 1) + sk + k - 3s = 2s(k - 1),$$

$$\text{so } \frac{b}{n} = \frac{1}{s}.$$

Case 1: YES Instance of BALANCED BICLIQUE implies EJR violation

Let (L', R') be the respective k -by- k biclique. Set

$$T = L', \quad N'_0 = \{i \in N_0 : v_i \in R'\}, \quad N' = N'_0 \cup N_1.$$

Then $|N'| = k + k(s - 1) = ks$. We claim that N' is T -cohesive:

- Every voter in N'_0 approves all projects in $T = L'$: since (L', R') is a biclique, every $u \in L'$ satisfies $(u, v_i) \in E$ for each $v_i \in R'$, so $L' \subseteq \{u \mid (u, v_i) \in E\} \subseteq A_i$. Every voter in N_1 approves all projects in T since $A_i = P_0 \cup P'_1 \supseteq P_0 = L \supseteq L' = T$.
- The budget share condition is satisfied:

$$c(T) = k = (2k - 2) \cdot \frac{ks}{2s(k - 1)} = b \cdot \frac{|N'|}{n}.$$

We have $\mu_i(T) = k$ for all $i \in N'$. However, for each voter $i \in N'$ we have $\mu_i(W) = k - 1$: voters in N'_0 satisfy $A_i \cap W = P_1$ (as no voter $i \in N'_0$ approves any project in P'_1), and voters in N_1 satisfy $A_i \cap W = P'_1$ (as no voter $i \in N_1$ approves any project in P_1), so in both cases

$$\mu_i(W) = |P_1| = |P'_1| = k - 1 < k = \mu_i(T).$$

Hence no voter in N' satisfies $\mu_i(W) \geq \mu_i(T)$, so W fails μ -EJR with respect to the T -cohesive group N' .

Case 2: EJR violation implies YES instance of BALANCED BICLIQUE

Suppose that W fails to satisfy μ -EJR. That is, there exists a T -cohesive group N' such that no voter $i \in N'$ satisfies $\mu_i(W) \geq \mu_i(T)$. Let $j = |T|$, so $|N'| \geq js$ by the budget share condition $\frac{b}{n} \cdot |N'| \geq c(T) = j$.

Since each voter in N_2 approves exactly one project, any T -cohesive group containing a voter from N_2 would have $|T| = 1$ and $|N'| = 1$, violating $|N'| \geq js \geq s > 1$. Hence $N' \cap N_2 = \emptyset$ and $N' \subseteq N_0 \cup N_1$.

Every voter in $N \setminus N_2$ has $\mu_i(W) = k - 1$ as established earlier, so the violation requires $j \geq k$. Since $|N'| \geq js \geq ks$, we have $|N' \cap N_1| > 0$: otherwise $N' \subseteq N_0$ would give $|N'| \leq s < ks$, a contradiction. Similarly, $|N' \cap N_0| \geq k$: otherwise $|N' \cap N_1| > k(s - 1) = |N_1|$, which is also a contradiction.

Since N' contains voters from both N_0 and N_1 , and T -cohesiveness requires all voters in N' to approve all projects in T , any project in T must be approved by both groups. Voters in N_1 approve only projects in $P_0 \cup P'_1$, and voters in N_0 approve only projects in $P_0 \cup P_1$; since $P_1 \cap P'_1 = \emptyset$, it follows that $T \subseteq P_0$. Thus there are at least k voters in $N' \cap N_0$ who all approve the same $j \geq k$ projects in $P_0 = L$. By construction, voter $i \in N_0$ corresponds to vertex $v_i \in R$, and approves $u \in P_0$ if and only if $(u, v_i) \in E$. Hence any set of k such voters and k such projects corresponds to an $k \times k$ biclique in the input graph, so the input is a YES-instance of BALANCED BICLIQUE. $\square$

6.2 A Naive Verification Algorithm

Despite this worst-case hardness, one can still attempt verification in practice. The Pabulib library includes the following naive implementation, which enumerates all possible project sets and voter groups:

Algorithm 3: Find EJR violation naive

Input: Approval based PB election I , utility function μ , and election outcome W .

Output: Boolean signaling if a violation witness has been found.

```

1 Function find_ejr_violation_naive( $I, \mu, W$ )
2   foreach  $T$  in Powerset ( $P$ ) do
3     foreach  $N'$  in Powerset ( $N$ ) do
4       if is_T_cohesive ( $T, N'$ ) then
5         if  $\mu_i(W) < \mu_i(T) \forall i \in N'$  then
6           return true                                      $\triangleright$  EJR violation
7   return false
    
```

This approach is exponential in both the number of projects and the number of voters, and is therefore infeasible for any real election where the set of voters can be as large as 100,000, and there are up to 200 projects, the largest approval-based election on [Pabulib \(2026\)](#). We now present our own contribution: a more efficient algorithm for verifying EJR, inspired by the Apriori Algorithm.

6.3 The Apriori Algorithm

The Apriori algorithm, first introduced by ([Agrawal and Srikant, 1994](#)), is built on a powerful observation: *any subset of a frequent itemset must itself be frequent*. Equivalently, and more useful in our case, if an itemset is infrequent, then every superset of it must also be infrequent. This is known as the Apriori property.

To see why this matters, consider a small store selling five items: {bread, butter, milk, eggs, jam}. Suppose we scan all transactions and find that bread, butter, milk is infrequent, i.e., the combination of these items appears in too few baskets to be interesting. Then we immediately know that {bread, butter, milk, eggs} and {bread, butter, milk, jam} (and any other superset) cannot be frequent either, so we never need to count them. Infrequent itemsets are pruned.

The algorithm exploits this by generating candidate itemsets of size k using only the frequent itemsets of size $k - 1$, deleting any candidate that contains a subset known to be infrequent. This means an exponentially large space of possible item combinations is never fully enumerated, large portions of it are discarded before any counting takes place.

In the worst case, when close to all itemsets are frequent, little pruning occurs, and the search space remains exponential in the number of items. In practice, however, a realistic threshold tends to produce enough infrequent itemsets early on to prune the candidate space aggressively, and the algorithm runs far more efficiently than worst-case analysis would suggest.

6.4 An Efficient EJR Verification Algorithm

The naive implementation has two issues: it is combinatorial in both projects and voters. We here present an algorithm for finding violations, which:

- Has polynomial complexity with respect to voters.
- Significantly reduces the search space for project sets.

First, we define the following notation:

Let $N^*(T) \subseteq N$ be the largest set of voters that approve all projects for some $T \subseteq P$, i.e.

$$N^*(T) = \{i \in N \mid T \subseteq A_i\}.$$

Let $U(T) \subseteq N^*(T)$ be the largest set of all unsatisfied, but approving, voters for some $T \subseteq P$, i.e.

$$U(T) = \{i \in N^*(T) \mid \mu_i(W) < \mu_i(T)\}.$$

Notice that $N^*(T)$ is a T-cohesive group if $c(T) \leq b \frac{|N^*(T)|}{|N|}$. We will now explain each step before presenting the full algorithm.

6.4.1 Verifying a Project Set

We will first describe how we are polynomial in the number of voters. First, we observe that given $T_1, T_2 \subseteq P$ such that $T = T_1 \cup T_2$, we can compute $N^*(T)$, efficiently since

$$N^*(T) = N^*(T_1) \cap N^*(T_2)$$

Once we have $N^*(T)$, we spend $O(nm)$ time to find $U(T)$, by iterating over all $i \in N^*(T)$ and computing $\mu_i(T)$ for each in $O(m)$ time. As discussed in Appendix A.1, a more careful implementation can reduce this to $O(n)$. But we use the $O(nm)$ solution.

6.4.2 Reducing the Search Space

The second optimization is to reduce the number of project sets to check for violations. The core intuition is as follows: if for a project set T , the set of voters approving voters $N^*(T)$, cannot collectively afford T , then no T -cohesive group can exist. This immediately rules out the existence of any T' -cohesive group where $T \subseteq T'$, since the approving voters of T' form a subset of those of T . Expressed differently, when T is unaffordable we have

$$c(T') \geq c(T) > b \frac{|N^*(T)|}{|N|} \geq b \frac{|N^*(T')|}{|N|},$$

and so any superset T' must also be unaffordable. This is essentially the Apriori property applied to cohesive groups, which allows us to exploit a bottom-up pruning approach. Beginning with project sets of size 1, we prune any set T for which $N^*(T)$ cannot afford T . We then iteratively build larger sets by joining pairs of unpruned sets that share the same $k - 1$ first items, in the style of the classic Apriori algorithm.

Example 12 (Search Space Reduction). Consider the example below for $|P| = 4$, and where sets $\{p_1, p_2\}$, $\{p_1, p_4\}$ are unaffordable for their corresponding set of approving voters. For $k = 2$ we join all sets where the first $k - 1 = 1$ projects are the same, giving us the only set of size 3: $\{p_2, p_3\} \cup \{p_2, p_4\} \rightarrow \{p_2, p_3, p_4\}$.

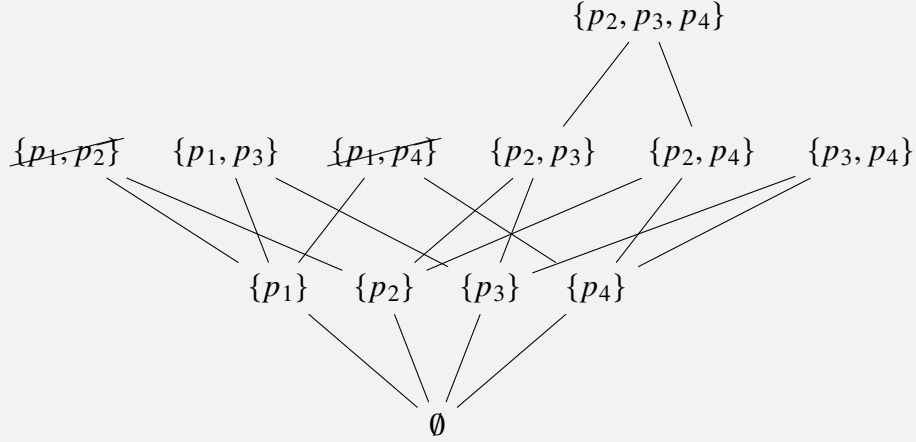


Figure 2: Project Set Lattice

In the classic Apriori algorithm, after generating new candidate sets, any set containing an unaffordable subset is pruned, requiring time linear in k to verify all subsets of size k . This pruning is justified when affordability checks are expensive, but in our setting the situation is reversed: verifying affordability of a new set is cheaper than checking all its subsets, so we omit this pruning step. A consequence of skipping subset-pruning is that the ordering of projects affects which candidates are generated. For instance, if only $\{p_2, p_3\}$ had been pruned, the $(k - 1)$ -join would still produce $\{p_1, p_2\} \cup \{p_1, p_3\} \rightarrow \{p_1, p_2, p_3\}$, despite $\{p_2, p_3\}$ being known unaffordable. Depending on the project ordering, the join may thus produce candidates that subset-pruning would have eliminated. However, since affordability of the new set is cheap to check directly, this is not a concern in our case.

One advantage of this join procedure is that each candidate is generated from exactly one combination of project sets, rather than by examining all possible combinations within a layer. Candidate generation therefore runs in time linear in the number of project sets produced. By the Apriori property, every valid superset must have all of its subsets valid. Considering only a single join can thus introduce additional candidates compared to a full subset check, but these candidates form only an overapproximation and are subsequently pruned when their affordability is evaluated. Implementation-wise, to perform efficient Apriori joins, we store the project sets not as sets but as sorted lists.

6.4.3 Algorithm

Below we define our algorithm for finding EJR violations. We have split the algorithm into three parts: how to create the first layer in the search space, how to build the following layer using Apriori, and the core algorithm that utilizes these algorithms and checks for violations.

First, the simple algorithm for building the first layer for all single projects and their voters is presented in Appendix A.2. Secondly, the algorithm to perform the apriori join for all project sets in the previous layer:

Algorithm 4: Pseudo-code of Build Next Layer.

Input: C which is the list of affordable $(T, N^*(T))$ pairs from the previous layer

Output: The list of project sets with size $k+1$ and their voters

```

1 Function build_next_layer ( $C$ )
2   if  $|C| = 0$  then
3     return empty list
4    $k \leftarrow |C_0[0]|$ 
5   next_layer  $\leftarrow$  empty list
6   for  $i \leftarrow 0$  to  $|C| - 1$  do
7     for  $j \leftarrow i + 1$  to  $|C| - 1$  do
8        $(T_i, N^*(T_i)) \leftarrow C_i$ 
9        $(T_j, N^*(T_j)) \leftarrow C_j$ 
10      if  $prefix_{k-1}(T_i) = prefix_{k-1}(T_j)$  then
11         $T' \leftarrow T_i + [T_j[k-1]]$                                  $\triangleright$  Project set union keeping order
12         $N^*(T') \leftarrow N^*(T_i) \cap N^*(T_j)$ 
13        Append  $(T', N^*(T'))$  to next_layer
14      else
15        break                                                         $\triangleright$  Break the j-loop
16  return next_layer
    
```

Lastly, the algorithm that uses the build layer functions and checks for a violation in the layer:

Algorithm 5: Pseudo-code of Find EJR Violation.

Input: Approval based PB election I , utility function μ , and election outcome W .

Output: Violation witness or none.

```

1 Function find_ejr_violation ( $I, \mu, W$ )
2   layer  $\leftarrow$  build_first_layer( $A, P$ )
3   while layer is not empty do
4     next_layer_candidates  $\leftarrow$  empty list
5     foreach  $(T, N^*(T)) \in$  layer do
6       if  $c(T) > b \frac{|N^*(T)|}{|N|}$  then                                 $\triangleright$  Prune if  $N^*(T)$  can't afford  $T$ 
7         continue
8       if  $c(T) \leq b \frac{|U(T)|}{|N|}$  then                                 $\triangleright$  Report violation if  $U(T)$  can afford  $T$ 
9         return violation witness
10      Append  $(T, N^*(T))$  into next_layer_candidates
11    layer  $\leftarrow$  build_next_layer(next_layer_candidates)
12  return no violation
    
```

6.5 Proof of Correctness

Proof. We wish to show, given an approval-based instance, utility function μ , and outcome W , that if our algorithm find_ejr_violation returns no witness, then the instance satisfies μ -EJR, and if the algorithm returns a witness, then the instance does not satisfy μ -EJR.

Case 1 find_ejr_violation returns a witness:

The algorithm returns a T' -cohesive group N' that violates μ -EJR. Thus, the instance does not satisfy μ -EJR.

Case 2 find_ejr_violation returns no witness:

In this case, the algorithm did not find any cohesive groups that violated μ -EJR. Assume for contradiction that there exists T' -cohesive group N' that violates μ -EJR. Now we have two ways for this to happen:

(a) The algorithm checked T' :

For any $i \in N'$ we have that $i \in U(T')$ which implies $N' \subseteq U(T')$. Since N' is a T' -cohesive group: $c(T') \leq b \frac{|N'|}{|N|} \leq b \frac{|U(T')|}{|N|}$. This is equivalent to the algorithm detecting a witness in line 8 and thus returns a witness, which is a contradiction.

(b) The algorithm did not check T' :

Then $t = |T'| > 1$ and there does not exist $T_1, T_2 \subseteq T'$, with $|T_1 \cap T_2| = t - 2$ where both T_1 and T_2 were not pruned. Assume without loss of generality that T_1 was pruned. We have $N' \subseteq N^*(T') = N^*(T_1) \cap N^*(T_2)$, which gives $|N'| \leq |N^*(T_1)|$. Since T_1 was pruned, the line 6 condition was met, i.e., $c(T_1) > b \frac{|N^*(T_1)|}{|N|}$. Finally, since $T_1 \subseteq T'$ we have $c(T_1) \leq c(T')$. Combining these, we get

$$c(T') \geq c(T_1) > b \frac{|N^*(T_1)|}{|N|} \geq b \frac{|N'|}{|N|}.$$

Specifically, this means that T' is not affordable by N' , which contradicts the fact that N' is a T' -cohesive group.

□

6.6 Algorithm Implementation Complexity

We now analyze the time complexity of our algorithm. Processing a single project set T takes $O(nm)$ time (see Section 6.4.1). The overall complexity therefore depends on the number of project sets examined; let s denote this quantity.

Constructing each new layer takes time proportional to s , since the loop in Algorithm 4 lines 6–7 happens once per project set – both existing and newly created. For each new set of size k , the algorithm inspects the $k - 2$ elements inherited from the two sets whose union forms T , with $k \leq m$. This contributes an $O(sm)$ term, which is dominated by the $O(s \cdot nm)$ cost of processing all s project sets, giving a total time complexity of $O(s \cdot nm)$.

Without pruning, we have $s = O(2^m)$. If instead we restrict attention to project sets of size at most κ (for some constant κ) while foregoing pruning, then

$$s = \sum_{i=0}^{\kappa} \binom{m}{i} \leq \kappa \cdot m^{\kappa} = O(m^{\kappa}),$$

and the overall complexity becomes $O(m^{\kappa} \cdot n\kappa)$.

For the parameter values observed in practice— $m = 200$ and $\kappa = 11$ —this bound is far too large for the algorithm to be tractable. Termination in practice therefore relies on pruning reducing s substantially below this worst-case figure.

6.7 Adapting the Algorithm to EJR-x & EJR-1

Adapting the algorithm to handle EJR-x and EJR-1 requires only minor changes. The overall structure of the algorithm remains the same, meaning we still search over project sets T and compute the maximal group $N^*(T)$ of voters who approve all projects in T . The only part that

changes is how we define the set $U(T)$ of voters in $N^*(T)$ who are not sufficiently represented by the outcome W .

Notice that if for some T -cohesive group, it is the case that $T \subseteq W$, then the group is immediately satisfied by definition. In that case, there is no need to compute the set of unsatisfied voters. Thus, in the following, we only consider the case where $T \setminus W$ is non-empty.

6.7.1 EJR-x

For EJR-x, we define the unsatisfied voters as:

$$U(T) = \{i \in N^*(T) \mid \mu_i(W) + \min_{p \in T \setminus W} \mu_i(p) \leq \mu_i(T)\}.$$

A voter is unsatisfied if their utility from W , even when adding their *worst* project in $T \setminus W$, does not reach $\mu_i(T)$. Since EJR-x requires satisfaction with respect to every project in $T \setminus W$, a voter who fails this weakest possible check cannot be satisfied by all projects.

6.7.2 EJR-1

For EJR-1, we define the unsatisfied voters as:

$$U(T) = \{i \in N^*(T) \mid \mu_i(W) + \max_{p \in T \setminus W} \mu_i(p) \leq \mu_i(T)\}.$$

A voter is unsatisfied if their utility from W , even when adding their *best* project in $T \setminus W$, does not reach $\mu_i(T)$. Since EJR-1 only requires satisfaction with respect to at least one project in $T \setminus W$, a voter who fails this strongest possible check cannot be satisfied by any project.

Replacing $U(T)$ as above in the algorithm *and* checking that $T \not\subseteq W$ are the only changes required to the algorithm. So the check becomes:

Algorithm 6: The updated violation condition.

```

1 if  $T \not\subseteq W \wedge c(T) \leq b \frac{|U(T)|}{|N|}$  then
2   return violation witness
    
```

The correctness proof carries over without further modification. The way in which candidate project sets are generated is completely unchanged, so case (b) is unaffected. For case (a), the assumed T' -cohesive group N' violating EJR-x or EJR-1 must satisfy $T' \not\subseteq W$, since otherwise no EJR violation could occur. The logic of case (a) therefore remains intact.

6.8 Parameterized Complexity of EJR Verification

One might hope that, even if verification is coNP-complete in general, the hardness is confined to some structural parameter of the election, allowing efficient verification when that parameter is

small. We show this hope is in vain: verification remains hard even when parameterized by some small k .

In classical complexity, NP-hardness is believed to imply that no polynomial-time algorithm exists relative to input size N . Parameterized complexity (Downey and Fellows, 2013) refines this picture by isolating the combinatorial explosion to a specific parameter k . We distinguish three classes:

- **FPT (Fixed-Parameter Tractable):** Solvable in $f(k) \cdot \text{poly}(N)$ time, where f is computable and depends only on k , quarantining all exponential growth to the parameter. This is the ideal outcome; for small k , the problem remains tractable regardless of input size.
- **XP (Slice-wise Polynomial):** Solvable in $O(N^{f(k)})$ time. For any fixed k the runtime is polynomial, but the exponent grows with k , making this strictly weaker than FPT.
- **W[1]-hard:** Believed to admit no FPT algorithm, assuming $\text{FPT} \neq \text{W}[1]$. This is the parameterized analogue of NP-hardness assuming $P \neq NP$.

We will show that verifying EJR is a W[1]-hard problem by, again, reducing from the BALANCED BICLIQUE problem, which is a foundational W[1]-hard problem (Lin, 2019).

6.8.1 Parameterized Tractability: FPT and XP Algorithms

The baseline tractability of EJR verification via subset enumeration is a known foundational result in the literature (Aziz et al., 2018a).

Theorem 8 (FPT w.r.t m). *Determining whether an outcome W satisfies EJR is Fixed-Parameter Tractable (FPT) parameterized by the number of projects m , running in $O(2^m \cdot nm)$ time.*

Proof. By iterating over all non-empty subsets $T \subseteq P$, we construct the maximal set of unsatisfied voters $U(T) = \{i \in N \mid T \subseteq A_i \text{ and } \mu_i(W) < \mu_i(T)\}$. If $|U(T)| \geq n \frac{c(T)}{b}$, a violation is found since the unsatisfied voters would then form a cohesive group. Constructing $U(T)$ requires $O(nm)$ time, and there are 2^m subsets, the total time is $O(2^m \cdot nm)$. Because the parameter m is mathematically isolated from n in the exponent, the problem is strictly FPT. $\square$

6.8.2 Parameterizing by Target Cardinality (k)

What if the total number of projects m is large, but we only wish to audit for violations involving T-cohesive groups of a bounded project set size k ?

Proposition 18 (XP w.r.t k). *Determining whether an outcome W contains an EJR violation for a T-cohesive group with $|T| = k$ is in XP, and can be done in $O(m^k \cdot nk)$ time.*

Proof. The algorithm from Section 6.4 runs in $O(m^\kappa \cdot n\kappa)$ to check all layers up to κ . Restricting to violations of a fixed size k yields an $O(m^k \cdot nk)$ running time. Because k appears in the exponent of m , this represents an XP algorithm. $\square$

Can we achieve a fundamentally faster algorithm than $O(m^k)$? Can we isolate k into a strict FPT bound like $f(k) \cdot \text{poly}(m)$? The rest of this section establishes that we cannot. The evaluation of EJR for standard algorithms is tightly linked to graph structures, rendering it strictly **W[1]-hard** with respect to k .

Furthermore, under the Exponential Time Hypothesis (ETH) (Impagliazzo and Paturi, 2001), W[1]-hardness via a reduction from BALANCED BICLIQUE generally implies that no $f(k)m^{o(k)}$ algorithm exists.

6.8.3 Context and Novelty of the Hardness Reductions

Before presenting the formal reductions, we highlight the theoretical novelty of our proofs compared to existing hardness results in the literature:

1. **Algorithm-Specific Output vs. Arbitrary Elections:** We have already established by Proposition 17 that verifying EJR for an arbitrary election outcome is computationally intractable (e.g., coNP-complete) based on the original proof by Aziz et al. (2017). We prove a significantly stronger condition: Verifying EJR is **W[1]-hard** even when the outcome is the exact, deterministic output of a widely deployed PB algorithm, Greedy $[\mu^\#]$.
2. **Exploiting Arbitrary-Cost Mechanics:** Our reductions use the mechanics of arbitrary-cost PB. By introducing small fractional costs, we artificially inflate the cost-effectiveness and load-efficiency of "trap" projects, blinding the algorithms to the true cohesive targets.

6.8.4 Parameterized Hardness: Greedy

Putting these ideas to use, we can show the following

Theorem 9. *Given an arbitrary-cost Participatory Budgeting instance, it is **W[1]-hard** parameterized by the target cardinality k to distinguish between the following two cases for the specific election outcome returned by Greedy $[\mu^\#]$:*

1. *The outcome satisfies μ^c -EJR.*
2. *The outcome violates μ^c -EJR-1 (thereby also violating EJR and EJR-x).*

Proof. The question for the original BALANCED BICLIQUE instance $(G = (L, R, E), k)$ is: *Does there exist $L' \subseteq L, R' \subseteq R$ with $|L'| = |R'| = k$ that form a complete bipartite graph?*

Let n be the number of vertices in R .

1. Core Election and "Empty Ballot" Padding. Let N_{orig} be the set of n original voters, corresponding one-to-one with the vertices in R .

We set the global budget to $b = n(k - 1)$. We add a set of $b - n$ padding voters denoted N_{empty} . This ensures that the total number of voters becomes $n_{total} = |N_{orig}| + |N_{empty}| = b$, equaling the budget. Assume without loss of generality $k \geq 2$ to get $|N_{empty}| \geq 0$ otherwise, the instance is trivial. The empty voters approve nothing. Because $n_{total} = b$, the requirement for T -cohesiveness group N' being able to afford T simplifies to $|N'| \geq c(T)$.

2. The Cost-Effectiveness Trap (P_{priv} vs P_{graph}).

- **Graph Projects (P_{graph}):** For each vertex $u \in L$ create a graph project p_u with unit cost.

Project p_u is approved by voter $i_v \in N_{orig}$ if $(u, v) \in E$. Each graph project is approved by at most n voters, so their cost-effectiveness (efficiency) defined as approvals-over-cost-ratio $\frac{\sum_{i \in N} A_i}{c(p)}$ is bounded by n .

- **Private Fractional Projects (P_{priv}):** For every original voter $i_v \in N_{orig}$, we create a large set of private projects. Each private project costs exactly $\delta = \frac{1}{n+1}$ and is approved exclusively by voter i_v . We create for each original voter $(n + 1)(k - 1)$ private project, such that their total cost equals $k - 1$. Thus, the efficiency for the private projects is $\frac{1}{\delta} = n + 1$.

3. Deterministic Execution Sequence of Greedy. Greedy $[\mu^\#]$ repeatedly selects the project with the highest approvals-over-cost-ratio. Because the private projects have efficiency $n + 1$, they strictly dominate the graph projects ($\leq n$). Greedy deterministically purchases the entirety of P_{priv} first.

The total cost to purchase all private projects is $n(k - 1)$. This exactly equals the total budget b , thus Greedy $[\mu^\#]$ perfectly exhausts the budget on P_{priv} , halts, and outputs exactly $W = P_{priv}$.

4. Evaluating the Parameter and the Axioms. Every original voter receives utility strictly equal to the cost of their private projects: $\mu_i^c(W) = k - 1$.

Case 1: YES Instance (G contains $k \times k$ biclique (R', L'))

Let $T = \{g_u : u \in L'\}$ and $N' = \{i_v : v \in R'\}$. Every voter in N' approves every project in T because all edges between R' and L' exists. Furthermore, we have $|T| = |L'| = |R'| = |N'| = k$. Since $b = n_{total}$ this group N' satisfies the quota to demand exactly k projects making N' a T -cohesive group. For any $i_v \in N'$ we have $\mu_i^c(T) = k$.

EJR-1 requires $\mu_i^c(W \cup p) > \mu_i^c(T)$ for some $i \in N'$ and $p \in T \setminus W$. But any such project has cost 1, so $\mu_i^c(W \cup p) = (k - 1) + 1 = k \not> k = \mu_i^c(T)$. The allocation therefore violates μ^c -EJR-1 for a target of size k .

Case 2: NO Instance (no biclique of size k exists)

We will show by contradiction that in this case, W satisfies EJR.

Assume by contradiction that there exists T -cohesive group N' violating EJR. Notice that T can contain no private projects, since that would imply $|N'| = 1$ and therefore also $c(T) \leq 1$ by definition. If T also contained a graph project, then $c(T) > 1$, a contradiction. So T would consist only of private projects, but all private projects are included in the outcome, and therefore $T \subseteq W$ which means N' does not violate EJR.

Therefore any violating T consists of only graph projects. For T to violate EJR, it must be the case for every voter $i \in N'$:

$$k - 1 = \mu_i^c(W) < \mu_i^c(T) = |T|.$$

Thus $|T| \geq k$. Since N' can afford all project in T , we have $|N'| \geq c(T) = |T| \geq k$.

Since every voter in N' approves of every project in T we can choose k voters and k projects that correspond to a $k \times k$ biclique, contradicting the assumption that W contains an EJR violation. $\square$

Together with the XP algorithm, this establishes a tight complexity boundary: verification is tractable for fixed k but resists any algorithm that would confine the hardness to k alone. Meaning that any EJR checking algorithm needs to be at least $O(m^k)$. Notably, this hardness is tied to the structured outcomes produced by Greedy, one of the most natural and widely studied algorithms in participatory budgeting.

Since our algorithm's worst-case runtime matches the XP bound, it is therefore essentially optimal in the sense that the exponential dependence on $k = \kappa$ cannot be avoided.

6.9 Violation Degree

When testing for EJR, we are interested not only in whether an outcome satisfies EJR, but also in quantifying how close it comes to doing so. We measure this by identifying the largest violation.

For a T -cohesive group N' and a voter $i \in N'$, we define the *satisfaction degree* α_i as

$$\mu_i(W) = \alpha_i \cdot \mu_i(T).$$

For a fully satisfied voter we set $\alpha_i = 1$; an unsatisfied voter has $\alpha_i < 1$, with smaller values indicating larger violations. Note that $\mu_i(T) > 0$ from the definition of group cohesiveness.

When the algorithm of Section 6.6 identifies a violating group T and its set of unsatisfied voters $U(T)$, we wish to find the *worst-case* subset of $U(T)$, that is, the smallest subset whose satisfaction degree is minimal. Formally, for a set of voters $S \subseteq N$, we define

$$\Gamma_T(S) = (\alpha_i)_{i \in S}$$

as the vector of satisfaction degrees of voters in S with respect to T . The minimum number of voters required to collectively afford T is

$$v(T) = \left\lceil \frac{\text{cost}(T) \cdot |N|}{b} \right\rceil.$$

The smallest set of unsatisfied voters yielding the worst satisfaction degree is then

$$U'(T) = \arg \min_{S \subseteq U(T), |S|=v(T)} \max_{i \in S} \alpha_i,$$

and we report the violation as the triple $(T, U'(T), \max \Gamma_T(U'(T)))$.

Example 13. Suppose $v(T) = 2$, so $U'(T)$ must have exactly two voters, and $U(T)$ consists of three voters with satisfaction degrees $\Gamma_T(U(T)) = \{0, \frac{1}{3}, \frac{2}{3}\}$. We select $U'(T) = \{1, 2\}$, yielding $\Gamma_T(U'(T)) = \{0, \frac{1}{3}\}$, and report the violation with satisfaction degree $\frac{1}{3}$.

However, stopping at the first detected violation may not yield the largest one overall. We therefore modify the algorithm to continue past the first violation, enumerating all project sets T that admit a violation. Among these, we report the minimum satisfaction degree:

$$\alpha^* = \min_{T: T \text{ admits an EJR violation}} \max \Gamma_T(U'(T)).$$

Since every project set capable of producing a violation is examined, α^* is guaranteed to find the largest violation present in the election. We will show that finding α^* is feasible on real elections in Section 7.4.

Intuitively, α^* is the satisfaction degree of the most satisfied person in the most violated group. Meaning that if all voters got $1 - \alpha^*$ more satisfaction, then there would be no violations.

We define the violation degree as

$$\phi = 1 - \alpha^*,$$

which quantifies how far an outcome is from satisfying the axiom. A value of $\phi = 0$ indicates no EJR violation, while $\phi = 1$ indicates that a cohesive group receives no satisfaction from the outcome W . Larger values thus reflect increasingly severe violations, and this measure will serve as our primary indicator of proportionality in the experimental evaluation of elections with EJR violations.

6.10 Why PJR and FJR Remain Infeasible

The algorithm developed in the previous section is feasible in practice because EJR has a useful structural property: for a fixed project set T , the largest possible voter group that could be T -cohesive is uniquely determined. Recall that this group is:

$$N^*(T) = \{i \in N : T \subseteq A_i\}$$

Every other T -cohesive group is a subset of $N^*(T)$. Hence, if $N^*(T)$ is too small to afford T , then no T -cohesive group exists, and T and every superset of T can be discarded.

For EJR, the remaining problem is then simply to check whether there are enough unsatisfied voters inside $N^*(T)$. That is, whether enough voters in $N^*(T)$ have utility less than $\mu(T)$ to form a T -cohesive group witnessing a violation. This is what allows the voter side of the problem to be handled without explicitly searching over all subsets of voters.

PJR. Recall that a violation of μ -PJR requires a T -cohesive group $N' \subseteq N$ such that:

$$\mu\left(W \cap \bigcup_{i \in N'} A_i\right) < \mu(T)$$

The important difference is that PJR does not check whether a single voter in N' is well represented. Instead, it checks whether the group as a whole is represented by the winning projects approved by at least one voter in the group. For a fixed project set T , every possible T -cohesive group is still a subset of $N^*(T)$. However, unlike for EJR, it is no longer sufficient to check only the maximal group $N^*(T)$ or the unsatisfied voters within it. This is because the PJR satisfaction term depends on the specific voters included in the group:

$$\bigcup_{i \in N'} A_i$$

If we add more voters to N' , this union can only become larger. Therefore, the value

$$\mu\left(W \cap \bigcup_{i \in N'} A_i\right)$$

can also increase. This means that the maximal group $N^*(T)$ may be sufficiently represented simply because its voters collectively approve many winning projects, while some smaller sufficiently large subgroup N' is not sufficiently represented. Thus, for fixed T , checking PJR would require us to determine whether there exists a subgroup $N' \subseteq N^*(T)$ that is large enough to afford T :

$$\frac{|N'|}{n} \cdot b \geq c(T)$$

but still receives too little group representation:

$$\mu \left(W \cap \bigcup_{i \in N'} A_i \right) < \mu(T)$$

This introduces a second combinatorial search over voter groups. The EJR algorithm avoids this search because, once T is fixed, the relevant voter group can be derived directly from $N^*(T)$. For PJR, the exact subgroup matters, since different subgroups of $N^*(T)$ may approve different winning projects. Consequently, the same maximal-supporter pruning that makes the EJR checker practical does not directly give a practical PJR checker.

FJR. In EJR and PJR, the project set T must be commonly approved by all voters in the group. FJR removes this requirement entirely. It asks that for every group $N' \subseteq N$ and every non-empty project set $T \subseteq P$ with:

$$\frac{|N'|}{n} \cdot b \geq c(T)$$

there exists some voter $i \in N'$ such that $\mu_i(W) \geq \mu_i(T)$. Equivalently, an FJR violation consists of a group N' and a project set T such that every voter in N' receives less utility from W than from the projects in T that they personally approve:

$$\mu_i(W) < \mu_i(T)$$

for all $i \in N'$, while N' is still large enough to afford T . The main difficulty is that there is no longer a well-defined maximal supporter group $N^*(T)$. For EJR and PJR, the voters in a T -cohesive group must all approve every project in T . This allows us to define $N^*(T)$, and every T -cohesive group must be a subset of this. For FJR, this is no longer true. The voters in N' do not need to approve all projects in T , and therefore T does not determine a unique maximal voter group in the same way.

This also removes the monotone structure used by our EJR checker. For EJR, adding more projects to T can only shrink the common supporter set $N^*(T)$ and increase the cost $c(T)$. Hence, if $N^*(T)$ cannot afford T , then no superset of T can become relevant. For FJR, however, voters are evaluated individually through the quantities $\mu_i(T)$. Adding a project to T may increase $\mu_i(T)$ for some voters, leave it unchanged for others, and increase the cost $c(T)$. These effects do not move in one direction, so the same pruning argument no longer applies.

For a fixed project set T , we can write the possible FJR-witnessing voters as:

$$U(T) = \{i \in N : \mu_i(W) < \mu_i(T)\}$$

These are exactly the voters who would be underrepresented with respect to T . Then T gives an FJR violation if this set is large enough to afford T , that is, if:

$$|U(T)| \geq \left\lceil \frac{n \cdot c(T)}{b} \right\rceil$$

This condition is easy to check once T is fixed. The problem is finding such a project set T . Unlike for EJR, if a project set T fails to witness a violation, we cannot safely discard all supersets of T . A superset $T' \supseteq T$ has higher cost, which makes it harder to afford, but it may also make more voters underrepresented by increasing $\mu_i(T')$. Hence, it is possible that T does not witness an FJR violation, while some larger set T' does. This prevents the downward closure property required for Apriori-style pruning. Therefore, the best algorithm we can come up with runs in $\theta(m^\kappa \cdot nm)$, since there is no pruning.

In summary, our EJR checker is feasible in practice because EJR has a structure that lets us collapse the voter side of the problem. For each project set T , the maximal common supporter group $N^*(T)$ is uniquely determined, and the possible EJR-witnessing voters are exactly the voters in $N^*(T)$ whose utility is below $\mu(T)$. PJR keeps the common-approval requirement, but the satisfaction condition depends on the exact subgroup $N' \subseteq N^*(T)$, since different subgroups may approve different winning projects. FJR goes further by removing the common-approval requirement altogether, so there is no longer a maximal supporter group $N^*(T)$ to rely on. Therefore, extending the same approach to PJR or FJR would require either an additional search over voter groups, or a search over project sets without the monotone pruning that makes the EJR algorithm practical.

7 Experiments

In this section, we apply the algorithm from Section 6 to real-world participatory budgeting elections from the Pabulib dataset. We use it to evaluate how well the considered voting rules satisfy EJR-based proportionality criteria in practice, to measure the performance of the algorithm itself on real-world instances, and to compute additional statistics that characterise the structure of the elections independently of any particular voting rule. The section is split into 4 parts: Section 7.3 for analyzing proportionality across rules and elections, Section 7.4 for analyzing the violation degree, Section 7.5 for analyzing the theoretical time complexity, and Section 7.6 for analyzing general statistics not related to EJR.

7.1 Pabulib & Data

Our experiments require a broad and realistic collection of participatory budgeting elections. We use the 1050 approval-voting elections available from Pabulib [Faliszewski et al. \(2023\)](#), a publicly accessible repository of real-world participatory budgeting data, after excluding two types of instances: fully funded elections, where the total cost of all projects falls within the budget, and the allocation problem is therefore trivial, and experimental elections, which were constructed for research purposes rather than drawn from real-world votes. Of the remaining 1050 elections, 882 are from Poland, reflecting the country’s widespread adoption of participatory budgeting at both the municipal and district level. The remaining elections come from a variety of countries, including the US (97), the Netherlands (41), and France (22).

The elections vary considerably in scale. At the smaller end, some involve as few as 84 voters and 14 projects. At the larger end, city-wide elections can attract upwards of 100,000 voters and feature up to 200 candidate projects.

The two most common vote types in the dataset are k -approval and knapsack voting. In k -approval, each voter may approve at most k projects; the most common choice in larger elections is $k = 15$. In knapsack voting, a voter’s approved set must respect the budget constraint $\sum_{p \in A_i} c(p) \leq b$, meaning voters can only approve projects whose combined cost does not exceed the total budget.

We also make use of Pabutools, a Python library developed by the same group, which provides implementations of the voting rules studied in this thesis: Greedy, Method of Equal Shares (MES), and Sequential Phragmén.

7.2 Evaluated Rules and Criteria

For rules with utility functions we evaluate the rule with both cardinality-based and cost-based utilities. The greedy scheme is denoted by $\text{Greedy}[\mu]$, where μ specifies the utility function used when selecting projects. The Method of Equal Shares is denoted by $\text{MES}[\mu]$, and Sequential Phragmén is denoted by Seq-Phragmén .

Since some of the considered voting rules may return outcomes that do not exhaust the budget, we also consider completed variants of these outcomes. In particular, we apply the Utilitarian Greedy Completion rule (U) to fill the remaining budget where possible. We write $\text{MES}[\mu_1]_{\mu_2}$ for the outcome obtained by first running MES with utility function μ_1 and then completing the outcome using $\text{GREEDY}[\mu_2]$. We use this completion method rather than Budget Expansion Completion (Add1), since Add1 is computationally impractical on large datasets: it requires repeatedly recomputing election outcomes under gradually expanded budgets. We also exclude Completion by Tweaking Utilities (EPS), because the experimental results in [Faliszewski et al. \(2023\)](#) suggest that it performs substantially worse on real-world data.

The resulting outcomes are evaluated with respect to the proportionality criteria introduced earlier: μ -EJR, μ -EJR- x , and μ -EJR-1. Finally, we also compute μ -EJR- ϕ , a strengthened experimental measure that continues after the first violation in order to quantify the degree of violation.

Several of our plots show results as functions of election features, such as vote length or the number of projects. Since elections are not uniformly distributed across these feature values, raw per-value violation rates tend to be noisy and unreliable, particularly at extreme values where few elections are observed. We therefore plot a cumulative moving average: for a given feature value x , we plot the average violation rate across all elections with feature value at most x . This smooths out local noise and reveals the underlying trend between the feature and violation rate. The estimate stabilizes as more data accumulates with increasing x , which is why curves may appear volatile at low feature values – where few elections have been observed – before settling into a stable trend. When reading the moving average graph, focus on the slope at each point. A long flat line doesn't necessarily indicate no correlation — it often means the data is sparse in that region.

7.3 EJR Verification Results

In this section, we will examine the violations identified by our algorithm.

EJR Satisfaction Results. Table 4 summarizes the results produced by our EJR checking algorithm. Each entry is the percentage of Pabulib elections for which the given rule satisfies the given axiom.

Algorithm	$\mu^\#$ -EJR	μ^c -EJR	μ^c -EJR-x	μ^c -EJR-1
Greedy[$\mu^\#$]	99.9%	94.8%	99.9%	99.9%
Greedy[μ^c]	73.0%	68.6%	74.2%	74.2%
MES[$\mu^\#$]	100.0%	63.0%	100.0%	100.0%
MES[μ^c]	100.0%	91.9%	100.0%	100.0%
Seq-Phragmén	100.0%	95.0%	100.0%	100.0%
MES[$\mu^\#$] _{$\mu^\#$}	100.0%	94.9%	100.0%	100.0%
MES[μ^c] _{μ^c}	100.0%	99.2%	100.0%	100.0%
Seq-Phragmén _{$\mu^\#$}	100.0%	95.7%	100.0%	100.0%
Seq-Phragmén _{μ^c}	100.0%	96.1%	100.0%	100.0%

Table 4: Results by Algorithm

The columns distinguish between the utility function used in the proportionality criterion. The column $\mu^\#$ -EJR refers to cardinality-based utility, where each approved selected project contributes one unit of utility. The remaining columns use cost-based utility μ^c , where utility depends on the costs of the approved selected projects.

For cardinality-based utility, we only include one column. This is because $\mu^\#$ -EJR, $\mu^\#$ -EJR-x, and $\mu^\#$ -EJR-1 coincide, as shown in Theorem 3. Thus, the single $\mu^\#$ -EJR column represents all three cardinality-based variants.

Running time. We now turn to the running time of our EJR verification algorithm from section 6. The goal of this section is to understand how the algorithm behaves on real-world PB elections from [Pabulib \(2026\)](#). In particular, we investigate which election characteristics seem to influence the running time in practice.

Figure 3 shows the moving average runtime of the three algorithms across five election characteristics. Across all plots, EJR-1 and EJR-x are consistently the slowest, with EJR being the fastest. For EJR-1 and EJR-x, the subset check $T \subseteq W$ is required for correctness and is always performed. For EJR, this check is not required for correctness, but we include it explicitly as an optimization. Without it, the EJR algorithm was in fact slower than EJR-1.

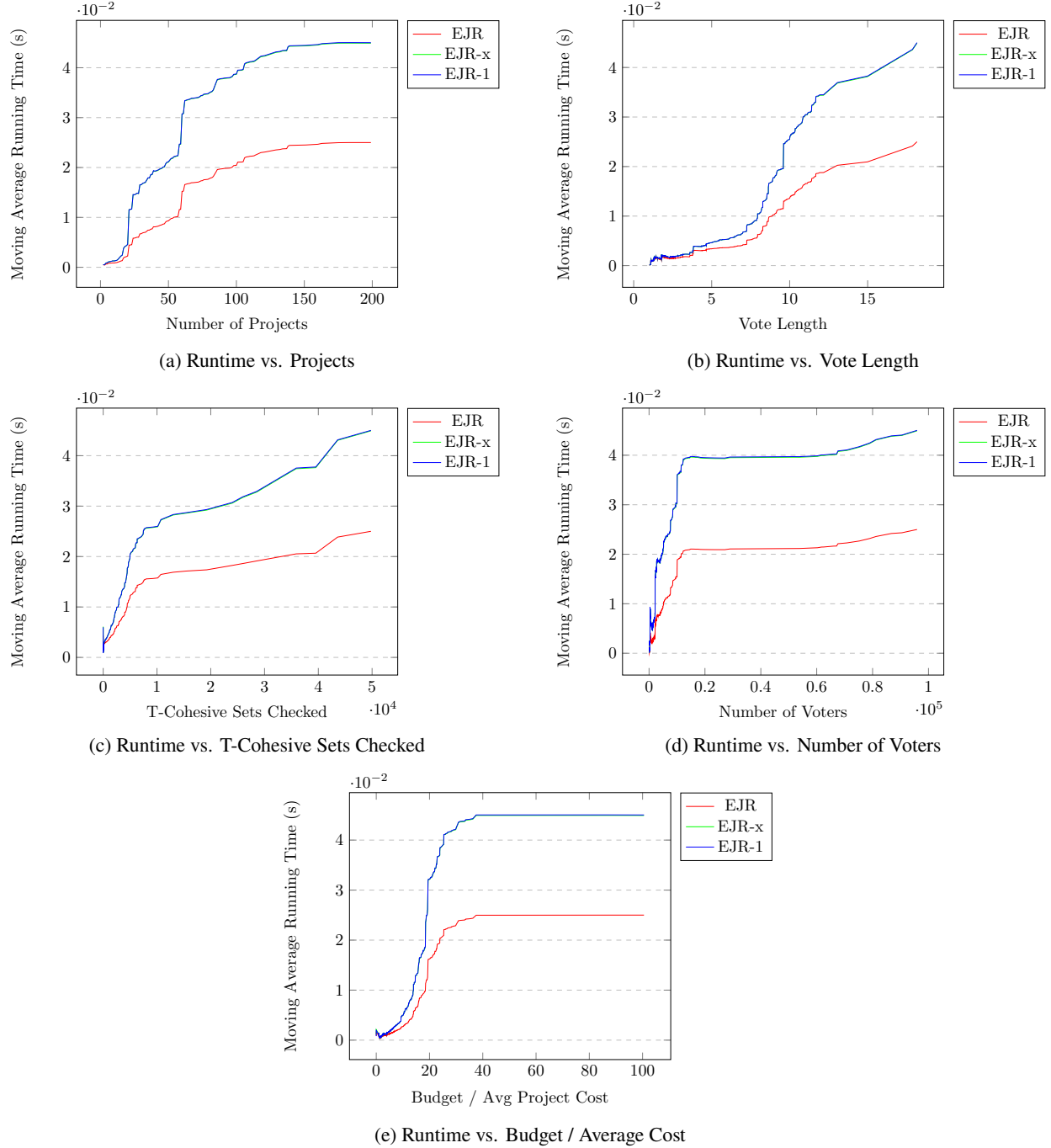


Figure 3: Relationship between algorithm runtime and various election characteristics.

All three algorithms exhibit broadly similar growth trends, with runtime increasing in the number of projects, vote length, number of T-cohesive sets checked, number of voters, and the budget-to-average-project-cost ratio. The sharpest growth appears in Figure 3e, where runtime rises steeply as the budget-to-average-cost ratio increases from 0 to around 20 before leveling off. Note that a single outlier in this plot causes the curve to appear to flatten earlier than it otherwise would. The

gap between EJR and the other two variants is consistent throughout, indicating that the added generality of EJR-x and EJR-1 comes at a modest but stable computational cost. In Figure 3, a steady upward trend suggests a correlation, while a flat or quickly saturating curve suggests the factor has limited influence – though plateaus should be interpreted with care, as they may simply reflect sparse data in certain regions of the x-axis rather than a true saturation effect.

- **T-Cohesive sets checked (Figure 3c).** This shows the clearest correlation, which is expected since the number of sets checked is directly tied to the work performed by the algorithm.
- **Vote length (Figure 3b).** Also a strong correlation, though more indirect: longer ballots mean more projects are approved per voter, which increases the number of cohesive groups that can form, in turn driving up the number of sets checked.
- **Number of projects (Figure 3a).** A positive correlation is visible, though with somewhat more noise than the two above. This is plausibly due to two competing effects: a larger m increases the number of possible project combinations the algorithm must consider, but also tends to spread approval votes more thinly across projects, reducing the number of voters approving any given project and thereby making cohesive group formation harder. The net effect is mostly positive in the dataset.
- **Number of voters (Figure 3d).** Runtime grows initially but plateaus in the middle, likely due to sparse data in that range rather than a true saturation effect. Overall, there appears to be a positive correlation, which is theoretically reasonable: more voters increase the likelihood of large overlapping approval sets, making it easier to form large, cohesive groups.
- **Budget-to-cost ratio (Figure 3e).** Ignoring the single outlier at the far right, this shows a strong positive correlation. The ratio increases when either the budget grows or the average project cost falls, both of which make the cohesiveness threshold $c(T) \leq \frac{|N'|}{n} \cdot b$ easier to satisfy, leading to more cohesive groups. Additionally, a higher ratio means more projects can be funded in a feasible outcome, increasing voter satisfaction under a cardinality-based utility model and making EJR violations harder to find, so the algorithm must search more exhaustively before terminating.

Many of the correlations above are arguably unsurprising, as metrics such as number of projects and voters all serve as proxies for election size, and larger elections naturally involve more work. As they are proxies of election size, they also include a certain correlation between the features, like more budget often means more voters. The most theoretically interesting correlation is therefore the budget-to-cost ratio, since it is not clearly related to election size and instead reflects a structural property of the election that directly affects both cohesive group formation and the difficulty of finding EJR violations.

Violation Feature Analysis. Looking at Table 4, one might wonder which election features increase the likelihood of a violation occurring. Next, we plot the violation rate as a function of the moving average of different election features

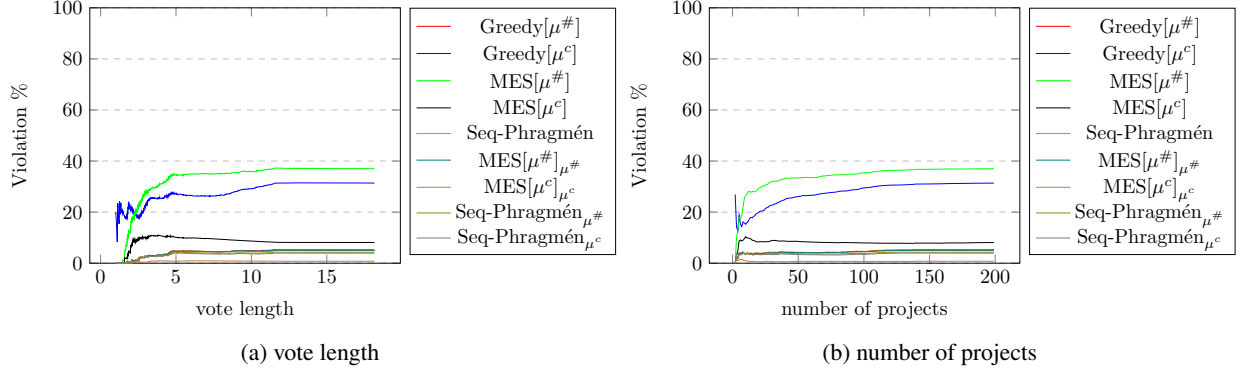


Figure 4: μ^c -EJR Violation Rate vs Features

A notable pattern emerges: for rules such as MES[$\mu^\#$] and Greedy[μ^c], the violation rate increases with both the average vote length and the number of projects, suggesting these rules struggle more in larger and more complex elections. Conversely, the better-performing rules – Sequential Phragmén, MES[μ^c], and their completion variants – remain consistently low across all feature values, indicating their proportionality guarantees are robust to election size. This asymmetry suggests that the gap between rules widens as elections grow larger, with the poorly-performing rules becoming increasingly unreliable while the better rules remain stable.

While we focus here on vote length and the number of projects, we observe broadly similar but less clear patterns across other election features, such as the number of voters and the average project cost; these are included in Appendix A.3.5.

7.4 Violation Degree

We now take a deep dive into the Pabulib elections in detail from the perspective of proportionality. Instead of recording only whether an outcome satisfies or violates EJR, we also examine the severity of violations and their distribution across voting rules. This gives a more nuanced picture of the empirical behavior of the rules on real PB elections.

As defined in Section 6.9, the violation degree, ϕ , is our quantifier for how proportional an outcome election is. A violation degree of 0 indicates that the outcome satisfies EJR, while $\phi = 1$ represents a maximal violation, meaning every voter in the most violating cohesive group approves no project in the outcome.

Distribution of violation degrees. We compute the violation degree for each of the elections in the Pabulib dataset under each voting rule and outcome type. Elections that satisfy EJR have $\phi = 0$ and are excluded from the analysis below, as the violation degree is only meaningful for elections that actually violate EJR.

Algorithm	$\#_V$	Mean	Median	Q_1	Q_3	Min	Max
Greedy[$\mu^\#$]	55	0.2218	0.1825	0.0939	0.2808	0.0122	0.8327
Greedy[μ^c]	329	0.9067	1.0000	1.0000	1.0000	0.0279	1.0000
MES[$\mu^\#$]	388	0.3100	0.3003	0.1314	0.4402	0.0006	0.8395
MES[μ^c]	85	0.2126	0.1652	0.0606	0.3000	0.0006	0.7519
Seq-Phragmén	52	0.2186	0.1805	0.0921	0.2907	0.0150	0.6319
MES[$\mu^\#$] $_{\mu^\#}$	53	0.2067	0.1805	0.0884	0.2793	0.0122	0.6319
MES[μ^c] $_{\mu^c}$	8	0.3564	0.4366	0.1835	0.4759	0.0396	0.6319
Seq-Phragmén $_{\mu^\#}$	45	0.2093	0.1805	0.0769	0.2793	0.0084	0.6319
Seq-Phragmén $_{\mu^c}$	41	0.2110	0.1848	0.0533	0.3030	0.0076	0.6319

 Table 5: Violation Degree Analysis for μ^c -EJR- ϕ

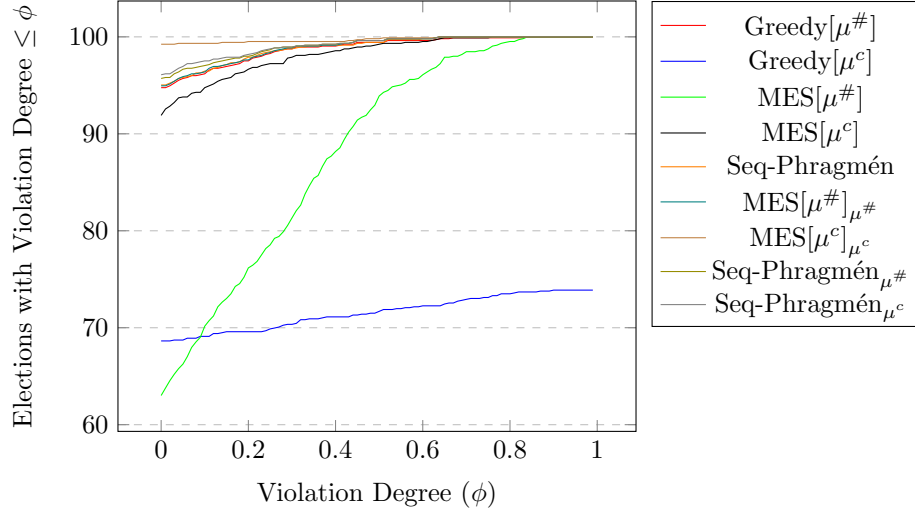
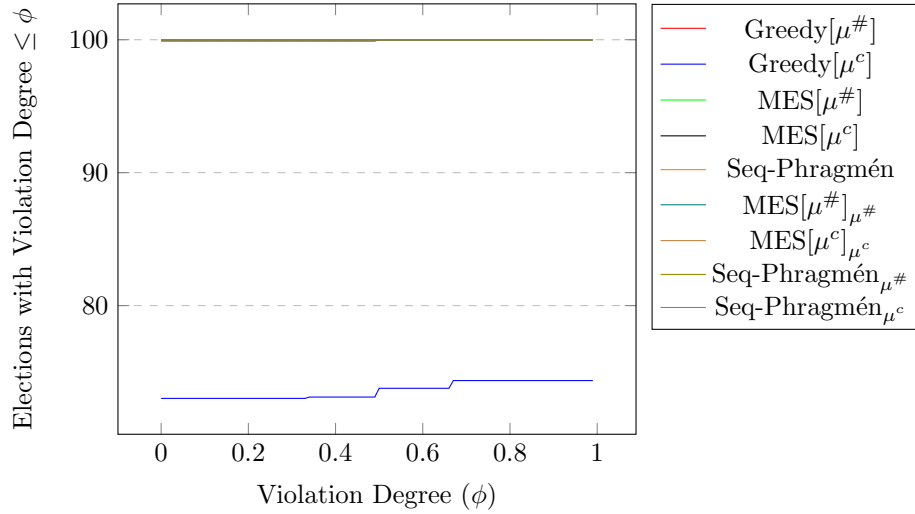
Algorithm	$\#_V$	Mean	Median	Q_1	Q_3	Min	Max
Greedy[$\mu^\#$]	1	0.5000	0.5000	0.5000	0.5000	0.5000	0.5000
Greedy[μ^c]	283	0.9782	1.0000	1.0000	1.0000	0.3333	1.0000

 Table 6: Violation Degree Analysis for $\mu^\#$ -EJR- ϕ

Tables 5 and 6 summarise the distribution of ϕ across violating elections, broken down by voting rule, for EJR- ϕ [μ^c] and EJR- ϕ [$\mu^\#$] respectively.

For μ^c -EJR- ϕ , the results reveal a clear contrast between Greedy[μ^c] and all other algorithms. Greedy[μ^c] produces 329 violating elections with a median violation degree of 1.0 and $Q_1 = 1.0$, indicating that at least three quarters of its violations are maximal. All other algorithms have median violation degrees in the range 0.17–0.44, suggesting that when they violate EJR they tend to do so only moderately.

For $\mu^\#$ -EJR- ϕ , violations are far rarer. Only Greedy[μ^c] produces a substantial number of violations ($\#_V = 283$), again with median 1.0 and $Q_1 = 1.0$. Greedy[$\mu^\#$] produces a single violating election with $\phi = 0.5$. MES, Sequential Phragmén, and all combined outcomes satisfy $\mu^\#$ -EJR- ϕ on every election in the dataset.


 Figure 5: μ^c -EJR Violation Degree Distribution

 Figure 6: $\mu^\#$ -EJR Violation Degree Distribution

Figures 5 and 6 show the empirical CDF of violation degrees across all elections, for both μ^c -EJR (top) and $\mu^\#$ -EJR (bottom). The y-axis reports the percentage of elections with violation degree at most ϕ . A curve that starts high indicates that many elections satisfy EJR outright ($\phi = 0$), while a curve that rises slowly indicates that violations, when they occur, tend to be severe. Notice that the data point for $\phi = 1$ is omitted, which is why Greedy[μ^c] never reaches 100%.

For μ^c -EJR, most algorithms start above 90% at $\phi = 0$, confirming that the majority of elections satisfy EJR outright. Greedy[μ^c] is the clear outlier, starting near 70% and rising only gradually to around 75% even at ϕ close to 1, consistent with the high proportion of maximal violations seen in

Table 5. $\text{MES}[\mu^\#]$ starts somewhat lower than the other non-Greedy algorithms but converges to 100% before $\phi = 1$.

For $\mu^\#$ -EJR, the picture is a bit different. All algorithms except $\text{Greedy}[\mu^c]$ sit at 100% for all values of ϕ , meaning they satisfy $\mu^\#$ -EJR on every election in the dataset. $\text{Greedy}[\mu^c]$ again stands apart, starting near 75% and rising only marginally, with violations spread across the full range of ϕ .

Running time. Computing the violation degree, ϕ , requires us to find all violations, next we plot the extra cost of finding all violations

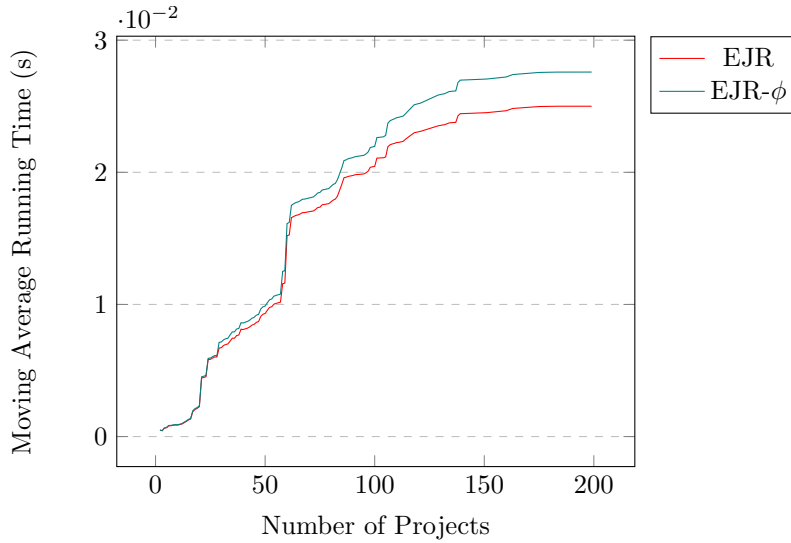


Figure 7: Comparing running time of EJR and EJR- ϕ

Computing the violation degree requires the algorithm to enumerate all violating project sets rather than terminating at the first detected violation. We compare the running time of the standard early-exit EJR algorithm against EJR- ϕ , which computes the full violation degree, to assess whether the additional computation remains feasible in practice.

For each election, we run both variants on the outcomes produced by all voting rules, yielding up to 16 data points per election – 8 voting rules across 2 utility functions. Since the goal is to assess feasibility rather than per-rule differences, we average across voting rules and outcome types. Figure 7 plots the moving average of running time against the cumulative number of projects processed.

As the figure shows, the two curves are nearly indistinguishable throughout, with EJR- ϕ adding only negligible overhead over the early-exit variant. Running times grow with the number of projects but remain well below 0.1 seconds even for the largest elections in the dataset, confirming that computing the violation degree is practically feasible.

7.5 Theoretical Runtime

In this section we analyze statistics relevant to the running time of our algorithm.

Largest project set checked. First, we analyze how far up the lattice we search / the largest size project sets we check (κ)

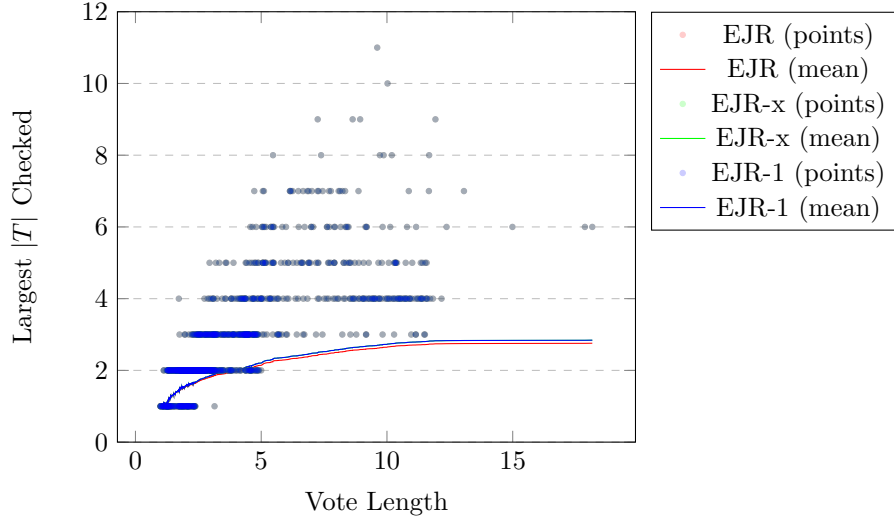


Figure 8: Largest Layer Checked

Figure 8 shows that the largest $|T|$ checked grows modestly with average vote length, averaging to 3 across all elections.

Each election contributes three points, one per axiom variant, though the EJR-x (green) and EJR-1 (blue) points coincide exactly and are therefore indistinguishable in the plot. For a small number of elections, EJR requires checking a strictly smaller largest layer than the other two variants, placing the red EJR point directly below its blue/green counterpart. These cases are rare enough that the red points are effectively invisible in the figure, though they are present.

Distribution of sets in layers. Next, we want to know how many cohesive sets checked in each layer:

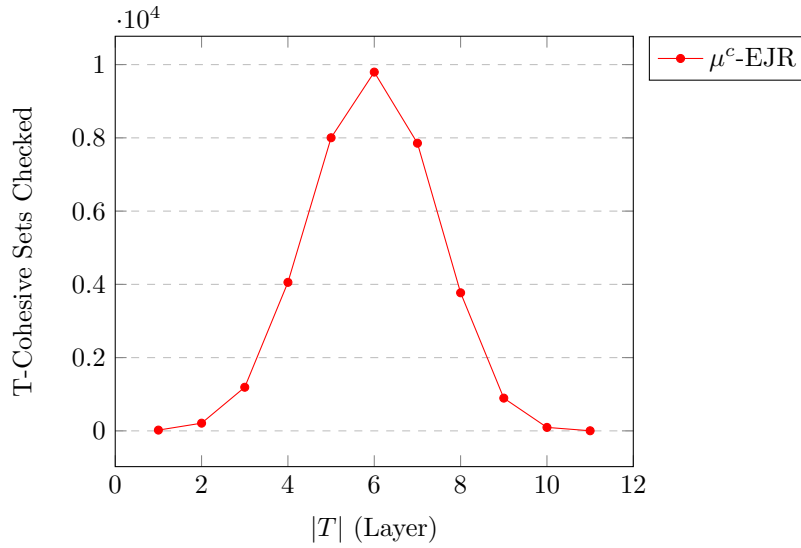


Figure 9: Cohesive sets checked per layer, Poland Warszawa 2018 Bialoleka obszar 3, with 21 projects

Figure 9 focuses on the single election where the algorithm reaches layer 11 before finding a witness, plotting the number of T -cohesive sets checked at each layer. The distribution follows a bell-shaped curve, peaking at layer 6 before falling sharply – a pattern that is representative of other large elections as well.

7.6 General Outcome Statistics

So far, the analysis has focused on proportionality violations detected by our EJR verification algorithm. We now turn to a broader set of descriptive statistics about the outcomes produced by the voting rules. These results are not themselves proportionality axioms, but they help explain the behavior observed above and give a more complete picture of how the rules perform on real PB elections.

In particular, we compare the rules in terms of achieved utility, budget usage, exclusion ratio, and the structure of the project sets that the verification algorithm considers. This allows us to relate the proportionality results to more direct properties of the outcomes, such as how much of the budget is spent, how many voters receive no utility, and how large the relevant cohesive project sets tend to be.

Voting Rule Statistics. We compare several metrics averaged across all elections: a voter’s average utility $\mu_{\text{rel.}}$ – based on both cardinal and cost-based utility – expressed relative to the greedy rule under the same μ ; the average share of the budget spent; and the average computation time required to determine the election outcome.

Algorithm	$\mu_{\text{rel.}}^{\#}$ (%)	$\mu_{\text{rel.}}^c$ (%)	Budget Usage (%)	Exhaustive (%)	Running Time (s)
Greedy[$\mu^{\#}$]	100.0	73.5	86.8	100.0	0.1008
Greedy[μ^c]	89.1	100.0	96.3	100.0	0.1619
MES[$\mu^{\#}$]	69.9	39.8	43.1	7.5	0.1596
MES[μ^c]	70.8	50.3	49.2	7.7	0.2103
Seq-Phragmén	96.7	71.4	81.0	66.0	0.4540
MES[$\mu^{\#}$] $_{\mu^{\#}}$	100.0	73.4	86.8	100.0	0.2159
MES[μ^c] $_{\mu^c}$	98.9	86.2	90.7	100.0	0.3093
Seq-Phragmén $_{\mu^{\#}}$	99.8	73.8	86.9	100.0	0.4877
Seq-Phragmén $_{\mu^c}$	99.9	74.2	87.4	100.0	0.5092

Table 7: Average Utility Comparison and Budget Usage by Algorithm

In both cases, the greedy scheme achieved the highest average utility. The MES with a completion rule also performed reasonably well on utility. Worth noting is the low budget utilization for the non-exhaustive MES rules. Regarding runtime, Seq-Phragmén was roughly twice as slow as MES, which in turn was roughly twice as slow as Greedy - though all methods ran fast enough that computation time should not be a deciding factor when selecting the best approach.

Exclusion Ratio. This next plot illustrates the exclusion ratio: The share of voters who receive zero utility from the elected outcome W . In other words, it captures the fraction of the electorate that is entirely unrepresented in the final selection. Formally, the exclusion ratio is defined as $\frac{|\{i \in N \mid \mu_i(W) = 0\}|}{|N|}$, where a higher value indicates that a larger portion of voters saw none of their preferred projects funded.

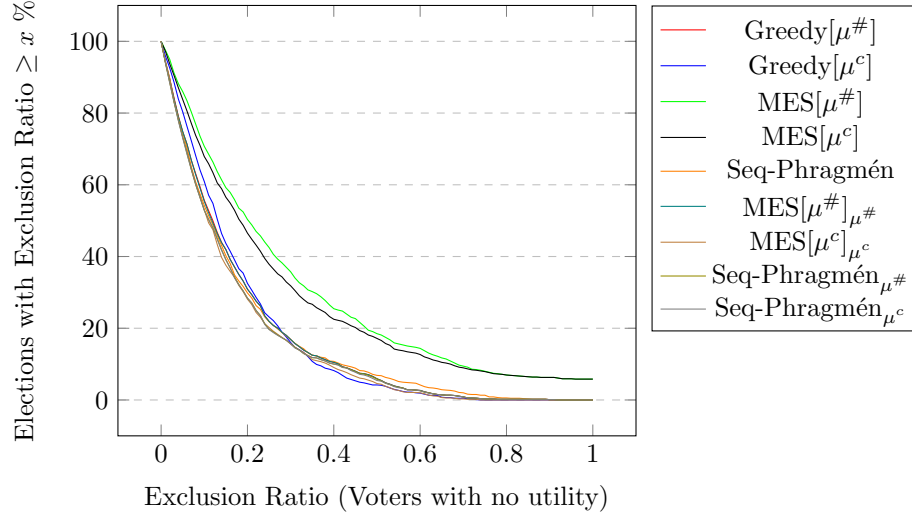


Figure 10: Exclusion Ratio

Most rules perform similarly, with the notable exception of MES[$\mu^\#$] and MES[μ^c]. Looking at the rightmost point of the curve, in 7% of elections every single voter receives zero utility. This is largely driven by 61 elections in which both MES rules produce an empty outcome, occurring because no project attracted enough supporting voters to collectively cover its cost.

8 Discussion

We now discuss the main findings from our experimental results. Our focus is on what the results suggest about the practical behavior of the considered voting rules and proportionality axioms. In particular, we discuss the feasibility of verifying EJR in practice, the severity of the observed violations, the performance of different voting rules, and the importance of completion rules.

8.1 Verifying EJR is W[1]-hard and yet doable in practice

Theorem 9 tells us that verifying EJR is W[1]-hard parameterized with respect to the largest cohesive group size κ , meaning we cannot hope for a running time that confines the hardness to κ alone. In our algorithm, κ is the largest project set size among all cohesive groups checked. As seen in Figure 8, this never exceeds 11 for any election. Even so, a worst-case running time of $O(m^\kappa \cdot n\kappa)$ would be infeasible for values of m approaching 200. This upper bound is nevertheless close to optimal given the W[1]-hardness of the problem.

And yet, as shown in Figure 3, our algorithm is not only feasible but fast – the average time to verify EJR on an election is less than 0.05s. The reason lies in the effectiveness of pruning. Figure 9 shows the number of cohesive sets actually checked in the hardest observed election, where $k = 11$ and $m = 21$. At layer size 6, we check approximately 10,000 groups rather than the worst-case $\binom{21}{6} = 54,264$. The difference becomes far more dramatic at higher layers: at layer 11, we check just 3 cohesive groups instead of the worst-case $\binom{21}{11} = 352,716$.

In practice, pruning reduces the search space so aggressively that our algorithm shatters the $O(m^k \cdot mn)$ bound. This does not contradict the W[1]-hardness result: that result concerns worst-case instances over all possible inputs, whereas our findings reflect the structure of real-world participatory budgeting elections, which appear to admit far fewer cohesive groups than the worst case. On adversarially constructed inputs, the problem remains infeasible.

The reason pruning works so well in practice comes down to a structural tension in the parameters. Increasing the number of projects, m , while holding the number of voters fixed means each voter’s approvals are spread more thinly across the project space. This reduces the overlap between voters, making it harder to form large cohesive groups, and so κ shrinks as m grows. Conversely, elections in which large cohesive groups can form tend to have few projects, keeping m small. The two quantities that drive the worst-case bound, m and κ , thus pull in opposite directions in real-world elections. Therefore, you rarely get both a large project space and large cohesive groups simultaneously. This is why the hardest observed case had $\kappa = 11$ with only $m = 21$ projects, and why adversarial instances that maximize both simultaneously do not arise in practice.

8.2 Violation degrees

Looking at the amount of violations is the most direct way to measure proportionality performance across voting rules, but the degree of those violations adds an important dimension to the analysis: How large is the violation. Table 5 presents these findings for μ^c -EJR, which is the most interesting case to inspect given its considerably higher violation count overall.

The most striking observation is the clear outlier: $\text{GREEDY}[\mu^c]$ exhibits a mean violation degree of 0.9067 with a median of 1.0, indicating that its violations are not merely frequent but essentially maximal – we will return to this in detail later. Setting that aside, a consistent pattern emerges among the remaining rules. Those that already produce few violations – Sequential Phragmén, $\text{MES}[\mu^c]$, and $\text{GREEDY}[\mu^\#]$ – also tend to produce the smallest ones. Low violation count and low violation degree appear to go hand in hand, suggesting these rules are not just lucky in avoiding violations but are structurally closer to satisfying μ^c -EJR. Notably, $\text{MES}[\mu^\#]$ is a mild exception: despite a relatively high violation count (388), its mean degree of 0.31 is only moderate.

Table 4: $\text{MES}[\mu^\#]$ has no μ^c -EJR-1 violations despite its many μ^c -EJR violations, while $\text{GREEDY}[\mu^c]$ violates μ^c -EJR-1 almost as often as it violates μ^c -EJR. When violations are small in degree – as with $\text{MES}[\mu^\#]$ – the relaxed axiom absorbs them; when they are near-maximal, as with $\text{GREEDY}[\mu^c]$, even the relaxation makes little difference.

Figure 7 shows that computing these violation degrees is tractable in practice. The intuition behind this is visible in Figure 9, which illustrates the general tendency for the number of cohesive groups checked per layer to decrease rapidly. Once the first violation is found, the search space has already been pruned so aggressively that exhaustively checking the remainder adds little overhead – again demonstrating the practical strength of the pruning strategy.

8.3 The Greedy Voting Rule in Practice

As discussed in Section 5.1.2, $\text{GREEDY}[\mu^c]$ is the voting rule most commonly used in practice. This is somewhat surprising in light of our experimental results, where $\text{GREEDY}[\mu^c]$ performs poorly on all proportionality measures. Table 4 shows that $\text{GREEDY}[\mu^c]$ satisfies μ^c -EJR in only 68.6% of the elections, which is the lowest satisfaction rate among the exhaustive rules. Notably, it is even significantly lower than $\text{GREEDY}[\mu^\#]$ which sits at 94.8%. The only rule with a lower rate is the non-exhaustive $\text{MES}[\mu^\#]$, which satisfies μ^c -EJR in only 63.0% of the elections, however with a much smaller violation degree as seen in Table 5.

The severity of these violations is also notable. In Figure 5, the μ^c -EJR violations produced by $\text{GREEDY}[\mu^c]$ have much higher violation degrees than those produced by the other rules. Across the 329 violation instances, the mean violation degree is approximately 90%. This means that, in the worst represented cohesive group, even the best represented voter receives only about 10% of the

utility required by μ^c -EJR. Moreover, $Q_1 = 1.0$, which indicates that in at least three-quarters of the violations, no voter in the relevant cohesive group receives any utility from the selected outcome. By contrast, the other voting rules have violation degrees below 44%, suggesting much less severe violations. A similar pattern can be observed for $\mu^\#$ -EJR in Figure 6.

This is unsurprising given the design of $\text{GREEDY}[\mu^c]$: it greedily selects projects by total approval votes, which is precisely the behavior proportionality axioms are designed to guard against. It will reliably fund the large majority-backed projects while leaving cohesive minorities – and their local bike paths and playgrounds – entirely unserved. Unlike proportional rules, it has no mechanism ensuring that a cohesive group receives even a single project it commonly supports.

This trade-off is visible in Table 7. $\text{GREEDY}[\mu^c]$ produces the highest average total utility among the considered rules. $\text{MES}[\mu^c]_{\mu^c}$ is the second-best rule by this measure, achieving 86.2% of the average utility achieved by $\text{GREEDY}[\mu^c]$. The lowest average utilities are obtained by the two non-exhaustive MES variants at 69.9% for $\text{MES}[\mu^\#]$ and 70.8% for $\text{MES}[\mu^c]$, which is consistent with their low average budget usage of 43.1% and 49.2%, respectively. Thus, $\text{GREEDY}[\mu^c]$ presents a clear utilitarian-proportionality trade-off: its strong welfare performance comes at the cost of systematic under-representation of cohesive minority groups.

8.4 The Role of Completion Rules

As discussed in Section 5, some voting rules, including MES and SEQPHRAG, may return non-exhaustive outcomes. This is also visible in our experiments in Table 7. The non-exhaustive variants have the lowest average budget usage: SEQPHRAG uses 81.0% of the budget on average, while MES uses only 43.1% and 49.2% for $\mu^\#$ and μ^c , respectively. The same pattern is visible for relative utility. $\text{MES}[\mu^\#]$ achieves only 69.9% of the utility obtained by $\text{GREEDY}[\mu^\#]$ on average, and similarly, $\text{MES}[\mu^c]$ achieves only 70.8% of the utility achieved by $\text{GREEDY}[\mu^c]$ on average. The effect is especially clear for the two non-completed MES variants, which are exhaustive in only 7.5% and 7.7% of the elections. Thus, the low budget usage and low relative utility are not merely caused by a few outlier instances but appear to be systematic features of MES without completion on this dataset. From a practical perspective, this is undesirable, since PB elections are typically held with a fixed public budget, and leaving a large share of that budget unspent is difficult to justify. For this reason, completion rules are important in practical applications, especially for rules such as MES.

Table 7 shows that utilitarian greedy completion has a substantial effect on both budget usage and welfare. For $\text{MES}[\mu^\#]_{\mu^\#}$ and $\text{MES}[\mu^c]_{\mu^c}$, average budget usage increases to 86.8% and 90.7%, respectively. Moreover, both completed MES variants are exhaustive in 100.0% of the elections, as expected when using greedy completion rules. The completed MES variants also achieve much higher utility than their non-exhaustive counterparts. Under cardinality-based utility, $\text{MES}[\mu^\#]_{\mu^\#}$

obtains the same average utility as $\text{GREEDY}[\mu^\#]$. Under cost-based utility, $\text{MES}[\mu^c]_{\mu^c}$ achieves 86.2% of the average utility obtained by $\text{GREEDY}[\mu^c]$.

These results show that completion rules are not merely an implementation detail. They can significantly change the utilitarian performance of a voting rule, especially for rules such as MES that might otherwise leave a large part of the budget unused. At the same time, completion should be interpreted carefully. Since the additional projects are selected after the original rule has terminated, the completed outcome should not always be understood as satisfying all structural properties of the original rule. For example, greedy completion may break priceability. However, for monotone satisfaction-based proportionality axioms such as EJR, adding projects cannot make voters less represented. Thus, completion offers a practically useful way of improving budget usage and welfare while retaining the proportionality already achieved by the initial outcome.

8.5 Why EJR-x and EJR-1 Seem Indistinguishable in Practice

Table 4 shows that μ^c -EJR-1 and μ^c -EJR-x yield identical violation counts across all elections in our dataset. We now explain why this is to be expected.

For EJR-x and EJR-1 to differ, there must exist an election where EJR-x is violated but EJR-1 is not. This requires two conditions to hold simultaneously, both of which turn out to be unlikely in practice.

The first condition is that the witnessing T -cohesive group N' must satisfy $|T \setminus W| \geq 2$, i.e., at least two projects in T are missing from the outcome. As shown in Figure 8, the mean size of the largest layer checked is around 3, meaning $|T|$ is small on average and never exceeds 11. Given that the average outcome size is $|W_{\text{avg}}| \approx 14$ (with a maximum of 135), having two or more projects from T absent from W is already unlikely.

The second condition is that the missing projects must create a split: at least one $p \in T \setminus W$ must satisfy $\mu_i(W \cup p) > \mu_i(T)$ while at least one other does not. For μ^c we need $c((W \cap A_i) \cup p) > c(T)$ for some but not all $p \in T \setminus W$. If this held for all missing projects, EJR-x would not be violated either; if it held for none, EJR-1 would be violated as well. Only the intermediate case separates them, which is a narrow condition unlikely to arise.

Since both conditions must hold simultaneously, it is unsurprising that the two notions are empirically indistinguishable.

8.6 Practical Takeaways

The experiments suggest that there is no single best rule independent of the desired objective. If the only goal is to maximize total cost-based utility, then $\text{GREEDY}[\mu^c]$ performs best. However, it also performs worst from a proportionality perspective, with frequent and often maximal EJR violations. Thus, if proportionality is a concern, it should be avoided.

Overall, the strongest practical choice appears to be $\text{MES}[\mu^c]_{\mu^c}$. It combines strong proportionality performance with high budget usage and reasonably high utility. Sequential Phragmén and its completed variants are also robust from a proportionality perspective, but achieve somewhat lower cost-based utility. By contrast, non-completed MES is difficult to recommend in practice, since it often leaves a large share of the budget unused.

A central takeaway is therefore that completion rules matter. They are not just a technical detail, but can substantially improve budget usage and welfare while preserving EJR-type guarantees, since adding projects cannot make voters worse off. In practical applications, completed proportional rules therefore seem preferable to both plain MES and purely greedy rules.

Finally, the results show that EJR verification itself is practically feasible. Even though the problem is theoretically hard in the worst case, our algorithm is fast on real-world Pabulib elections, and computing violation degrees adds little overhead. This makes EJR verification useful not only as a theoretical tool, but also as a practical diagnostic for comparing outcomes and identifying severe representation failures.

9 Future Work

Completion rules. Completion rules are an essential component of MES as a voting rule, yet we have only experimented with basic greedy completion in this work. One direction for future work is to explore more sophisticated completion rules, and we believe our violation degrees provide a natural foundation for such an approach. Specifically, when MES produces a non-exhaustive outcome W that violates EJR, the following procedure could be used to complete it in a proportionality-aware manner:

1. Run MES to obtain outcome W .
2. If W is non-exhaustive and violates EJR, run our EJR- ϕ algorithm to compute the largest violation degree ϕ_i for each voter i . Reweight voter utilities as $\mu_i \leftarrow \phi_i \cdot \mu_i$, so that satisfied voters contribute zero weight and the most severely underrepresented voters are prioritized. Apply a completion rule – such as utilitarian greedy – using these reweighted utilities to obtain an exhaustive outcome W' .

The key motivation here is that standard greedy completion ignores the proportionality structure already established by MES, whereas this approach directly targets the voters most harmed by the incomplete outcome. Evaluating whether this leads to stronger proportionality guarantees in theory and in practice remains an open question.

Stronger proportionality axioms. A second direction concerns stronger proportionality axioms. EJR is, perhaps surprisingly, satisfied in the vast majority of practical instances we tested, which raises the question of whether it is the right benchmark. Full Justified Representation (FJR) is a strictly stronger axiom that may better discriminate between outcomes in practice, and developing efficient algorithms for verifying or approximating FJR would be a valuable contribution, particularly since our hardness results suggest that exact verification of proportionality properties is generally intractable. A further strengthening, Strong-EJR, requires every member of a cohesive group to be satisfied rather than just one; however, since it is not satisfiable in general, it has attracted limited theoretical interest and falls outside the scope of this thesis. We note, though, that our algorithm extends naturally to detecting violations of Strong-EJR (see Appendix A.3.1).

Examine new utility functions. Another direction is to study proportionality under a broader class of utility functions. In this thesis, the experiments focus mainly on cardinality-based utility $\mu^\#$ and cost-based utility μ^c , but both the theoretical results and the empirical conclusions may depend on this choice. A natural next step is therefore to extend the relevant proofs from these specific utilities to arbitrary additive utility functions, identifying which arguments rely only on additivity and monotonicity and which rely on the special structure of $\mu^\#$ or μ^c . Beyond additive utilities, it would also be interesting to examine utility functions such as DNS, where voter satisfaction is not

simply the sum of approved project values. Testing the considered rules under such utility functions could show whether the observed proportionality patterns are robust, or whether they are specific to the two utility models studied here.

Cardinal elections. A further direction is to extend the verification approach from approval-based PB elections to cardinal elections, where voters assign explicit scores or utilities to projects. This would require adapting the notion of cohesion, since groups would no longer be characterized simply by a common set of approved projects. Instead, one would need to identify when a group has sufficiently high shared utility for some project set T , and then check whether the outcome gives at least one voter in the group comparable satisfaction. Such an extension would make it possible to test EJR-style proportionality in richer voting models, but it may also change both the algorithmic structure and the hardness results. Investigating which pruning techniques still apply, and whether similar practical running times can be achieved, would therefore be a natural continuation of this work.

10 Conclusion

This thesis studied proportionality in approval-based participatory budgeting from both a theoretical and practical perspective. The starting point was the tension between welfare and fairness: a voting rule may allocate the budget efficiently in terms of total utility while still leaving cohesive minority groups underrepresented. Proportionality axioms such as EJR, EJR- α , EJR-1, PJR, and FJR provide a formal language for describing this problem, but they also raise a computational question: can we actually verify whether real PB outcomes satisfy such guarantees?

On the theoretical side, we presented the relevant proportionality axioms and examined their relationships under different utility functions. We also studied the computational difficulty of verifying proportionality. These results show that proportionality verification is inherently hard in the worst case, and that this hardness cannot simply be avoided by choosing a better implementation. In particular, the $W[1]$ -hardness result for EJR verification helps explain why the problem should be expected to be difficult on adversarial instances.

The main algorithmic contribution of the thesis is an EJR verification algorithm based on searching over cohesive project sets while pruning large parts of the search space. Although the worst-case running time remains exponential, the algorithm performs very well on real-world PB elections. This suggests that real PB instances have much more structure than worst-case examples: large cohesive groups are rare, and the search space collapses quickly in practice. The same approach also makes it feasible to compute violation degrees, giving a more informative measure than simply recording whether EJR is satisfied or violated.

The experimental results on Pabulib show that proportionality guarantees do matter in practice. Most proportional rules perform well, while $\text{GREEDY}[\mu^c]$ is a clear outlier. It achieves high cost-based utility, but it also produces frequent and severe EJR violations, often leaving the witnessing cohesive group completely unrepresented. This illustrates the central welfare-proportionality trade-off in participatory budgeting: maximizing total utility alone can come at the cost of systematic minority exclusion.

Among the rules tested, the best overall practical performance is obtained by completed proportional rules, especially $\text{MES}[\mu^c]_{\mu^c}$. It combines strong proportionality performance with high budget usage and reasonably high welfare. The experiments also show that completion rules are not merely a technical detail. Without completion, MES often leaves a large part of the budget unused; with completion, it becomes much more attractive as a practical rule while preserving the EJR-type representation already achieved by the initial outcome.

Overall, the thesis shows that proportionality in participatory budgeting is both theoretically subtle and practically measurable. Worst-case verification is hard, but real-world verification is feasible. Greedy rules can perform well in terms of welfare, but may fail badly in terms of representation.

Completed proportional rules offer a more balanced alternative. These findings suggest that proportionality verification can serve not only as a theoretical tool, but also as a practical diagnostic for evaluating and comparing PB outcomes before they are used in real decision-making.

References

- Rakesh Agrawal and Ramakrishnan Srikant. Fast algorithms for mining association rules in large databases. In Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo, editors, *VLDB'94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile*, pages 487–499. Morgan Kaufmann, 1994. URL <http://www.vldb.org/conf/1994/P487.PDF>.
- Haris Aziz and Nisarg Shah. Participatory budgeting: Models and approaches. *CoRR*, abs/2003.00606, 2020. URL <https://arxiv.org/abs/2003.00606>.
- Haris Aziz, Markus Brill, Vincent Conitzer, Edith Elkind, Rupert Freeman, and Toby Walsh. Justified representation in approval-based committee voting. *Social Choice and Welfare*, 48(2):461–485, 2017. doi: 10.1007/S00355-016-1019-3. URL <https://doi.org/10.1007/s00355-016-1019-3>.
- Haris Aziz, Edith Elkind, Shenwei Huang, Martin Lackner, Luis Sanchez-Fernandez, and Piotr Skowron. On the complexity of extended and proportional justified representation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018a. doi: 10.1609/aaai.v32i1.11478. URL <https://ojs.aaai.org/index.php/AAAI/article/view/11478>.
- Haris Aziz, Barton E. Lee, and Nimrod Talmon. Proportionally representative participatory budgeting: Axioms and algorithms. In Elisabeth André, Sven Koenig, Mehdi Dastani, and Gita Sukthankar, editors, *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS 2018, Stockholm, Sweden, July 10-15, 2018*, pages 23–31. International Foundation for Autonomous Agents and Multiagent Systems Richland, SC, USA / ACM, 2018b. URL <http://dl.acm.org/citation.cfm?id=3237394>.
- Markus Brill, Rupert Freeman, Svante Janson, and Martin Lackner. Phragmén’s voting methods and justified representation. In Satinder Singh and Shaul Markovitch, editors, *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence, February 4-9, 2017, San Francisco, California, USA*, pages 406–413. AAAI Press, 2017. doi: 10.1609/AAAI.V31I1.10598. URL <https://doi.org/10.1609/aaai.v31i1.10598>.
- Markus Brill, Stefan Forster, Martin Lackner, Jan Maly, and Jannik Peters. Proportionality in approval-based participatory budgeting. In Brian Williams, Yiling Chen, and Jennifer Neville, editors, *Thirty-Seventh AAAI Conference on Artificial Intelligence, AAAI 2023, Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence, IAAI 2023, Thirteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023*, pages 5524–5531. AAAI Press, 2023. doi: 10.1609/AAAI.V37I5.25686. URL <https://doi.org/10.1609/aaai.v37i5.25686>.

- George B. Dantzig. Discrete-variable extremum problems. *Operations Research*, 5(2):266–277, 1957. ISSN 0030364X, 15265463. URL <http://www.jstor.org/stable/167356>.
- Rodney G Downey and Michael R Fellows. *Fundamentals of Parameterized Complexity*. Springer, 2013.
- Piotr Faliszewski, Jaroslaw Flis, Dominik Peters, Grzegorz Pierczynski, Piotr Skowron, Dariusz Stolicki, Stanislaw Szufa, and Nimrod Talmon. Participatory budgeting: Data, tools and analysis. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*, pages 2667–2674. ijcai.org, 2023. doi: 10.24963/IJCAI.2023/297. URL <https://doi.org/10.24963/ijcai.2023/297>.
- M. R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979. ISBN 0-7167-1044-7.
- Russell Impagliazzo and Ramamohan Paturi. On the complexity of k-sat. *Journal of Computer and System Sciences*, 62(2):367–375, 2001.
- Svante Janson. Phragmén’s and thiele’s election methods. *CoRR*, abs/1611.08826, 2016. URL <http://arxiv.org/abs/1611.08826>.
- Yusuf Hakan Kalayci, Jiasen Liu, and David Kempe. Full proportional justified representation, 2025. URL <https://arxiv.org/abs/2501.12015>.
- Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.
- Hans Kellerer, Ulrich Pferschy, and David Pisinger. *Knapsack problems*. Springer, 2004. ISBN 978-3-540-40286-2. doi: 10.1007/978-3-540-24777-7. URL <https://doi.org/10.1007/978-3-540-24777-7>.
- Bingkai Lin. The parameterized complexity of k-biclique, 2019. URL <https://arxiv.org/abs/1406.3700>.
- Maike Los, Zoé Christoff, and Davide Grossi. Proportional budget allocations: Towards a systematization. In Luc De Raedt, editor, *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, pages 398–404. ijcai.org, 2022. doi: 10.24963/IJCAI.2022/57. URL <https://doi.org/10.24963/ijcai.2022/57>.
- Pabulib. Pabulib: Participatory budgeting library, 2026. URL <https://pabulib.org/>.
- Georgios Papasotiropoulos, Seyedeh Zeinab Pishbin, Oskar Skibski, Piotr Skowron, and Tomasz Was. Method of equal shares with bounded overspending, 2025. URL <https://arxiv.org/abs/2409.15005>.
- Dominik Peters and Piotr Skowron. Proportionality and the limits of welfarism. In Péter Biró, Jason D. Hartline, Michael Ostrovsky, and Ariel D. Procaccia, editors, *EC ’20: The 21st ACM*

Conference on Economics and Computation, Virtual Event, Hungary, July 13-17, 2020, pages 793–794. ACM, 2020. doi: 10.1145/3391403.3399465. URL <https://doi.org/10.1145/3391403.3399465>.

Dominik Peters, Grzegorz Pierczynski, and Piotr Skowron. Proportional participatory budgeting with additive utilities. In Marc’Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 12726–12737, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/69f8ea31de0c00502b2ae571fbab1f95-Abstract.html>.

Simon Rey and Jan Maly. The (computational) social choice take on indivisible participatory budgeting. *CoRR*, abs/2303.00621, 2023. doi: 10.48550/ARXIV.2303.00621. URL <https://doi.org/10.48550/arXiv.2303.00621>.

Nimrod Talmon and Piotr Faliszewski. A framework for approval-based budgeting methods. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*, pages 2181–2188. AAAI Press, 2019. doi: 10.1609/AAAI.V33I01.33012181. URL <https://doi.org/10.1609/aaai.v33i01.33012181>.

A Appendix

A.1 Alternative Way to Find Unsatisfied Voters

An alternative way to find $U(T)$, would have been to compute the utility as: $\mu_i(T) = \mu_i(T_1) + \mu_i(T_2 \setminus T_1)$. Notice that $|T_2 \setminus T_1| = 1$ and that $\mu_i(T_1)$ has been computed before. Meaning that utilizing the additive utilities would have been simply adding $\mu_i(T_2 \setminus T_1)$ to $\mu_i(T_1)$, in $O(1)$ time. Finding $U(T)$ would still take $O(n)$ time, as it still requires looping over $N^*(T)$ to find those with more utility than their $\mu_i(W)$

A slightly worse optimization, only for approval elections, would be to note that $\mu_i(T)$ is the same for all $i \in N^*(T)$, since they agree on T . That would give $O(n + m)$.

A.2 Algorithms

Algorithm 7: Pseudo-code of Build First Layer.

Input: Approval profile A , Project set P

Output: The $(T, N^*(T))$ list for $|T| = 1$

```

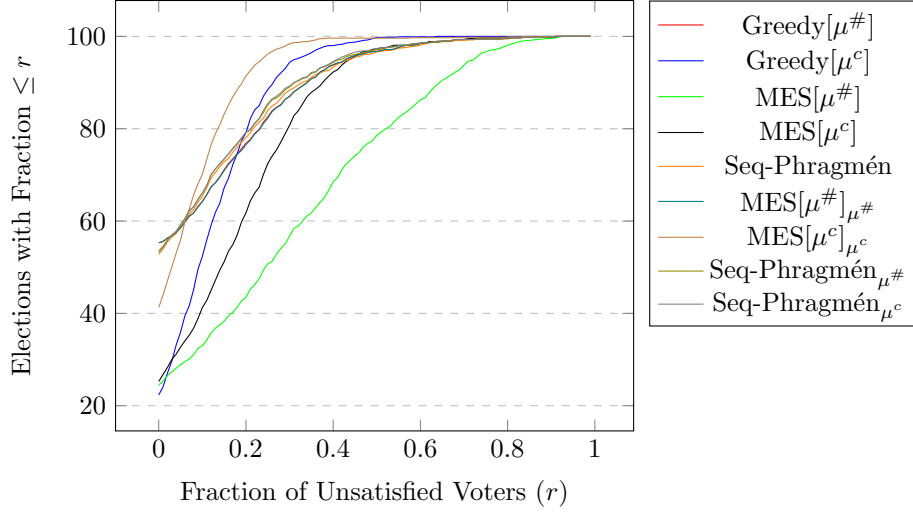
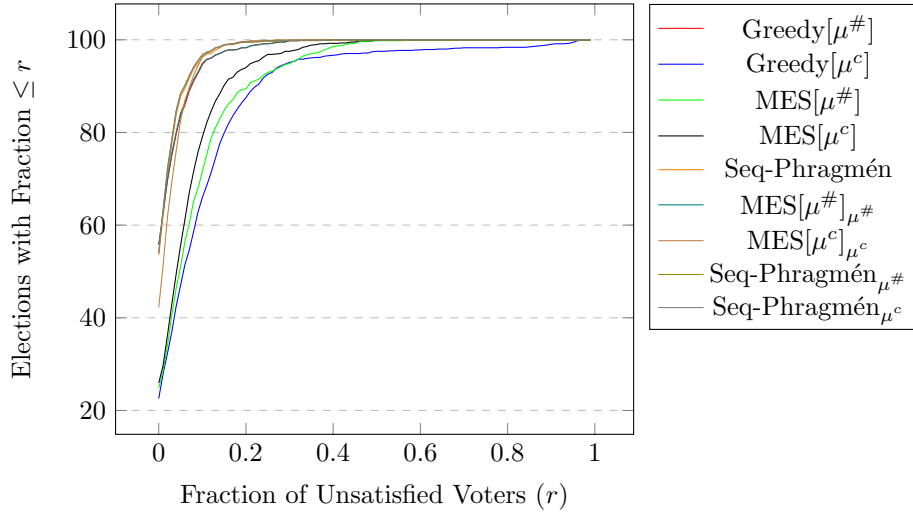
1 Function build_first_layer ( $A, P$ )
2   layer  $\leftarrow$  empty list
3   foreach  $p \in P$  do
4     Append ( $[p, \emptyset]$ ) into layer
5   foreach ( $voter, approval\_set$ )  $\in A$  do
6     foreach  $p \in approval\_set$  do
7       Insert voter into  $N^*(T)$  for layer[p]
8   return layer
    
```

A.3 Experiment Data

This includes some of the graphs we decided not to include directly in the report.

A.3.1 Unsatisfied voters

Let $\mathcal{U} = \bigcup_T U(T)$ for all T that are T-cohesive. I.e., all voters that are ever unsatisfied. Notice that this includes voters who are unsatisfied in a T-cohesive group that does not contain a violation. Here we plot $\frac{|\mathcal{U}|}{|N|}$ using μ -EJR- ϕ


 Figure 11: Unsatisfied Voters CDF EJR[μ^c]

 Figure 12: Unsatisfied Voters CDF EJR[$\mu^\#$]

If we focus on $r = 0$, then this equates to Strong-EJR, in which there is a violation if any are unsatisfied, as opposed to if all are.

A.3.2 Theoretical Runtime

We write that the running time is $O(s \cdot n\kappa)$, as such plotting $\frac{\text{time}}{n\kappa}$ over s should give us a straight line, for s being the exact number of T-cohesive sets checked.

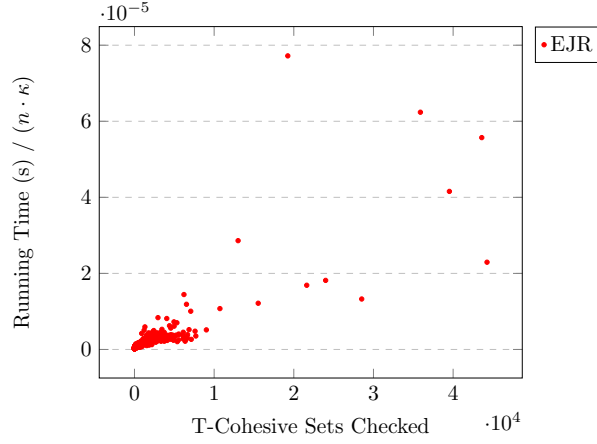


Figure 13: Running time vs. Theoretical time

The expected running time is shown in Figure 13. However, based on the plot, we can't say much. We attribute this, among other things, to the large upper bound on n .

A.3.3 Violation Degree vs. Vote Length

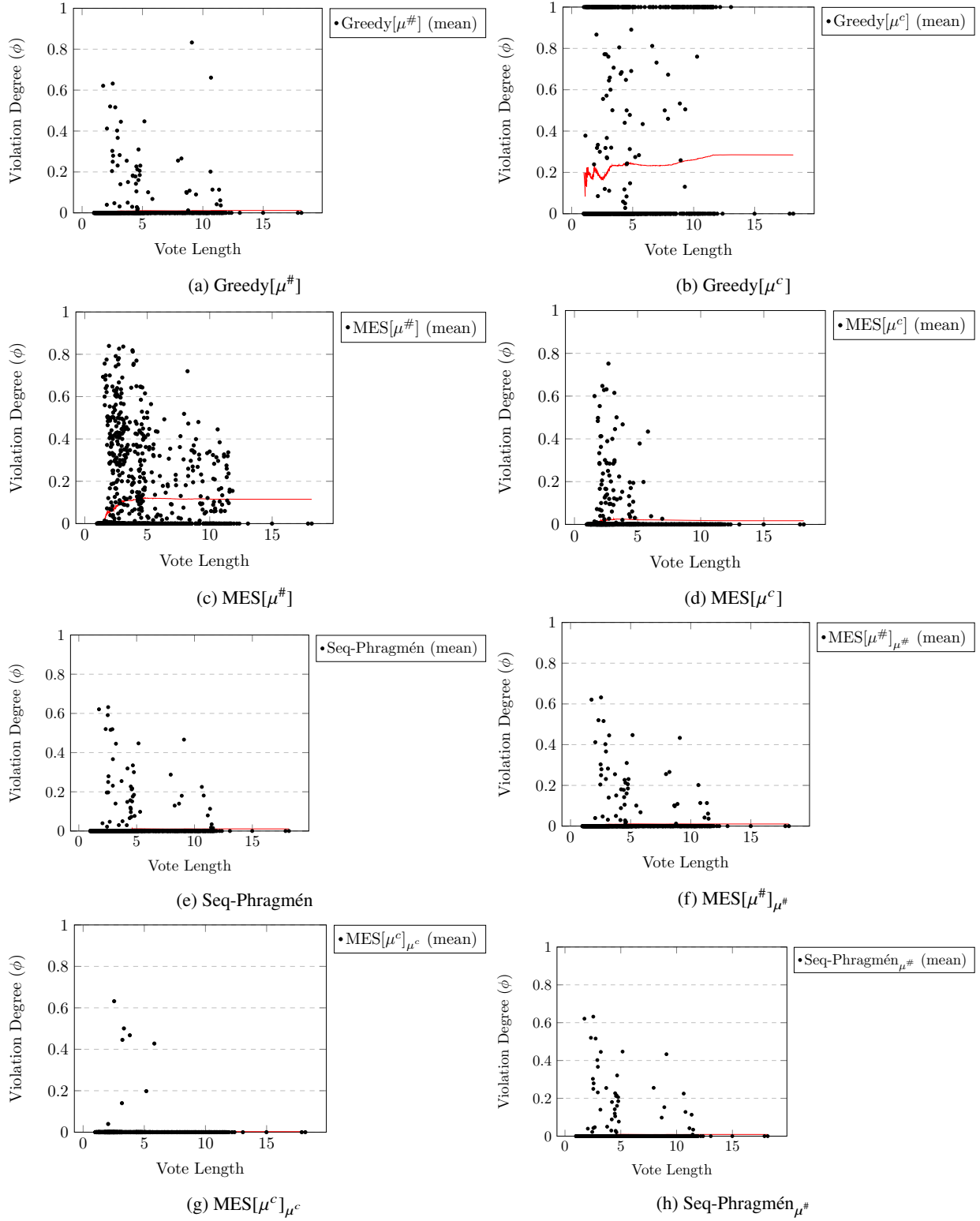
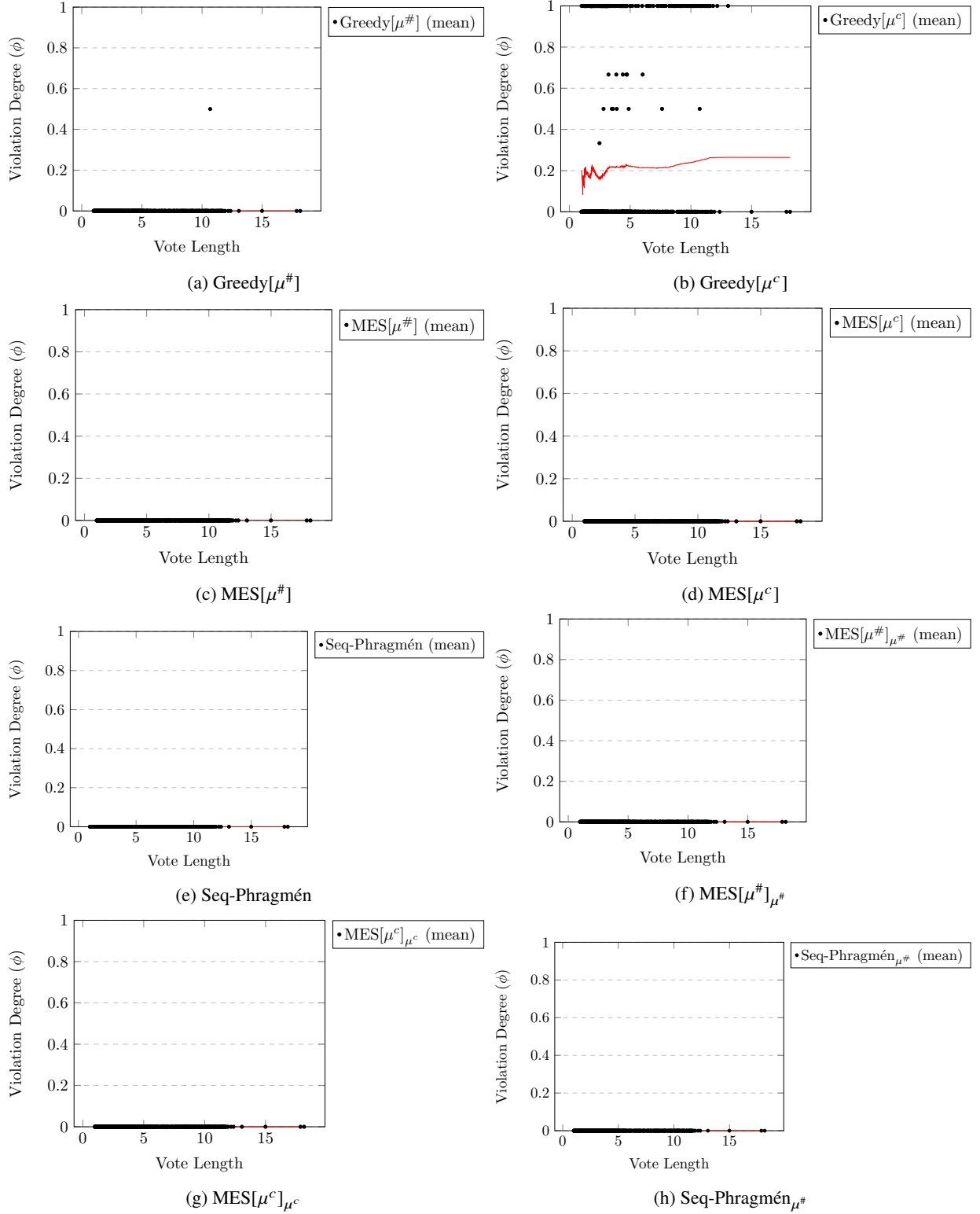
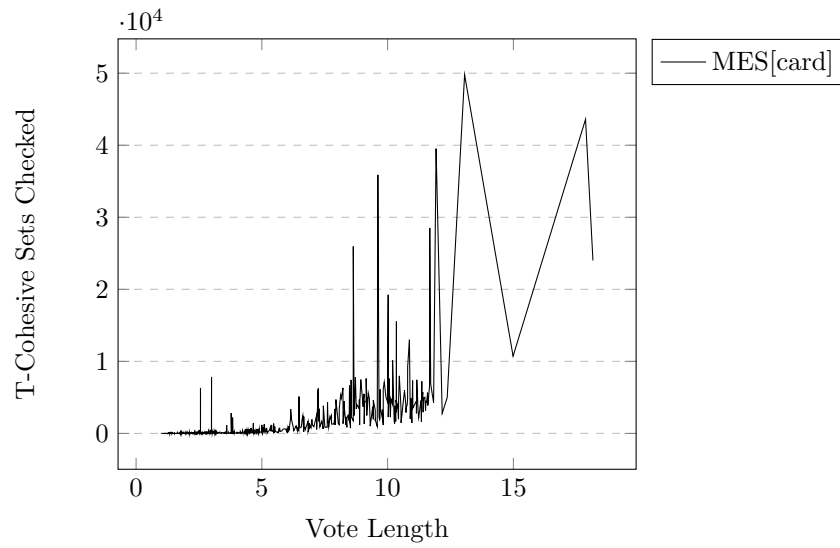


Figure 14: μ^c -EJR Violation Degree vs. Vote Length

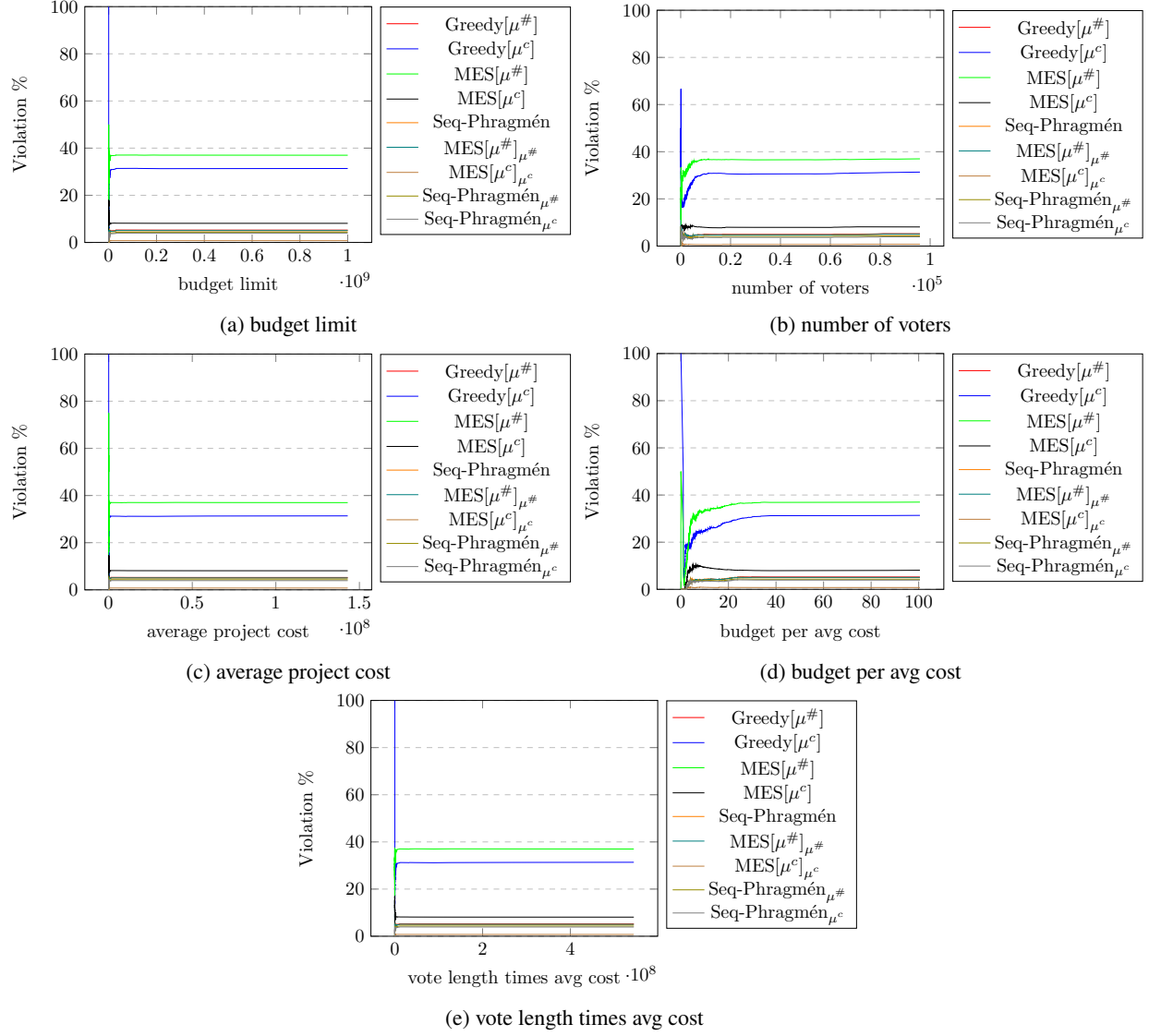

 Figure 15: $\mu^\#$ -EJR Violation Degree vs Vote Length

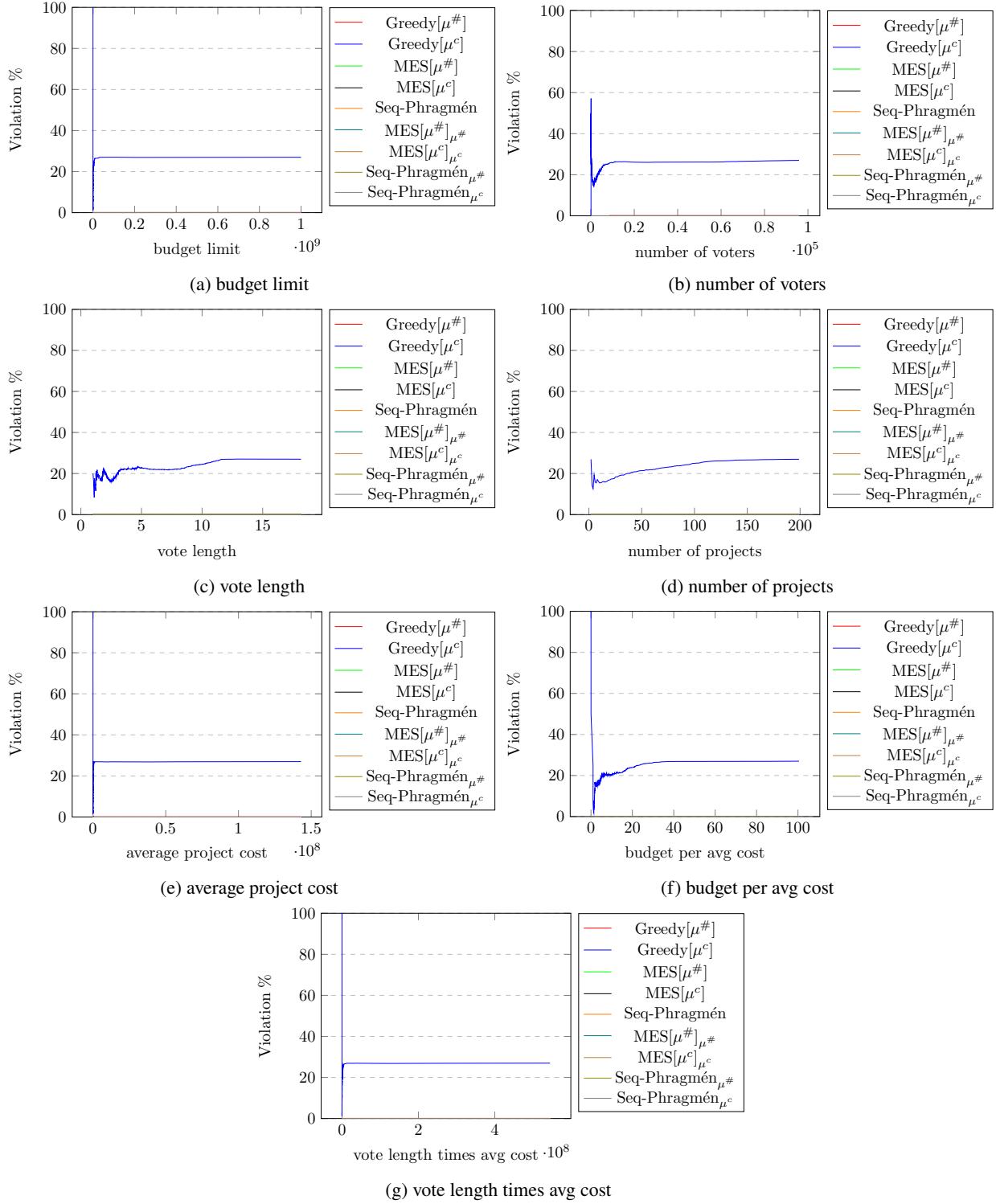
A.3.4 T cohesive sets checked



A.3.5 Violation Percentage Plots

Here we have all the plots for violations based on features.


 Figure 16: μ^c -EJR Violation Rate vs Features


 Figure 17: $\mu^\#$ -EJR Violation Rate vs Features